

Python 3 — Biblioteca Estándar

Tomo 6 de 6

Contenido de este tomo:

1. codeop— Compile Python code
2. Importing Modules
3. zipimport— Import modules from Zip archives
4. pkgutil— Package extension utility
5. modulefinder— Find modules used by a script
6. runpy— Locating and executing Python modules
7. importlib— The implementation of import
8. importlib.resources— Package resource reading, opening and access
9. importlib.resources.abc— Abstract base classes for resources
10. importlib.metadata— Accessing package metadata
11. The initialization of sys.path module search path
12. Python Language Services
13. ast— Abstract syntax trees
14. symtable— Access to the compiler's symbol tables
15. token— Constants used with Python parse trees
16. keyword— Testing for Python keywords
17. tokenize— Tokenizer for Python source
18. tabnanny— Detection of ambiguous indentation
19. pycbr— Python module browser support
20. py_compile— Compile Python source files
21. compileall— Byte-compile Python libraries
22. dis— Disassembler for Python bytecode
23. pickletools— Tools for pickle developers
24. MS Windows Specific Services
25. msvcrt— Useful routines from the MS VC++ runtime

- 26. winreg— Windows registry access
- 27. winsound— Sound-playing interface for Windows
- 28. Unix-specific services
- 29. shlex— Simple lexical analysis
- 30. posix— The most common POSIX system calls
- 31. pwd— The password database
- 32. grp— The group database
- 33. termios— POSIX style tty control
- 34. tty— Terminal control functions
- 35. pty— Pseudo-terminal utilities
- 36. fcntl— Thefcntl and ioctl system calls
- 37. resource— Resource usage information
- 38. syslog— Unix syslog library routines
- 39. Modules command-line interface (CLI)
- 40. Superseded Modules
- 41. getopt— C-style parser for command line options
- 42. Removed Modules
- 43. Security Considerations

codeop— Compile Python code

Source code: Lib/codeop.py

The `codeop` module provides utilities upon which the Python read-eval-print loop can be emulated, as is done in the `code` module. As a result, you probably don't want to use the module directly; if you want to include such a loop in your program you probably want to use the `code` module instead.

There are two parts to this job:

- Being able to tell if a line of input completes a Python statement: in short, telling whether to print '>>>' or '...' next.
- Remembering which future statements the user has entered, so subsequent input can be compiled with these in effect.

The `codeop` module provides a way of doing each of these things, and a way of doing them both.

To do just the former:

`codeop.compile_command(source,filename='<input>',symbol='single')`

Tries to compile `source`, which should be a string of Python code and return a code object if `source` is valid Python code. In that case, the `filename` attribute of the code object will be `filename`, which defaults to '`<input>`'. Returns `None` if `source` is not valid Python code, but is a prefix of valid Python code. If there is a problem with `source`, an exception will be raised. `SyntaxError` is raised if there is invalid Python syntax, and `OverflowError` or `ValueError` if there is an invalid literal. The `symbol` argument det

`class codeop.Compile`

Instances of this class have `__call__()` methods identical in signature to the built-in function `compile()`, but with the difference that if the instance compiles program text containing a `__future__` statement, the instance 'remembers' and compiles all subsequent program texts with the statement in force.

`class codeop.CommandCompiler`

Instances of this class have `__call__()` methods identical in signature to `compile_command()`; the difference is that if the instance compiles program text containing a `__future__` statement, the instance 'remembers' and compiles all subsequent program texts with the statement in force.

Importing Modules

The modules described in this chapter provide new ways to import other Python modules and hooks for customizing the import process.

The full list of modules described in this chapter is:

- `zipimport`— Import modules from Zip archives
`zipimporter` Objects Examples
- `pkgutil`— Package extension utility
- `modulefinder`— Find modules used by a script
Example usage of `ModuleFinder`
- `runpy`— Locating and executing Python modules
- `importlib`— The implementation of import
Introduction Functions
`importlib.abc`— Abstract base classes related to import
`importlib.machinery`— Importers and path hooks
`importlib.util`— Utility code for importers
Examples
Importing programmatically
Checking if a module can be imported
Importing a source file directly
Implementing lazy imports
Setting up an importer
Approximating `importlib.import_module()`
- `importlib.resources`— Package resource reading, opening and access
Functional API
- `importlib.resources.abc`— Abstract base classes for resources
- `importlib.metadata`— Accessing package metadata
Overview
Functional API
Entry points
Distribution metadata
Distribution versions
Distribution files
Distribution requirements
Mapping import to distribution packages
Distributions
Distribution Discovery
Implementing Custom Providers
Example
- The initialization of `sys.path`
module search path
Virtual Environments
`_pth` files
Embedded Python

zipimport— Import modules from Zip archives

Source code: `Lib/zipimport.py`

This module adds the ability to import Python modules (`*.py`, `*.pyc`) and packages from ZIP-format archives. It is usually not needed to use the `zipimport` module explicitly; it is automatically used by the built-in import mechanism for `sys.path` items that are paths to ZIP archives.

Typically, `sys.path` is a list of directory names as strings. This module also allows an item of `sys.path` to be a string naming a ZIP file archive. The ZIP archive can contain a subdirectory structure to support package imports, and a path within the archive can be specified to only import from a subdirectory. For example, the `pathexample.zip/lib/` would only import from the `lib/` subdirectory within the archive.

Any files may be present in the ZIP archive, but importers are only invoked for `.py` and `.pyc` files. ZIP import of dynamic modules (`.pyd`, `.so`) is disallowed. Note that if an archive only contains `.py` files, Python will not attempt to modify the archive by adding the corresponding `.pyc` file, meaning that if a ZIP archive doesn't contain `.pyc` files, importing may be rather slow.

Changed in version 3.13: ZIP64 is supported

Changed in version 3.8: Previously, ZIP archives with an archive comment were not supported.

See also

PKZIP Application Note

PEP 273- Import Modules from Zip Archives

`importlib`- The implementation of the import machinery

Documentation on the ZIP file format by Phil Katz, the creator of the format and algorithms used.

Written by James C. Ahlstrom, who also provided an implementation. Python 2.3 follows the specification in PEP 273, but uses an implementation written by Just van Rossum that uses the import hooks described in PEP 302.

Package providing the relevant protocols for all importers to implement.

This module defines an exception:

`zipimport.ZipImportError`

Exception raised by `zipimporter` objects. It's a subclass of `ImportError`, so it can be caught as `ImportError`, too.

zipimporter Objects

`zipimporter` is the class for importing ZIP files.

`class zipimport.zipimporter(archivepath)`

`create_module(spec)`

`exec_module(module)`

`find_spec(fullname, target=None)`

`get_code(fullname)`

`get_data(pathname)`

`get_filename(fullname)`

get_source(fullname)

is_package(fullname)

load_module(fullname)

invalidate_caches()

archive

prefix

Create a new `zipimporter` instance. `archivepath` must be a path to a ZIP file, or to a specific path within a ZIP file. For example, an `archivepath` of `foo/bar.zip/lib` will look for modules in the `lib` directory inside the ZIP file `foo/bar.zip` (provided that it exists). `ZipImportError` is raised if `archivepath` doesn't point to a valid ZIP archive. Changed in version 3.12: Methods `find_loader()` and `find_module()`, deprecated in 3.10 are now removed. Use `find_spec()` instead. `create_module(spec)` Implementation of `importlib.ab`

Implementation of `importlib.abc.Loader.create_module()` that returns `None` to explicitly request the default semantics. Added in version 3.10.

Implementation of `importlib.abc.Loader.exec_module()`. Added in version 3.10.

An implementation of `importlib.abc.PathEntryFinder.find_spec()`. Added in version 3.10.

Return the code object for the specified module. Raise `ZipImportError` if the module couldn't be imported.

Return the data associated with `pathname`. Raise `OSError` if the file wasn't found. Changed in version 3.3: `IOError` used to be raised, it is now an alias of `OSError`.

Return the value `__file__` would be set to if the specified module was imported. Raise `ZipImportError` if the module couldn't be imported. Added in version 3.1.

Return the source code for the specified module. Raise `ZipImportError` if the module couldn't be found, return `None` if the archive does contain the module, but has no source for it.

Return `True` if the module specified by `fullname` is a package. Raise `ZipImportError` if the module couldn't be found.

Load the module specified by `fullname`. `fullname` must be the fully qualified (dotted) module name. Returns the imported module on success, raises `ZipImportError` on failure. Deprecated since version 3.10, will be removed in version 3.15: Use `exec_module()` instead.

Clear out the internal cache of information about files found within the ZIP archive. Added in version 3.10.

The file name of the importer's associated ZIP file, without a possible subpath.

The subpath within the ZIP file where modules are searched. This is the empty string for `zipimporter` objects which point to the root of the ZIP file.

Examples

Here is an example that imports a module from a ZIP archive - note that the `zipimport` module is not explicitly used.

```
$ unzip -l example_archive.zip
Archive:  example_archive.zip
  Length      Date    Time    Name
-----
   8467   01-01-00  12:30   example.py
-----
   8467                      1 file
```

```
>>> import sys
>>> # Add the archive to the front of the module search path
>>> sys.path.insert(0, 'example_archive.zip')
>>> import example
>>> example.__file__
'example_archive.zip/example.py'
```

pkgutil— Package extension utility

Source code: `Lib/pkgutil.py`

This module provides utilities for the import system, in particular package support.

class `pkgutil.ModuleInfo(module_finder, name, ispkg)`

A namedtuple that holds a brief summary of a module's info. Added in version 3.6.

`pkgutil.extend_path(path, name)`

Extend the search path for the modules which comprise a package. Intended use is to place the following code in a package's `__init__.py`: `from pkgutil import extend_path; __path__ = extend_path(__path__, __name__)` For each directory on `sys.path` that has a subdirectory that matches the package name, add the subdirectory to the package's `__path__`. This is useful if one wants to distribute different parts of a single logical package as multiple directories. It also looks for `*.pkg` files beginning where `*` matches then

`pkgutil.get_importer(path_item)`

Retrieve a finder for the given `path_item`. The returned finder is cached in `sys.path_importer_cache` if it was newly created by a path hook. The cache (or part of it) can be cleared manually if a rescan of `sys.path_hooks` is necessary. Changed in version 3.3: Updated to be based directly on `importlib` rather than relying on the package internal PEP 302 import emulation.

`pkgutil.iter_importers(fullname="")`

Yield finder objects for the given module name. If `fullname` contains a '.', the finders will be for the package containing `fullname`, otherwise they will be all registered top level finders (i.e. those on both `sys.meta_path` and `sys.path_hooks`). If the named module is in a package, that package is imported as a side effect of invoking this function. If no module name is specified, all top level finders are produced. Changed in version 3.3: Updated to be based directly on `importlib` rather than relying on the package

`pkgutil.iter_modules(path=None, prefix="")`

Yields `ModuleInfo` for all submodules on `path`, or, if `path` is `None`, all top-level modules on `sys.path`. `path` should be either `None` or a list of paths to look for modules in. `prefix` is a string to output on the front of every module name on output. Note Only works for a finder which defines an `iter_modules()` method. This interface is non-standard, so the module also provides implementations for `importlib.machinery.FileFinder` and `zipimport.zipimporter`. Changed in version 3.3: Updated to be based directly on `importlib` rather

`pkgutil.walk_packages(path=None, prefix="", onerror=None)`

Yields `ModuleInfo` for all modules recursively on `path`, or, if `path` is `None`, all accessible modules. `path` should be either `None` or a list of paths to look for modules in. `prefix` is a string to output on the front of every module name on output. Note that this function must import all packages (not all modules!) on the given `path`, in order to access the `__path__` attribute to find submodules. `onerror` is a function which gets called with one argument (the name of the package which was being imported) if any exception occurs

`pkgutil.get_data(package, resource)`

Get a resource from a package. This is a wrapper for the `loader.get_data` API. The `package` argument should be the name of a package, in standard module format (`foo.bar`). The `resource` argument should be in the form of a relative filename, using `/` as the path separator. The parent directory name `..` is not allowed, and nor is a rooted name (starting with `/`). The function returns a binary string that is the contents of the specified resource. For packages located in the filesystem, which have already been imported

`pkgutil.resolve_name(name)`

Resolve a name to an object. This functionality is used in numerous places in the standard library (see [bpo-12915](#)) - and equivalent functionality is also in widely used third-party packages such as `setuptools`, `Django` and `Pyramid`. It is expected that `name` will be a string in one of the following formats, where `W` is shorthand for a valid Python identifier and `.` stands for a literal period in these pseudo-regexes: `W(W)*W(W)*:(W(W)*)?` The first form is intended for backward compatibility only. It assu

modulefinder— Find modules used by a script

Source code: `Lib/modulefinder.py`

This module provides a `ModuleFinder` class that can be used to determine the set of modules imported by a script. `modulefinder.py` can also be run as a script, giving the filename of a Python script as its argument, after which a report of the imported modules will be printed.

`modulefinder.AddPackagePath(pkg_name, path)`

Record that the package named `pkg_name` can be found in the specified `path`.

`modulefinder.ReplacePackage(oldname, newname)`

Allows specifying that the module named `oldname` is in fact the package named `newname`.

`class modulefinder.ModuleFinder(path=None, debug=0, excludes=[], replace_paths=[])`

`report()`

`run_script(pathname)`

`modules`

This class provides `run_script()` and `report()` methods to determine the set of modules imported by a script. `path` can be a list of directories to search for modules; if not specified, `sys.path` is used. `debug` sets the debugging level; higher values make the class print debugging messages about what it's doing. `excludes` is a list of module names to exclude from the analysis. `replace_paths` is a list of `(oldpath, newpath)` tuples that will be replaced in module paths. `report()` Print a report to standard output that list

Print a report to standard output that lists the modules imported by the script and their paths, as well as modules that are missing or seem to be missing.

Analyze the contents of the `pathname` file, which must contain Python code.

A dictionary mapping module names to modules. See Example usage of `ModuleFinder`.

Example usage of `ModuleFinder`

The script that is going to get analyzed later on (`bacon.py`):

```
import re, itertools

try:
    import baconhameggs
except ImportError:
    pass

try:
    import guido.python.ham
except ImportError:
    pass
```

The script that will output the report of `bacon.py`:

```
from modulefinder import ModuleFinder

finder = ModuleFinder()
finder.run_script('bacon.py')

print('Loaded modules:')
for name, mod in finder.modules.items():
```

```

    print('%s: ' % name, end='')
    print(', '.join(list(mod.globalnames.keys())[:3]))

print('-'*50)
print('Modules not imported:')
print('\n'.join(finder.badmodules.keys()))

```

Sample output (may vary depending on the architecture):

```

Loaded modules:
_types:
copyreg:  _inverted_registry,_slotnames,__all__
re._compiler:  isstring,_sre,_optimize_unicode
_sre:
re._constants:  REPEAT_ONE,makedict,AT_END_LINE
sys:
re:  __module__,finditer,_expand
itertools:
__main__:  re,itertools,baconhammeggs
re._parser:  _PATTERNENDERS,SRE_FLAG_UNICODE
array:
types:  __module__,IntType,TypeType
-----
Modules not imported:
guido.python.ham
baconhammeggs

```

runpy— Locating and executing Python modules

Source code: `Lib/runpy.py`

The `runpy` module is used to locate and run Python modules without importing them first. Its main use is to implement the `-m` command line switch that allows scripts to be located using the Python module namespace rather than the filesystem.

Note that this is not a sandbox module - all code is executed in the current process, and any side effects (such as cached imports of other modules) will remain in place after the functions have returned.

Furthermore, any functions and classes defined by the executed code are not guaranteed to work correctly after a `runpy` function has returned. If that limitation is not acceptable for a given use case, `importlib` is likely to be a more suitable choice than this module.

The `runpy` module provides two functions:

`runpy.run_module(mod_name,init_globals=None,run_name=None,alter_sys=False)`

Execute the code of the specified module and return the resulting module's globals dictionary. The module's code is first located using the standard import mechanism (refer to PEP 302 for details) and then executed in a fresh module namespace. The `mod_name` argument should be an absolute module name. If the module name refers to a package rather than a normal module, then that package is imported and the `__main__` submodule within that package is then executed and the resulting module globals dictionary

`runpy.run_path(path_name,init_globals=None,run_name=None)`

Execute the code at the named filesystem location and return the resulting module's globals dictionary. As with a script name supplied to the CPython command line, `file_path` may refer to a Python source file, a compiled bytecode file or a valid `sys.path` entry containing a `__main__` module (e.g. a zipfile containing a top-level `__main__.py` file). For a simple script, the specified code is simply executed in a fresh module namespace. For a valid `sys.path` entry (typically a zipfile or directory), the entry is

See also

PEP 338– Executing modules as scripts

PEP 366– Main module explicit relative imports

PEP 451– A ModuleSpec Type for the Import System

PEP written and implemented by Nick Coghlan.

PEP written and implemented by Nick Coghlan.

PEP written and implemented by Eric Snow

Command line and environment- CPython command line details

The `importlib.import_module()` function

importlib— The implementation of import

Added in version 3.1.

Source code: `Lib/importlib/__init__.py`

Introduction

The purpose of the `importlib` package is three-fold.

One is to provide the implementation of the `import` statement (and thus, by extension, the `__import__()` function) in Python source code. This provides an implementation of `import` which is portable to any Python interpreter. This also provides an implementation which is easier to comprehend than one implemented in a programming language other than Python.

Two, the components to implement `import` are exposed in this package, making it easier for users to create their own custom objects (known generically as an `importer`) to participate in the import process.

Three, the package contains modules exposing additional functionality for managing aspects of Python packages:

- `importlib.metadata` presents access to metadata from third-party distributions.
- `importlib.resources` provides routines for accessing non-code “resources” from Python packages.

See also

The `import` statement

Packages specification

The `__import__()` function

The initialization of the `sys.path` module search path

PEP 235

PEP 263

PEP 302

PEP 328

PEP 366

PEP 420

PEP 451

PEP 488

PEP 489

PEP 552

PEP 3120

PEP 3147

The language reference for the `import` statement.

Original specification of packages. Some semantics have changed since the writing of this document (e.g. redirecting based on `None` in `sys.modules`).

The `import` statement is syntactic sugar for this function.

The initialization of `sys.path`.

Import on Case-Insensitive Platforms

Defining Python Source Code Encodings

New Import Hooks

Imports: Multi-Line and Absolute/Relative

Main module explicit relative imports

Implicit namespace packages

A `ModuleSpec` Type for the Import System

Elimination of PYO files

Multi-phase extension module initialization

Deterministic pycs

Using UTF-8 as the Default Source Encoding

PYC Repository Directories

Functions

`importlib.__import__(name, globals=None, locals=None, fromlist=(), level=0)`

An implementation of the built-in `__import__()` function. Note that programmatic importing of modules should use `import_module()` instead of this function.

`importlib.import_module(name, package=None)`

Import a module. The `name` argument specifies what module to import in absolute or relative terms (e.g. either `pkg.mod` or `..mod`). If the name is specified in relative terms, then the `package` argument must be set to the name of the package which is to act as the anchor for resolving the package name (e.g. `import_module('..mod', 'pkg.subpkg')` will import `pkg.mod`). The `import_module()` function acts as a simplifying wrapper around `importlib.__import__()`. This means all semantics of the function are derived from `importlib.__import__()`.

`importlib.invalidate_caches()`

Invalidate the internal caches of finders stored at `sys.meta_path`. If a finder implements `invalidate_caches()` then it will be called to perform the invalidation. This function should be called if any modules are created/installed while your program is running to guarantee all finders will notice the new module's existence. Added in version 3.3. Changed in version 3.10: Namespace packages created/installed in a different `sys.path` location after the same namespace was already imported are noticed.

`importlib.reload(module)`

Reload a previously imported module. The argument must be a module object, so it must have been successfully imported before. This is useful if you have edited the module source file using an external editor and want to try out the new version without leaving the Python interpreter. The return value is the module object (which can be different if re-importing causes a different object to be placed in `sys.modules`). When `reload()` is executed: Python module's code is recompiled and the module-level co

`importlib.abc` – Abstract base classes related to import

Source code:Lib/importlib/abc.py

Theimportlib.abcmodule contains all of the core abstract base classes used byimport. Some subclasses of the core abstract base classes are also provided to help in implementing the core ABCs.

ABC hierarchy:

```
object
+-- MetaPathFinder
+-- PathEntryFinder
+-- Loader
    +-- ResourceLoader -----+
    +-- InspectLoader          |
    +-- ExecutionLoader -----+
                                   +-- FileLoader
                                   +-- SourceLoader
```

classimportlib.abc.MetaPathFinder

find_spec(fullname,path,target=None)

invalidate_caches()

An abstract base class representing a meta path finder. Added in version 3.3. Changed in version 3.10: No longer a subclass of Finder. find_spec(fullname,path,target=None) An abstract method for finding a spec for the specified module. If this is a top-level import, path will be None. Otherwise, this is a search for a subpackage or module and path will be the value of __path__ from the parent package. If a spec cannot be found, None is returned. When passed in, target is a module object that the finder may use to

An abstract method for finding a spec for the specified module. If this is a top-level import, path will be None. Otherwise, this is a search for a subpackage or module and path will be the value of __path__ from the parent package. If a spec cannot be found, None is returned. When passed in, target is a module object that the finder may use to make a more educated guess about what spec to return. importlib.util.spec_from_loader() may be useful for implementing concrete MetaPathFinders. Added in version 3.4.

An optional method which, when called, should invalidate any internal cache used by the finder. Used by importlib.invalidate_caches() when invalidating the caches of all finders on sys.meta_path. Changed in version 3.4: Returns None when called instead of NotImplemented.

classimportlib.abc.PathEntryFinder

find_spec(fullname,target=None)

invalidate_caches()

An abstract base class representing a path entry finder. Though it bears some similarities to MetaPathFinder, PathEntryFinder is meant for use only within the path-based import subsystem provided by importlib.machinery.PathFinder. Added in version 3.3. Changed in version 3.10: No longer a subclass of Finder. find_spec(fullname,target=None) An abstract method for finding a spec for the specified module. The finder will search for the module only within the path entry to which it is assigned. If a spec cannot

An abstract method for finding a spec for the specified module. The finder will search for the module only within the path entry to which it is assigned. If a spec cannot be found, None is returned. When passed in, target is a module object that the finder may use to make a more educated guess about what spec to return. importlib.util.spec_from_loader() may be useful for implementing concrete PathEntryFinders. Added in version 3.4.

An optional method which, when called, should invalidate any internal cache used by the finder. Used by importlib.machinery.PathFinder.invalidate_caches() when invalidating the caches of all cached finders.

classimportlib.abc.Loader

create_module(spec)

exec_module(module)

load_module(fullname)

An abstract base class for a loader. See PEP 302 for the exact definition for a loader. Loaders that wish to support resource reading should implement `get_resource_reader()` method as specified by `importlib.resources.abc.ResourceReader`. Changed in version 3.7: Introduced the optional `get_resource_reader()` method. **create_module(spec)** A method that returns the module object to use when importing a module. This method may return `None`, indicating that default module creation semantics should take place. Added in

A method that returns the module object to use when importing a module. This method may return `None`, indicating that default module creation semantics should take place. Added in version 3.4. Changed in version 3.6: This method is no longer optional when `exec_module()` is defined.

An abstract method that executes the module in its own namespace when a module is imported or reloaded. The module should already be initialized when `exec_module()` is called. When this method exists, `create_module()` must be defined. Added in version 3.4. Changed in version 3.6: `create_module()` must also be defined.

A legacy method for loading a module. If the module cannot be loaded, `ImportError` is raised, otherwise the loaded module is returned. If the requested module already exists in `sys.modules`, that module should be used and reloaded. Otherwise the loader should create a new module and insert it into `sys.modules` before any loading begins, to prevent recursion from the import. If the loader inserted a module and the load fails, it must be removed by the loader from `sys.modules`; modules already in `sys.modules`

class importlib.abc.ResourceLoader

abstract method get_data(path)

Superseded by `TraversableResources` An abstract base class for a loader which implements the optional PEP 302 protocol for loading arbitrary resources from the storage back-end. Deprecated since version 3.7: This ABC is deprecated in favour of supporting resource loading through `importlib.resources.abc.TraversableResources`. This class exists for backwards compatibility only with other ABCs in this module. **abstract method get_data(path)** An abstract method to return the bytes for the data located at `path`. Loaders

An abstract method to return the bytes for the data located at `path`. Loaders that have a file-like storage back-end that allows storing arbitrary data can implement this abstract method to give direct access to the data stored. `OSError` is to be raised if the `path` cannot be found. The `path` is expected to be constructed using a module's `__file__` attribute or an item from a package's `__path__`. Changed in version 3.4: Raises `OSError` instead of `NotImplementedError`.

class importlib.abc.InspectLoader

get_code(fullname)

abstract method get_source(fullname)

is_package(fullname)

static source_to_code(data, path='<string>')

exec_module(module)

load_module(fullname)

An abstract base class for a loader which implements the optional PEP 302 protocol for loaders that inspect modules. **get_code(fullname)** Return the code object for a module, or `None` if the module does not have a code object (as would be the case, for example, for a built-in module). Raise an `ImportError` if loader cannot find the

requested module. Note While the method has a default implementation, it is suggested that it be overridden if possible for performance. Changed in version 3.4: No longer abstract and

Return the code object for a module, or None if the module does not have a code object (as would be the case, for example, for a built-in module). Raise an ImportError if loader cannot find the requested module. Note While the method has a default implementation, it is suggested that it be overridden if possible for performance. Changed in version 3.4: No longer abstract and a concrete implementation is provided.

An abstract method to return the source of a module. It is returned as a text string using universal newlines, translating all recognized line separators into '\n' characters. Returns None if no source is available (e.g. a built-in module). Raises ImportError if the loader cannot find the module specified. Changed in version 3.4: Raises ImportError instead of NotImplementedError.

An optional method to return a true value if the module is a package, a false value otherwise. ImportError is raised if the loader cannot find the module. Changed in version 3.4: Raises ImportError instead of NotImplementedError.

Create a code object from Python source. The data argument can be whatever the compile() function supports (i.e. string or bytes). The path argument should be the "path" to where the source code originated from, which can be an abstract concept (e.g. location in a zip file). With the subsequent code object one can execute it in a module by running exec(code, module.__dict__). Added in version 3.4. Changed in version 3.5: Made the method static.

Implementation of Loader.exec_module(). Added in version 3.4.

Implementation of Loader.load_module(). Deprecated since version 3.4, will be removed in version 3.15: use exec_module() instead.

class importlib.abc.ExecutionLoader

abstractmethod get_filename(fullname)

An abstract base class which inherits from InspectLoader that, when implemented, helps a module to be executed as a script. The ABC represents an optional PEP 302 protocol. **abstractmethod get_filename(fullname)** An abstract method that is to return the value of `__file__` for the specified module. If no path is available, ImportError is raised. If source code is available, then the method should return the path to the source file, regardless of whether a bytecode was used to load the module. Changed in version

An abstract method that is to return the value of `__file__` for the specified module. If no path is available, ImportError is raised. If source code is available, then the method should return the path to the source file, regardless of whether a bytecode was used to load the module. Changed in version 3.4: Raises ImportError instead of NotImplementedError.

class importlib.abc.FileLoader(fullname, path)

name

path

load_module(fullname)

abstractmethod get_filename(fullname)

abstractmethod get_data(path)

An abstract base class which inherits from ResourceLoader and ExecutionLoader, providing concrete implementations of ResourceLoader.get_data() and ExecutionLoader.get_filename(). The fullname argument is a fully resolved name of the module the loader is to handle. The path argument is the path to the file for the module. Added in version 3.3. **name** The name of the module the loader can handle. **path** Path to the file of the

`module.load_module(fullname)` Calls `super().load_module()`. Deprecatd since version 3.4, will b

The name of the module the loader can handle.

Path to the file of the module.

Calls `super().load_module()`. Deprecatd since version 3.4, will be removed in version 3.15: Use `Loader.exec_module()` instead.

Returns path.

Reads path as a binary file and returns the bytes from it.

`classimportlib.abc.SourceLoader`

`ExecutionLoader.get_filename()`

`path_stats(path)`

`path_mtime(path)`

`set_data(path,data)`

`get_code(fullname)`

`exec_module(module)`

`load_module(fullname)`

`get_source(fullname)`

`is_package(fullname)`

An abstract base class for implementing source (and optionally bytecode) file loading. The class inherits from both `ResourceLoader` and `ExecutionLoader`, requiring the implementation of: `ResourceLoader.get_data()` `ExecutionLoader.get_filename()` Should only return the path to the source file; sourceless loading is not supported. The abstract methods defined by this class are to add optional bytecode file support. Not implementing these optional methods (or causing them to raise `NotImplementedError`) causes t

Should only return the path to the source file; sourceless loading is not supported.

Optional abstract method which returns a dict containing metadata about the specified path. Supported dictionary keys are: 'mtime' (mandatory): an integer or floating-point number representing the modification time of the source code; 'size' (optional): the size in bytes of the source code. Any other keys in the dictionary are ignored, to allow for future extensions. If the path cannot be handled, `OSError` is raised. Added in version 3.3. Changed in version 3.4: Raise `OSError` instead of `NotImplementedError`.

Optional abstract method which returns the modification time for the specified path. Deprecatd since version 3.3: This method is deprecated in favour of `path_stats()`. You don't have to implement it, but it is still available for compatibility purposes. Raise `OSError` if the path cannot be handled. Changed in version 3.4: Raise `OSError` instead of `NotImplementedError`.

Optional abstract method which writes the specified bytes to a file path. Any intermediate directories which do not exist are to be created automatically. When writing to the path fails because the path is read-only (`errno.EACCES/PermissionError`), do not propagate the exception. Changed in version 3.4: No longer raises `NotImplementedError` when called.

Concrete implementation of `inspect.Loader.get_code()`.

Concrete implementation of `Loader.exec_module()`. Added in version 3.4.

Concrete implementation of `Loader.load_module()`. Deprecatd since version 3.4, will be removed in version 3.15: Use `exec_module()` instead.

Concrete implementation of `inspect.Loader.get_source()`.

Concrete implementation of `inspect.Loader.is_package()`. A module is determined to be a package if its file path (as provided by `ExecutionLoader.get_filename()`) is a file named `__init__` when the file extension is removed and the module name itself does not end in `__init__`.

importlib.machinery– Importers and path hooks

Source code: `Lib/importlib/machinery.py`

This module contains the various objects that help import find and load modules.

importlib.machinery.SOURCE_SUFFIXES

A list of strings representing the recognized file suffixes for source modules. Added in version 3.3.

importlib.machinery.DEBUG_BYTECODE_SUFFIXES

A list of strings representing the file suffixes for non-optimized bytecode modules. Added in version 3.3. Deprecated since version 3.5: Use `BYTECODE_SUFFIXES` instead.

importlib.machinery.OPTIMIZED_BYTECODE_SUFFIXES

A list of strings representing the file suffixes for optimized bytecode modules. Added in version 3.3. Deprecated since version 3.5: Use `BYTECODE_SUFFIXES` instead.

importlib.machinery.BYTECODE_SUFFIXES

A list of strings representing the recognized file suffixes for bytecode modules (including the leading dot). Added in version 3.3. Changed in version 3.5: The value is no longer dependent on `__debug__`.

importlib.machinery.EXTENSION_SUFFIXES

A list of strings representing the recognized file suffixes for extension modules. Added in version 3.3.

importlib.machinery.all_suffixes()

Returns a combined list of strings representing all file suffixes for modules recognized by the standard import machinery. This is a helper for code which simply needs to know if a filesystem path potentially refers to a module without needing any details on the kind of module (for example, `inspect.getmodulename()`). Added in version 3.3.

class importlib.machinery.BuiltinImporter

An importer for built-in modules. All known built-in modules are listed in `sys.builtin_module_names`. This class implements the `importlib.abc.MetaPathFinder` and `importlib.abc.Loader` ABCs. Only class methods are defined by this class to alleviate the need for instantiation. Changed in version 3.5: As part of PEP 489, the builtin importer now implements `Loader.create_module()` and `Loader.exec_module()`.

class importlib.machinery.FrozenImporter

An importer for frozen modules. This class implements the `importlib.abc.MetaPathFinder` and `importlib.abc.Loader` ABCs. Only class methods are defined by this class to alleviate the need for instantiation. Changed in version 3.4: Gained `create_module()` and `exec_module()` methods.

class importlib.machinery.WindowsRegistryFinder

Finder for modules declared in the Windows registry. This class implements the `importlib.abc.MetaPathFinder` ABC. Only class methods are defined by this class to alleviate the need for instantiation. Added in version 3.3. Deprecated since version 3.6: Use `site` configuration instead. Future versions of Python may not enable this finder by default.

classimportlib.machinery.PathFinder

classmethodfind_spec(fullname,path=None,target=None)

classmethodinvalidate_caches()

A Finder for `sys.path` and package `__path__` attributes. This class implements `theimportlib.abc.MetaPathFinderABC`. Only class methods are defined by this class to alleviate the need for instantiation. `classmethodfind_spec(fullname,path=None,target=None)` Class method that attempts to find `aspecfor` the module specified by `fullname` on `sys.path` or, if defined, on `path`. For each path entry that is searched, `sys.path_importer_cache` is checked. If a non-`false` object is found then it is used as the path entry `find` to `look`

Class method that attempts to find `aspecfor` the module specified by `fullname` on `sys.path` or, if defined, on `path`. For each path entry that is searched, `sys.path_importer_cache` is checked. If a non-`false` object is found then it is used as the path entry `find` to look for the module being searched for. If no entry is found in `sys.path_importer_cache`, then `sys.path_hooks` is searched for a finder for the path entry and, if found, is stored in `sys.path_importer_cache` along with being queried about the module. If

Calls `importlib.abc.PathEntryFinder.invalidate_caches()` on all finders stored in `sys.path_importer_cache` that define the method. Otherwise entries in `sys.path_importer_cache` set to `None` are deleted. Changed in version 3.7: Entries of `None` in `sys.path_importer_cache` are deleted.

classimportlib.machinery.FileFinder(path,*loader_details)

path

find_spec(fullname,target=None)

invalidate_caches()

classmethodpath_hook(*loader_details)

A concrete implementation of `importlib.abc.PathEntryFinder` which caches results from the file system. The `path` argument is the directory for which the finder is in charge of searching. The `loader_details` argument is a variable number of 2-item tuples each containing a loader and a sequence of file suffixes the loader recognizes. The loaders are expected to be callables which accept two arguments of the module's name and the path to the file found. The finder will cache the directory contents as necessary

The path the finder will search in.

Attempt to find the spec to handle `fullname` within `path`. Added in version 3.4.

Clear out the internal cache.

A class method which returns a closure for use on `sys.path_hooks`. An instance of `FileFinder` is returned by the closure using the `path` argument given to the closure directly and `loader_details` indirectly. If the argument to the closure is not an existing directory, `ImportError` is raised.

classimportlib.machinery.SourceFileLoader(fullname,path)

name

path

is_package(fullname)

path_stats(path)

set_data(path,data)

load_module(name=None)

A concrete implementation of `importlib.abc.SourceLoader` by subclassing `importlib.abc.FileLoader` and providing some concrete implementations of other methods. Added in version 3.3.

name The name of the module that this loader will handle.

path The path to the source file.

is_package(fullname) Return `True` if `path` appears to be for a package.

path_stats(path) Concrete implementation of `importlib.abc.SourceLoader.path_stats()`.

set_data(path, data) Concrete implementation of `importlib.abc.SourceLoader.set_data()`.

load_module(name) Concrete implementation of `importlib.abc.Loader.load_module()` where specifying the name of the module to load is optional. Deprecated since version 3.6, will be removed in version 3.15. Use `importlib.abc.Loader.exec_module()` instead.

The name of the module that this loader will handle.

The path to the source file.

Return `True` if `path` appears to be for a package.

Concrete implementation of `importlib.abc.SourceLoader.path_stats()`.

Concrete implementation of `importlib.abc.SourceLoader.set_data()`.

Concrete implementation of `importlib.abc.Loader.load_module()` where specifying the name of the module to load is optional. Deprecated since version 3.6, will be removed in version 3.15. Use `importlib.abc.Loader.exec_module()` instead.

`class importlib.machinery.SourcelessFileLoader(fullname, path)`

name

path

is_package(fullname)

get_code(fullname)

get_source(fullname)

load_module(name=None)

A concrete implementation of `importlib.abc.FileLoader` which can import bytecode files (i.e. no source code files exist). Please note that direct use of bytecode files (and thus not source code files) inhibits your modules from being usable by all Python implementations or new versions of Python which change the bytecode format. Added in version 3.3.

name The name of the module the loader will handle.

path The path to the bytecode file.

is_package(fullname) Determines if the module is a package based on `path`.

The name of the module the loader will handle.

The path to the bytecode file.

Determines if the module is a package based on `path`.

Returns the code object for `name` created from `path`.

Returns `None` as bytecode files have no source when this loader is used.

`class importlib.machinery.ExtensionFileLoader(fullname, path)`

name

path

create_module(spec)

exec_module(module)

is_package(fullname)

get_code(fullname)

get_source(fullname)

get_filename(fullname)

A concrete implementation of `importlib.abc.ExecutionLoader` for extension modules. The `fullname` argument specifies the name of the module the loader is to support. The `path` argument is the path to the extension module's file. Note that, by default, importing an extension module will fail in subinterpreters if it doesn't implement multi-phase init (see PEP 489), even if it would otherwise import successfully. Added in version 3.3. Changed in version 3.12: Multi-phase init is now required for use in subinterpreter.

Name of the module the loader supports.

Path to the extension module.

Creates the module object from the given specification in accordance with PEP 489. Added in version 3.5.

Initializes the given module object in accordance with PEP 489. Added in version 3.5.

Returns `True` if the file path points to a package's `__init__` module based on `EXTENSION_SUFFIXES`.

Returns `None` as extension modules lack a code object.

Returns `None` as extension modules do not have source code.

Returns `path`. Added in version 3.4.

class importlib.machinery.NamespaceLoader(name, path, path_finder)

A concrete implementation of `importlib.abc.InspectLoader` for namespace packages. This is an alias for a private class and is only made public for introspecting the `__loader__` attribute on namespace packages:

```
>>> from importlib.machinery import NamespaceLoader
>>> import my_namespace
>>> isinstance(my_namespace.__loader__, NamespaceLoader)
True
>>> import importlib.abc
>>> isinstance(my_namespace.__loader__, importlib.abc.Loader)
True
```

 Added in version 3.11.

class importlib.machinery.ModuleSpec(name, loader, *, origin=None, loader_state=None, is_package=None)

name

loader

origin

submodule_search_locations

loader_state

cached

parent

has_location

A specification for a module's import-system-related state. This is typically exposed as the module's `__spec__` attribute. Many of these attributes are also available directly on a module: for example, `module.__spec__.origin == module.__file__`. Note, however, that while the values are usually equivalent, they can differ since there is no synchronization between the two objects. For example, it is possible to update the module's `__file__` at runtime and this will not be automatically reflected in the module's `__spec__`.

The module's fully qualified name (see `module.__name__`). The finders should always set this attribute to a non-empty string.

The loader used to load the module (see `module.__loader__`). The finders should always set this attribute.

The location the loader should use to load the module (see `module.__file__`). For example, for modules loaded from a .py file this is the filename. The finders should always set this attribute to a meaningful value for the loader to

use. In the uncommon case that there is not one (like for namespace packages), it should be set to `None`.

A (possibly empty) sequence of strings enumerating the locations in which a package's submodules will be found (see `module.__path__`). Most of the time there will only be a single directory in this list. The finder should set this attribute to a sequence, even an empty one, to indicate to the import system that the module is a package. It should be set to `None` for non-package modules. It is set automatically later to a special object for namespace packages.

The finder may set this attribute to an object containing additional, module-specific data to use when loading the module. Otherwise it should be set to `None`.

The filename of a compiled version of the module's code (see `module.__cached__`). The finder should always set this attribute but it may be `None` for modules that do not need compiled code stored.

(Read-only) The fully qualified name of the package the module is in (or the empty string for a top-level module). See `module.__package__`. If the module is a package then this is the same as `name`.

`True` if the spec's `origin` refers to a loadable location, `False` otherwise. This value impacts how `origin` is interpreted and how the module's `__file__` is populated.

`class importlib.machinery.AppleFrameworkLoader(name, path)`

name

path

A specialization of `importlib.machinery.ExtensionFileLoader` that is able to load extension modules in Framework format. For compatibility with the iOS App Store, all binary modules in an iOS app must be dynamic libraries, contained in a framework with appropriate metadata, stored in the `Frameworks` folder of the packaged app. There can be only a single binary per framework, and there can be no executable binary material outside the Frameworks folder. To accommodate this requirement, when running on iOS,

Name of the module the loader supports.

Path to the `.framework` file for the extension module.

importlib.util – Utility code for importers

Source code: `Lib/importlib/util.py`

This module contains the various objects that help in the construction of an importer.

`importlib.util.MAGIC_NUMBER`

The bytes which represent the bytecode version number. If you need help with loading/writing bytecode then consider `importlib.abc.SourceLoader`. Added in version 3.4.

`importlib.util.cache_from_source(path, debug_override=None, *, optimization=None)`

Return the PEP 3147/PEP 488 path to the byte-compiled file associated with the source `path`. For example, if `path` is `foo/bar/baz.py` the return value would be `foo/bar/__pycache__/baz.cpython-32.pyc` for Python 3.2. The `cpython-32` string comes from the current magic tag (see `get_tag()`; if `sys.implementation.cache_tag` is not defined then `NotImplementedError` will be raised). The `optimization` parameter is used to specify the optimization level of the bytecode file. An empty string represents no optimization, so `foo/ba`

`importlib.util.source_from_cache(path)`

Given the `path` to a PEP 3147 file name, return the associated source code file path. For example, if `path` is `foo/bar/__pycache__/baz.cpython-32.pyc` the returned path would be `foo/bar/baz.py`. `path` need not exist, however if it does not conform to PEP 3147 or PEP 488 format, a `ValueError` is raised.

If `sys.implementation.cache_tag` is not defined, `NotImplementedError` is raised. Added in version 3.4. Changed in version 3.6: Accepts a path-like object.

`importlib.util.decode_source(source_bytes)`

Decode the given bytes representing source code and return it as a string with universal newlines (as required by `importlib.abc.Loader.get_source()`). Added in version 3.4.

`importlib.util.resolve_name(name, package)`

Resolve a relative module name to an absolute one. If `name` has no leading dots, then `name` is simply returned. This allows for usage such as `importlib.util.resolve_name('sys', __spec__.parent)` without doing a check to see if the `package` argument is needed. `ImportError` is raised if `name` is a relative module name but `package` is a false value (e.g. `None` or the empty string). `ImportError` is also raised if a relative name would escape its containing package (e.g. requesting `..bacon` from within `the spam package`). Added in version 3.4.

`importlib.util.find_spec(name, package=None)`

Find the `spec` for a module, optionally relative to the specified `package` name. If the module is in `sys.modules`, then `sys.modules[name].__spec__` is returned (unless the `spec` would be `None` or is not set, in which case `ValueError` is raised). Otherwise a search using `sys.meta_path` is done. `None` is returned if no `spec` is found. If `name` is for a submodule (contains a dot), the parent module is automatically imported. `name` and `package` work the same as for `import_module()`. Added in version 3.4. Changed in version 3.7: Raises `ModuleNotFoundError`.

`importlib.util.module_from_spec(spec)`

Create a new module based on `spec` and `spec.loader.create_module()`. If `spec.loader.create_module()` does not return `None`, then any pre-existing attributes will not be reset. Also, `NoAttributeError` will be raised if triggered while accessing `spec` or setting an attribute on the module. This function is preferred over using `types.ModuleType` to create a new module as `spec` is used to set as many import-controlled attributes on the module as possible. Added in version 3.5.

`importlib.util.spec_from_loader(name, loader, *, origin=None, is_package=None)`

A factory function for creating a `ModuleSpec` instance based on a loader. The parameters have the same meaning as they do for `ModuleSpec`. The function uses available loader APIs, such as `inspect.Loader.is_package()`, to fill in any missing information on the `spec`. Added in version 3.4.

`importlib.util.spec_from_file_location(name, location, *, loader=None, submodule_search_locations=None)`

A factory function for creating a `ModuleSpec` instance based on the path to a file. Missing information will be filled in on the `spec` by making use of loader APIs and by the implication that the module will be file-based. Added in version 3.4. Changed in version 3.6: Accepts a path-like object.

`importlib.util.source_hash(source_bytes)`

Return the hash of `source_bytes` as bytes. A hash-based .pyc file embeds the `source_hash()` of the corresponding source file's contents in its header. Added in version 3.7.

`importlib.util._incompatible_extension_module_restrictions(*, disable_check)`

A context manager that can temporarily skip the compatibility check for extension modules. By default the check is enabled and will fail when a single-phase init module is imported in a subinterpreter. It will also fail for a multi-phase init module that doesn't explicitly support a per-interpreter GIL, when imported in an interpreter with its own GIL. Note that this function is meant to accommodate an unusual case; one which is likely to eventually go away. There's a pretty good chance this

`class importlib.util.LazyLoader(loader)`

classmethodfactory(loader)

A class which postpones the execution of the loader of a module until the module has an attribute accessed. This class only works with loaders that define `exec_module()` as control over what module type is used for the module is required. For those same reasons, the loader's `create_module()` method must return `None` or a type for which its `__class__` attribute can be mutated along with not using slots. Finally, modules which substitute the object placed into `sys.modules` will not work as there is no way to properly

A class method which returns a callable that creates a lazy loader. This is meant to be used in situations where the loader is passed by class instead of by instance. `suffices=importlib.machinery.SOURCE_SUFFIXES`
`oader=importlib.machinery.SourceFileLoader`
`lazy_loader=importlib.util.LazyLoader`
`factory(loader)`
`finder=importlib.machinery.FileFinder(path,(lazy_loader,suffices))`

Examples

Importing programmatically

To programmatically import a module, use `importlib.import_module()`.

```
import importlib

itertools = importlib.import_module('itertools')
```

Checking if a module can be imported

If you need to find out if a module can be imported without actually doing the import, then you should use `importlib.util.find_spec()`.

Note that if `name` is a submodule (contains a dot), `importlib.util.find_spec()` will import the parent module.

```
import importlib.util
import sys

# For illustrative purposes.
name = 'itertools'

if name in sys.modules:
    print(f"{name!r} already in sys.modules")
elif (spec := importlib.util.find_spec(name)) is not None:
    # If you chose to perform the actual import ...
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    print(f"{name!r} has been imported")
else:
    print(f"can't find the {name!r} module")
```

Importing a source file directly

This recipe should be used with caution: it is an approximation of an import statement where the file path is specified directly, rather than `sys.path` being searched. Alternatives should first be considered first, such as modifying `sys.path` when a proper module is required, or using `runpy.run_path()` when the global namespace resulting from running a Python file is appropriate.

To import a Python source file directly from a path, use the following recipe:

```
import importlib.util
import sys

def import_from_path(module_name, file_path):
```

```

spec = importlib.util.spec_from_file_location(module_name, file_path)
module = importlib.util.module_from_spec(spec)
sys.modules[module_name] = module
spec.loader.exec_module(module)
return module

# For illustrative purposes only (use of `json` is arbitrary).
import json
file_path = json.__file__
module_name = json.__name__

# Similar outcome as `import json`.
json = import_from_path(module_name, file_path)

```

Implementing lazy imports

The example below shows how to implement lazy imports:

```

>>> import importlib.util
>>> import sys
>>> def lazy_import(name):
...     spec = importlib.util.find_spec(name)
...     loader = importlib.util.LazyLoader(spec.loader)
...     spec.loader = loader
...     module = importlib.util.module_from_spec(spec)
...     sys.modules[name] = module
...     loader.exec_module(module)
...     return module
...
>>> lazy_typing = lazy_import("typing")
>>> #lazy_typing is a real module object,
>>> #but it is not loaded in memory yet.
>>> lazy_typing.TYPE_CHECKING
False

```

Setting up an importer

For deep customizations of import, you typically want to implement an importer. This means managing both the finder and loader side of things. For finders there are two flavours to choose from depending on your needs: a meta path finder or a path entry finder. The former is what you would put on `sys.meta_path` while the latter is what you create using `path entry hooks` on `sys.path_hooks` which works with `sys.path_entries` to potentially create a finder. This example will show you how to register your own importers so that import will use them (for creating an importer for yourself, read the documentation for the appropriate classes defined within this package):

```

import importlib.machinery
import sys

# For illustrative purposes only.
SpamMetaPathFinder = importlib.machinery.PathFinder
SpamPathEntryFinder = importlib.machinery.FileFinder
loader_details = (importlib.machinery.SourceFileLoader,
                  importlib.machinery.SOURCE_SUFFIXES)

# Setting up a meta path finder.
# Make sure to put the finder in the proper location in the list in terms of
# priority.
sys.meta_path.append(SpamMetaPathFinder)

# Setting up a path entry finder.
# Make sure to put the path hook in the proper location in the list in terms
# of priority.
sys.path_hooks.append(SpamPathEntryFinder.path_hook(loader_details))

```

Approximating `importlib.import_module()`

Import itself is implemented in Python code, making it possible to expose most of the import machinery through `importlib`. The following helps illustrate the various APIs that `importlib` exposes by providing an approximate implementation of `importlib.import_module()`:

```
import importlib.util
import sys

def import_module(name, package=None):
    """An approximate implementation of import."""
    absolute_name = importlib.util.resolve_name(name, package)
    try:
        return sys.modules[absolute_name]
    except KeyError:
        pass

    path = None
    if '.' in absolute_name:
        parent_name, _, child_name = absolute_name.rpartition('.')
        parent_module = import_module(parent_name)
        path = parent_module.__spec__.submodule_search_locations
    for finder in sys.meta_path:
        spec = finder.find_spec(absolute_name, path)
        if spec is not None:
            break
    else:
        msg = f'No module named {absolute_name!r}'
        raise ModuleNotFoundError(msg, name=absolute_name)
    module = importlib.util.module_from_spec(spec)
    sys.modules[absolute_name] = module
    spec.loader.exec_module(module)
    if path is not None:
        setattr(parent_module, child_name, module)
    return module
```

importlib.resources– Package resource reading, opening and access

Source code: `Lib/importlib/resources/__init__.py`

Added in version 3.7.

This module leverages Python's import system to provide access to resources within packages.

“Resources” are file-like resources associated with a module or package in Python. The resources may be contained directly in a package, within a subdirectory contained in that package, or adjacent to modules outside a package. Resources may be text or binary. As a result, a package's Python module sources (.py), compilation artifacts (pycache), and installation artifacts (like reserved filenames in directories) are technically de-facto resources of that package. In practice, however, resources are primarily those non-Python artifacts exposed specifically by the package author.

Resources can be opened or read in either binary or text mode.

Resources are roughly akin to files inside directories, though it's important to keep in mind that this is just a metaphor. Resources and packages do not have to exist as physical files and directories on the file system: for example, a package and its resources can be imported from a zip file using `zipimport`.

■ Nota: Note This module provides functionality similar to `pkg_resources.BasicResourceAccess` without the performance overhead of that package. This makes reading resources included in packages easier, with more stable and consistent semantics. The standalone backport of this module provides more information on using `importlib.resources` and migrating from `pkg_resources` to `importlib.resources`.

Loaders that wish to support resource reading should implement a `get_resource_reader(fullname)` method as specified by `importlib.resources.abc.ResourceReader`.

class importlib.resources.Anchor

Represents an anchor for resources, either a module object or a module name as a string. Defined as `Union[str, ModuleType]`.

importlib.resources.files(anchor: Anchor | None = None)

Returns a `Traversable` object representing the resource container (think directory) and its resources (think files). A `Traversable` may contain other containers (think subdirectories). `anchor` is an optional `Anchor`. If the anchor is a package, resources are resolved from that package. If a module, resources are resolved adjacent to that module (in the same package or the package root). If the anchor is omitted, the caller's module is used. Added in version 3.9. Changed in version 3.12: `package` parameter was

importlib.resources.as_file(traversable)

Given a `Traversable` object representing a file or directory, typically from `importlib.resources.files()`, return a context manager for use in a `with` statement. The context manager provides a `pathlib.Path` object. Exiting the context manager cleans up any temporary file or directory created when the resource was extracted from e.g. a zip file. Use `as_file` when the `Traversable` methods (`read_text`, etc) are insufficient and an actual file or directory on the file system is required. Added in version 3.9. Changed in

Functional API

A set of simplified, backwards-compatible helpers is available. These allow common operations in a single function call.

For all the following functions:

- `anchor` is an `Anchor`, as `infiles()`. Unlike `infiles`, it may not be omitted.
- `path_names` are components of a resource's path name, relative to the anchor. For example, to get the text of resource named `info.txt`, use: `importlib.resources.read_text(my_module, "info.txt")` Like `Traversable.joinpath`, The individual components should use forward slashes (/) as path separators. For example, the following are equivalent: `importlib.resources.read_binary(my_module, "pics/painting.png")` `import`

`importlib.resources.open_binary(anchor,*path_names)`

Open the named resource for binary reading. See the introduction for details on `anchor` and `path_names`. This function returns a `BinaryIO` object, that is, a binary stream open for reading. This function is roughly equivalent to: `files(anchor).joinpath(*path_names).open('rb')` Changed in version 3.13: Multiple `path_names` are accepted.

`importlib.resources.open_text(anchor,*path_names,encoding='utf-8',errors='strict')`

Open the named resource for text reading. By default, the contents are read as strict UTF-8. See the introduction for details on `anchor` and `path_names`. `encoding` and `errors` have the same meaning as in built-in `open()`. For backward compatibility reasons, the `encoding` argument must be given explicitly if there are multiple `path_names`. This limitation is scheduled to be removed in Python 3.15. This function returns a `TextIO` object, that is, a text stream open for reading. This function is roughly equivalent to: `files(a`

`importlib.resources.read_binary(anchor,*path_names)`

Read and return the contents of the named resource as bytes. See the introduction for details on `anchor` and `path_names`. This function is roughly equivalent to: `files(anchor).joinpath(*path_names).read_bytes()` Changed in version 3.13: Multiple `path_names` are accepted.

`importlib.resources.read_text(anchor,*path_names,encoding='utf-8',errors='strict')`

Read and return the contents of the named resource as str. By default, the contents are read as strict UTF-8. See the introduction for details on `anchor` and `path_names`. `encoding` and `errors` have the same meaning as in built-in `open()`. For backward compatibility reasons, the `encoding` argument must be given explicitly if there are multiple `path_names`. This limitation is scheduled to be removed in Python 3.15. This function is roughly equivalent to: `files(anchor).joinpath(*path_names).read_text(encoding=encoding)` Chan

`importlib.resources.path(anchor,*path_names)`

Provides the path to the resource as an actual file system path. This function returns a context manager for use in a `with` statement. The context manager provides a `pathlib.Path` object. Exiting the context manager cleans up any temporary files created, e.g. when the resource needs to be extracted from a zip file. For example, the `stat()` method requires an actual file system path; it can be used like this: `with importlib.resources.path(anchor, "resource.txt") as fspath: result = fspath.stat()` See the introduction fo

`importlib.resources.is_resource(anchor,*path_names)`

Return `True` if the named resource exists, otherwise `False`. This function does not consider directories to be resources. See the introduction for details on `anchor` and `path_names`. This function is roughly equivalent to: `files(anchor).joinpath(*path_names).is_file()` Changed in version 3.13: Multiple `path_names` are accepted.

`importlib.resources.contents(anchor,*path_names)`

Return an iterable over the named items within the package or path. The iterable returns names of resources (e.g. files) and non-resources (e.g. directories) as str. The iterable does not recurse into subdirectories. See the introduction for details on `anchor` and `path_names`. This function is roughly equivalent to: `for resource in files(anchor).joinpath(*path_names).iterdir(): yield resource.name` `Deprecated since version 3.11: Prefer iterdir() as above, which offers more control over the results and richer functio`

importlib.resources.abc– Abstract base classes for resources

Source code: `Lib/importlib/resources/abc.py`

Added in version 3.11.

class `importlib.resources.abc.ResourceReader`

abstractmethod `open_resource(resource)`

abstractmethod `resource_path(resource)`

abstractmethod `is_resource(path)`

abstractmethod `contents()`

Superseded by `TraversableResources`. An abstract base class to provide the ability to read resources. From the perspective of this ABC, a resource is a binary artifact that is shipped within a package. Typically this is something like a data file that lives next to the `__init__.py` file of the package. The purpose of this class is to help abstract out the accessing of such data files so that it does not matter if the package and its data file(s) are stored e.g. in a zip file versus on the file system. For an

Returns an opened, file-like object for binary reading of the resource. If the resource cannot be found, `FileNotFoundError` is raised.

Returns the file system path to the resource. If the resource does not concretely exist on the file system, `raise FileNotFoundError`.

Returns `True` if the named path is considered a resource. `FileNotFoundError` is raised if path does not exist. Changed in version 3.10: The argument `name` was renamed to `path`.

Returns an iterable of strings over the contents of the package. Do note that it is not required that all names returned by the iterator be actual resources, e.g. it is acceptable to return names for which `is_resource()` would be `false`. Allowing non-resource names to be returned is to allow for situations where how a package and its resources are stored are known a priori and the non-resource names would be useful. For instance, returning subdirectory names is allowed so that when it is known that the

class `importlib.resources.abc.Traversable`

name

abstractmethod `iterdir()`

abstractmethod `is_dir()`

abstractmethod `is_file()`

abstractmethod `joinpath(*pathsegments)`

abstractmethod `__truediv__(child)`

abstractmethod `open(mode='r', *args, **kwargs)`

`read_bytes()`

`read_text(encoding=None)`

An object with a subset of `pathlib.Path` methods suitable for traversing directories and opening files. For a representation of the object on the file-system, use `importlib.resources.as_file().name`. Abstract. The base name of this object without any parent references. `abstractmethod iterdir()` Yield `Traversable` objects in `self`. `abstractmethod is_dir()` Return `True` if `self` is a directory. `abstractmethod is_file()` Return `True` if `self` is a

`file.abspathmethodjoinpath(*pathsegments)` Traverse directories according to path segments

Abstract. The base name of this object without any parent references.

Yield Traversable objects in self.

Return True if self is a directory.

Return True if self is a file.

Traverse directories according to path segments and return the result as a Traversable. Each path segments argument may contain multiple names separated by forward slashes (`/`, `posixpath.sep`). For example, the following are equivalent: `files.joinpath('subdir', 'subsudir', 'file.txt')` `files.joinpath('subdir/subsudir/file.txt')` Note that some Traversable implementations might not be updated to the latest version of the protocol. For compatibility with such implementations, provide a single argument without path segments

Return Traversable child in self. Equivalent to `joinpath(child)`.

mode may be 'r' or 'rb' to open as text or binary. Return a handle suitable for reading (same as `pathlib.Path.open`). When opening as text, accepts encoding parameters such as those accepted by `io.TextIOWrapper`.

Read contents of self as bytes.

Read contents of self as text.

`class importlib.resources.abc.TraversableResources`

`abstractmethod files()`

An abstract base class for resource readers capable of serving the `importlib.resources.files()` interface. Subclasses `ResourceReader` and provides concrete implementations of the `ResourceReader`'s abstract methods. Therefore, any loader supplying `TraversableResources` also supplies `ResourceReader`. Loaders that wish to support resource reading are expected to implement this interface. `abstractmethod files()` Returns `importlib.resources.abc.TraversableObject` for the loaded package.

Returns `importlib.resources.abc.TraversableObject` for the loaded package.

importlib.metadata– Accessing package metadata

Added in version 3.8.

Changed in version 3.10: `importlib.metadata` is no longer provisional.

Source code: `Lib/importlib/metadata/__init__.py`

`importlib.metadata` is a library that provides access to the metadata of an installed `Distribution` Package, such as its entry points or its top-level names (Import Packages, modules, if any). Built in part on Python's import system, this library intends to replace similar functionality in the `entry point API` and `metadata API` of `pkg_resources`. Along with `importlib.resources`, this package can eliminate the need to use the older and less efficient `pkg_resources` package.

`importlib.metadata` operates on third-party distribution packages installed into Python's `site-packages` directory via tools such as `pip`. Specifically, it works with distributions with discoverable `dist-info`, `egg-info` directories, and metadata defined by the `Core` metadata specifications.

Important

These are not necessarily equivalent to or correspond 1:1 with the top-level import package names that can be imported inside Python code. One distribution package can contain multiple import packages (and single modules), and one top-level import package may map to multiple distribution packages if it is a namespace package. You can use `packages_distributions()` to get a mapping between them.

By default, distribution metadata can live on the file system or in zip archives on `sys.path`. Through an extension mechanism, the metadata can live almost anywhere.

See also

<https://importlib-metadata.readthedocs.io/>

The documentation for `importlib_metadata`, which supplies a backport of `importlib.metadata`. This includes an API reference for this module's classes and functions, as well as a migration guide for existing users of `pkg_resources`.

Overview

Let's say you wanted to get the version string for a `Distribution` Package you've installed using `pip`. We start by creating a virtual environment and installing something into it:

```
$ python -m venv example
$ source example/bin/activate
(example) $ python -m pip install wheel
```

You can get the version string for `wheel` by running the following:

```
(example) $ python
>>> from importlib.metadata import version
>>> version('wheel')
'0.32.3'
```

You can also get a collection of entry points selectable by properties of the `EntryPoint` (typically 'group' or 'name'), such as `console_scripts`, `distutils.commands` and others. Each group contains a collection of `EntryPoint` objects.

You can get the metadata for a distribution:

```
>>> list(metadata('wheel'))
['Metadata-Version', 'Name', 'Version', 'Summary', 'Home-page', 'Author', 'Author-email', 'Maintainer', 'Maintainer-email']
```


You can also get a distribution's version number, list its constituent files, and get a list of the distribution's Distribution requirements.

exceptionimportlib.metadata.PackageNotFoundError

Subclass of `ModuleNotFoundError` raised by several functions in this module when queried for a distribution package which is not installed in the current Python environment.

Functional API

This package provides the following functionality via its public API.

Entry points

importlib.metadata.entry_points(select_params)**

Returns a `EntryPoint` instance describing entry points for the current environment. Any given keyword parameters are passed to the `select()` method for comparison to the attributes of the individual entry point definitions. Note: it is not currently possible to query for entry points based on their `EntryPoint.dist` attribute (as different `Distribution` instances do not currently compare equal, even if they have the same attributes)

classimportlib.metadata.EntryPoints

Details of a collection of installed entry points. Also provides a `groups` attribute that reports all identified entry point groups, and a `names` attribute that reports all identified entry point names.

classimportlib.metadata.EntryPoint

Details of an installed entry point. Each `EntryPoint` instance has `name`, `group`, and `value` attributes and a `load()` method to resolve the value. There are also `module`, `attr`, and `extras` attributes for getting the components of the `value` attribute, and `dist` for obtaining information regarding the distribution package that provides the entry point.

Query all entry points:

```
>>> eps = entry_points()
```

The `entry_points()` function returns a `EntryPoint` object, a collection of all `EntryPoint` objects with `names` and `groups` attributes for convenience:

```
>>> sorted(eps.groups)
['console_scripts', 'distutils.commands', 'distutils.setup_keywords', 'egg_info.writers', 'setuptools.installat
```

`EntryPoint` has a `select()` method to select entry points matching specific properties. Select entry points in the `console_scripts` group:

```
>>> scripts = eps.select(group='console_scripts')
```

Equivalently, since `entry_points()` passes keyword arguments through to `select`:

```
>>> scripts = entry_points(group='console_scripts')
```

Pick out a specific script named "wheel" (found in the wheel project):

```
>>> 'wheel' in scripts.names
True
>>> wheel = scripts['wheel']
```

Equivalently, query for that entry point during selection:

```
>>> (wheel,) = entry_points(group='console_scripts', name='wheel')
>>> (wheel,) = entry_points().select(group='console_scripts', name='wheel')
```

Inspect the resolved entry point:

```
>>> wheel
EntryPoint(name='wheel', value='wheel.cli:main', group='console_scripts')
>>> wheel.module
'wheel.cli'
>>> wheel.attr
'main'
>>> wheel.extras
[]
>>> main = wheel.load()
>>> main
<function main at 0x103528488>
```

The group and name are arbitrary values defined by the package author and usually a client will wish to resolve all entry points for a particular group. Read the [setuptools docs](#) for more information on entry points, their definition, and usage.

Changed in version 3.12: The “selectable” entry points were introduced in `importlib_metadata` 3.6 and Python 3.10. Prior to those changes, `entry_points` accepted no parameters and always returned a dictionary of entry points, keyed by group. With `importlib_metadata` 5.0 and Python 3.12, `entry_points` always returns an `EntryPoint` object. See [backports.entry_points_selectable](#) for compatibility options.

Changed in version 3.13: `EntryPoint` objects no longer present a tuple-like interface (`__getitem__()`).

Distribution metadata

`importlib.metadata.metadata(distribution_name)`

Return the distribution metadata corresponding to the named distribution package as a `PackageMetadata` instance. Raises `PackageNotFoundError` if the named distribution package is not installed in the current Python environment.

`class importlib.metadata.PackageMetadata`

A concrete implementation of the `PackageMetadata` protocol. In addition to providing the defined protocol methods and attributes, subscripting the instance is equivalent to calling the `get()` method.

Every Distribution Package includes some metadata, which you can extract using the `metadata()` function:

```
>>> wheel_metadata = metadata('wheel')
```

The keys of the returned data structure name the metadata keywords, and the values are returned unparsed from the distribution metadata:

```
>>> wheel_metadata['Requires-Python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
```

`PackageMetadata` also presents `jsonattribute` that returns all the metadata in a JSON-compatible form per [PEP 566](#):

```
>>> wheel_metadata.json['requires_python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
```

The full set of available metadata is not described here. See the [PyPA Core metadata specification](#) for additional details.

Changed in version 3.10: The `Description` is now included in the metadata when presented through the payload. Line continuation characters have been removed.

The `jsonattribute` was added.

Distribution versions

`importlib.metadata.version(distribution_name)`

Return the installed distribution package version for the named distribution package. Raises `PackageNotFoundError` if the named distribution package is not installed in the current Python environment.

The `version()` function is the quickest way to get a Distribution Package's version number, as a string:

```
>>> version('wheel')
'0.32.3'
```

Distribution files

`importlib.metadata.files(distribution_name)`

Return the full set of files contained within the named distribution package. Raises `PackageNotFoundError` if the named distribution package is not installed in the current Python environment. Returns `None` if the distribution is found but the installation database records reporting the files associated with the distribution package are missing.

`class importlib.metadata.PackagePath`

A `pathlib.PurePath` derived object with additional `dist`, `size`, and `hash` properties corresponding to the distribution package's installation metadata for that file.

The `files()` function takes a Distribution Package name and returns all of the files installed by this distribution. Each file is reported as a `PackagePath` instance. For example:

```
>>> util = [p for p in files('wheel') if 'util.py' in str(p)][0]
>>> util
PackagePath('wheel/util.py')
>>> util.size
859
>>> util.dist
<importlib.metadata._hooks.PathDistribution object at 0x101e0cef0>
>>> util.hash
<FileHash mode: sha256 value: bYkw5oMccfazVCoYQwKkkemoVyMAFoR34mmKBx8R1NI>
```

Once you have the file, you can also read its contents:

```
>>> print(util.read_text())
import base64
import sys
...
def as_bytes(s):
    if isinstance(s, text_type):
        return s.encode('utf-8')
    return s
```

You can also use the `locate()` method to get the absolute path to the file:

```
>>> util.locate()
PosixPath('/home/gustav/example/lib/site-packages/wheel/util.py')
```

In the case where the metadata file listing files (`RECORD` or `SOURCES.txt`) is missing, `files()` will return `None`. The caller may wish to wrap calls to `files()` in `always_iterable` or otherwise guard against this condition if the target distribution is not known to have the metadata present.

Distribution requirements

`importlib.metadata.requires(distribution_name)`

Return the declared dependency specifiers for the named distribution package. Raises `PackageNotFoundError` if the named distribution package is not installed in the current Python environment.

To get the full set of requirements for a Distribution Package, use the `requires()` function:

```
>>> requires('wheel')
["pytest (>=3.0.0) ; extra == 'test'", "pytest-cov ; extra == 'test'"]
```

Mapping import to distribution packages

`importlib.metadata.packages_distributions()`

Return a mapping from the top level module and import package names found via `sys.meta_path` to the names of the distribution packages (if any) that provide the corresponding files. To allow for namespace packages (which may have members provided by multiple distribution packages), each top level import name maps to a list of distribution names rather than mapping directly to a single name.

A convenience method to resolve the Distribution Package name (or names, in the case of a namespace package) that provide each importable top-level Python module or Import Package:

```
>>> packages_distributions()
{'importlib_metadata': ['importlib-metadata'], 'yaml': ['PyYAML'], 'jaraco': ['jaraco.classes', 'jaraco.function']}
```

Some editable installs, do not supply top-level names, and thus this function is not reliable with such installs.

Added in version 3.10.

Distributions

`importlib.metadata.distribution(distribution_name)`

Return a `Distribution` instance describing the named distribution package. Raises `PackageNotFoundError` if the named distribution package is not installed in the current Python environment.

`class importlib.metadata.Distribution`

Details of an installed distribution package. Note: different `Distribution` instances do not currently compare equal, even if they relate to the same installed distribution and accordingly have the same attributes.

While the module level API described above is the most common and convenient usage, you can get all of that information from the `Distribution` class. `Distribution` is an abstract object that represents the metadata for a Python Distribution Package. You can get the concrete `Distributions` subclass instance for an installed distribution package by calling the `distribution()` function:

```
>>> from importlib.metadata import distribution
>>> dist = distribution('wheel')
>>> type(dist)
<class 'importlib.metadata.PathDistribution'>
```

Thus, an alternative way to get the version number is through the `Distribution` instance:

```
>>> dist.version
'0.32.3'
```

There are all kinds of additional metadata available on `Distribution` instances:

```
>>> dist.metadata['Requires-Python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
>>> dist.metadata['License']
'MIT'
```

For editable packages, an `origin` property may present PEP 610 metadata:

```
>>> dist.origin.url
'file:///path/to/wheel-0.32.3.editable-py3-none-any.whl'
```

The full set of available metadata is not described here. See the PyPACore metadata specification for additional details.

Added in version 3.13: The `origin` property was added.

Distribution Discovery

By default, this package provides built-in support for discovery of metadata for file system and zip file Distribution Packages. This metadata finder search defaults to `sys.path`, but varies slightly in how it interprets those values from how other import machinery does. In particular:

- `importlib.metadata` does not honor bytes objects on `sys.path`.
- `importlib.metadata` will incidentally honor `pathlib.Path` objects on `sys.path` even though such values will be ignored for imports.

Implementing Custom Providers

`importlib.metadata` address two API surfaces, one for consumers and another for providers. Most users are consumers, consuming metadata provided by the packages. There are other use-cases, however, where users wish to expose metadata through some other mechanism, such as alongside a custom importer. Such a use case calls for a custom provider.

Because Distribution Package metadata is not available through `sys.path` searches, or package loaders directly, the metadata for a distribution is found through import system finders. To find a distribution package's metadata, `importlib.metadata` queries the list of meta path finders on `sys.meta_path`.

The implementation has hooks integrated into the `PathFinder`, serving metadata for distribution packages found on the file system.

The abstract class `importlib.abc.MetaPathFinder` defines the interface expected of finders by Python's import system. `importlib.metadata` extends this protocol by looking for an optional `find_distributions` callable on the finders from `sys.meta_path` and presents this extended interface as the `DistributionFinder` abstract base class, which defines this abstract method:

```
@abc.abstractmethod
def find_distributions(context=DistributionFinder.Context()) -> Iterable[Distribution]:
    """Return an iterable of all Distribution instances capable of
    loading the metadata for packages for the indicated ``context``.
    """
```

The `DistributionFinder.Context` object provides `path` and `name` properties indicating the path to search and name to match and may supply other relevant context sought by the consumer.

In practice, to support finding distribution package metadata in locations other than the file system, subclass `Distribution` and implement the abstract methods. Then from a custom finder, return instances of this derived `Distribution` in the `find_distributions()` method.

Example

Imagine a custom finder that loads Python modules from a database:

```
class DatabaseImporter(importlib.abc.MetaPathFinder):
    def __init__(self, db):
        self.db = db

    def find_spec(self, fullname, target=None) -> ModuleSpec:
        return self.db.spec_from_name(fullname)
```

```
sys.meta_path.append(DatabaseImporter(connect_db(...)))
```

That importer now presumably provides importable modules from a database, but it provides no metadata or entry points. For this custom importer to provide metadata, it would also need to implement `DistributionFinder`:

```
from importlib.metadata import DistributionFinder

class DatabaseImporter(DistributionFinder):
    ...

    def find_distributions(self, context=DistributionFinder.Context()):
        query = dict(name=context.name) if context.name else {}
        for dist_record in self.db.query_distributions(query):
            yield DatabaseDistribution(dist_record)
```

In this way, `query_distributions` would return records for each distribution served by the database matching the query. For example, if `requests-1.0` is in the database, `find_distributions` would yield a `DatabaseDistribution` for `Context(name='requests')` or `Context(name=None)`.

For the sake of simplicity, this example ignores `context.path`. The `path` attribute defaults to `sys.path` and is the set of import paths to be considered in the search. A `DatabaseImporter` could potentially function without any concern for a search path. Assuming the importer does no partitioning, the “path” would be irrelevant. In order to illustrate the purpose of `path`, the example would need to illustrate a more complex `DatabaseImporter` whose behavior varied depending on `sys.path`/`PYTHONPATH`. In that case, the `find_distributions` should honor the `context.path` and only yield `Distributions` pertinent to that path.

`DatabaseDistribution`, then, would look something like:

```
class DatabaseDistribution(importlib.metadata.Distribution):
    def __init__(self, record):
        self.record = record

    def read_text(self, filename):
        """
        Read a file like "METADATA" for the current distribution.
        """
        if filename == "METADATA":
            return f"""Name: {self.record.name}
Version: {self.record.version}
"""
        if filename == "entry_points.txt":
            return "\n".join(
                f"[{ep.group}]\n{ep.name}={ep.value}"
                for ep in self.record.entry_points)

    def locate_file(self, path):
        raise RuntimeError("This distribution has no file system")
```

This basic implementation should provide metadata and entry points for packages served by the `DatabaseImporter`, assuming that the records supply suitable `name`, `version`, and `entry_points` attributes.

The `DatabaseDistribution` may also provide other metadata files, like `RECORD` (required for `Distribution.files`) or override the implementation of `Distribution.files`. See the source for more inspiration.

The initialization of `sys.path` search path

A module search path is initialized when Python starts. This module search path may be accessed at `sys.path`.

The first entry in the module search path is the directory that contains the input script, if there is one. Otherwise, the first entry is the current directory, which is the case when executing the interactive shell, `a-c` command, or `m` module.

The `PYTHONPATH` environment variable is often used to add directories to the search path. If this environment variable is found then the contents are added to the module search path.

■ Nota: Note `PYTHONPATH` will affect all installed Python versions/environments. Be wary of setting this in your shell profile or global environment variables. This module offers more nuanced techniques as mentioned below.

The next items added are the directories containing standard Python modules as well as any extension modules that these modules depend on. Extension modules are `.pyd` files on Windows and `.so` files on other platforms. The directory with the platform-independent Python modules is called `prefix`. The directory with the extension modules is called `exec_prefix`.

The `PYTHONHOME` environment variable may be used to set the `prefix` and `exec_prefix` locations. Otherwise these directories are found by using the Python executable as a starting point and then looking for various 'landmark' files and directories. Note that any symbolic links are followed so the real Python executable location is used as the search starting point. The Python executable location is called `home`.

Once `home` is determined, the `prefix` directory is found by first looking for `pythonmajorversionminorversion.zip` (python311.zip). On Windows the zip archive is searched for in `home` and on Unix the archive is expected to be in `lib`. Note that the expected zip archive location is added to the module search path even if the archive does not exist. If no archive was found, Python on Windows will continue the search for `prefix` by looking for `Lib\os.py`. Python on Unix will look for `lib/pythonmajorversion.minorversion/os.py` (`lib/python3.11/os.py`). On Windows `prefix` and `exec_prefix` are the same, however on other platforms `lib/pythonmajorversion.minorversion/lib-dynload` (`lib/python3.11/lib-dynload`) is searched for and used as an anchor for `exec_prefix`. On some platforms `lib` may be `lib64` or another value, see `sys.platlibdir` and `PYTHONPLATLIBDIR`.

Once found, `prefix` and `exec_prefix` are available at `sys.base_prefix` and `sys.base_exec_prefix` respectively.

If `PYTHONHOME` is not set, and `pyvenv.cfg` file is found alongside the main executable, or in its parent directory, `sys.prefix` and `sys.exec_prefix` get set to the directory containing `pyvenv.cfg`, otherwise they are set to the same value as `sys.base_prefix` and `sys.base_exec_prefix`, respectively. This is used by Virtual Environments.

Finally, this module is processed and `site-packages` directories are added to the module search path. A common way to customize the search path is to create `sitecustomize` or `usercustomize` modules as described in this module documentation.

■ Nota: Note Certain command line options may further affect path calculations. See `-E`, `-I`, `-s` and `-S` for further details.

Changed in version 3.14: `sys.prefix` and `sys.exec_prefix` are now set to the `pyvenv.cfg` directory during the path initialization. This was previously done by `site`, therefore affected by `-S`.

Virtual Environments

Virtual environments place `pyvenv.cfg` file in their `prefix`, which causes `sys.prefix` and `sys.exec_prefix` to point to them, instead of the base installation.

The `prefix` and `exec_prefix` values of the base installation are available at `sys.base_prefix` and `sys.base_exec_prefix`.

As well as being used as a marker to identify virtual environments, `pyvenv.cfg` may also be used to configure the site initialization. Please refer to site's virtual environments documentation.

■ **Note:** `PYTHONHOME` overrides the `pyvenv.cfg` detection.

■ **Note:** There are other ways how “virtual environments” could be implemented, this documentation refers implementations based on the `pyvenv.cfg` mechanism, such as `venv`. Most virtual environment implementations follow the model set by `venv`, but there may be exotic implementations that diverge from it.

`_pth` files

To completely override `sys.path` create a `_pth` file with the same name as the shared library or executable (python.`_pth` or python311.`_pth`). The shared library path is always known on Windows, however it may not be available on other platforms. In the `_pth` file specify one line for each path to add to `sys.path`. The file based on the shared library name overrides the one based on the executable, which allows paths to be restricted for any program loading the runtime if desired.

When the file exists, all registry and environment variables are ignored, isolated mode is enabled, and `site` is not imported unless one line in the file specifies `import site`. Blank paths and lines starting with `#` are ignored. Each path may be absolute or relative to the location of the file. Import statements other than `site` are not permitted, and arbitrary code cannot be specified.

Note that `.pth` files (without leading underscore) will be processed normally by the `site` module when `import site` has been specified.

Embedded Python

If Python is embedded within another application `Py_InitializeFromConfig()` and the `PyConfig` structure can be used to initialize Python. The path specific details are described at [Python Path Configuration](#).

See also

- [Finding modules](#) for detailed Windows notes.
- [Using Python on Unix platforms](#) for Unix details.

Python Language Services

Python provides a number of modules to assist in working with the Python language. These modules support tokenizing, parsing, syntax analysis, bytecode disassembly, and various other facilities.

These modules include:

- `ast`— Abstract syntax treesAbstract grammarNode classesRoot nodesLiteralsVariablesExpressionsSubscriptingComprehensionsStatementsImportsControl flowPattern matchingType annotationsType parametersFunction and class definitionsAsync and awaitasthelpersCompiler flagsCommand-line usage
- `symtable`— Access to the compiler's symbol tablesGenerating Symbol TablesExamining Symbol TablesCommand-Line Usage
- `token`— Constants used with Python parse trees
- `keyword`— Testing for Python keywords
- `tokenize`— Tokenizer for Python sourceTokenizing InputCommand-Line UsageExamples
- `tabnanny`— Detection of ambiguous indentation
- `pyclbr`— Python module browser supportFunction ObjectsClass Objects
- `py_compile`— Compile Python source filesCommand-Line Interface
- `compileall`— Byte-compile Python librariesCommand-line usePublic functions
- `dis`— Disassembler for Python bytecodeCommand-line interfaceBytecode analysisAnalysis functionsPython Bytecode InstructionsOpcode collections
- `pickletools`— Tools for pickle developersCommand-line usageCommand-line optionsProgrammatic interface

ast— Abstract syntax trees

Source code: `Lib/ast.py`

The `ast` module helps Python applications to process trees of the Python abstract syntax grammar. The abstract syntax itself might change with each Python release; this module helps to find out programmatically what the current grammar looks like.

An abstract syntax tree can be generated by passing `ast.PyCF_ONLY_AST` as a flag to the `compile()` built-in function, or using the `parse()` helper provided in this module. The result will be a tree of objects whose classes all inherit from `ast.AST`. An abstract syntax tree can be compiled into a Python code object using the built-in `compile()` function.

Abstract grammar

The abstract grammar is currently defined as follows:

```
-- ASDL's 4 builtin types are:
-- identifier, int, string, constant
```

```
module Python
{
    mod = Module(stmt* body, type_ignore* type_ignores)
        | Interactive(stmt* body)
        | Expression(expr body)
        | FunctionType(expr* argtypes, expr returns)

    stmt = FunctionDef(identifier name, arguments args,
                        stmt* body, expr* decorator_list, expr? returns,
                        string? type_comment, type_param* type_params)
        | AsyncFunctionDef(identifier name, arguments args,
                           stmt* body, expr* decorator_list, expr? returns,
                           string? type_comment, type_param* type_params)

        | ClassDef(identifier name,
                   expr* bases,
                   keyword* keywords,
                   stmt* body,
                   expr* decorator_list,
                   type_param* type_params)
        | Return(expr? value)

        | Delete(expr* targets)
        | Assign(expr* targets, expr value, string? type_comment)
        | TypeAlias(expr name, type_param* type_params, expr value)
        | AugAssign(expr target, operator op, expr value)
        -- 'simple' indicates that we annotate simple name without parens
        | AnnAssign(expr target, expr annotation, expr? value, int simple)

        -- use 'orelse' because else is a keyword in target languages
        | For(expr target, expr iter, stmt* body, stmt* orelse, string? type_comment)
        | AsyncFor(expr target, expr iter, stmt* body, stmt* orelse, string? type_comment)
        | While(expr test, stmt* body, stmt* orelse)
        | If(expr test, stmt* body, stmt* orelse)
        | With(withitem* items, stmt* body, string? type_comment)
        | AsyncWith(withitem* items, stmt* body, string? type_comment)

        | Match(expr subject, match_case* cases)

        | Raise(expr? exc, expr? cause)
```

```

| Try(stmt* body, excepthandler* handlers, stmt* orelse, stmt* finalbody)
| TryStar(stmt* body, excepthandler* handlers, stmt* orelse, stmt* finalbody)
| Assert(expr test, expr? msg)

| Import(alias* names)
| ImportFrom(identifier? module, alias* names, int? level)

| Global(identifier* names)
| Nonlocal(identifier* names)
| Expr(expr value)
| Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string the parser uses
attributes (int lineno, int col_offset, int? end_lineno, int? end_col_offset)

-- BoolOp() can use left & right?
expr = BoolOp(boolop op, expr* values)
| NamedExpr(expr target, expr value)
| BinOp(expr left, operator op, expr right)
| UnaryOp(unaryop op, expr operand)
| Lambda(arguments args, expr body)
| IfExp(expr test, expr body, expr orelse)
| Dict(expr?* keys, expr* values)
| Set(expr* elts)
| ListComp(expr elt, comprehension* generators)
| SetComp(expr elt, comprehension* generators)
| DictComp(expr key, expr value, comprehension* generators)
| GeneratorExp(expr elt, comprehension* generators)
-- the grammar constrains where yield expressions can occur
| Await(expr value)
| Yield(expr? value)
| YieldFrom(expr value)
-- need sequences for compare to distinguish between
-- x < 4 < 3 and (x < 4) < 3
| Compare(expr left, cmpop* ops, expr* comparators)
| Call(expr func, expr* args, keyword* keywords)
| FormattedValue(expr value, int conversion, expr? format_spec)

```

Node classes

class ast.AST

_fields

_field_types

lineno

col_offset

end_lineno

end_col_offset

This is the base of all AST node classes. The actual node classes are derived from the `Parser/Python.asdlfile`, which is reproduced above. They are defined in the `_astC` module and re-exported in `ast`. There is one class defined for each left-hand side symbol in the abstract grammar (for example, `ast.stmt` or `ast.expr`). In addition, there is one class defined for each constructor on the right-hand side; these classes inherit from the classes for the left-hand side trees. For example, `ast.BinOp` inherits from

Each concrete class has an attribute `_fields` which gives the names of all child nodes. Each instance of a concrete class has one attribute for each child node, of the type as defined in the grammar. For

example, `ast.BinOp` instances have an attribute `left` of type `ast.expr`. If these attributes are marked as optional in the grammar (using a question mark), the value might be `None`. If the attributes can have zero-or-more values (marked with an asterisk), the values are represented as Python lists. All poss

The `_field_types` attribute on each concrete class is a dictionary mapping field names (as also listed in `_fields`) to their types. >>> `ast.TypeVar._field_types`{'name': <class 'str'>, 'bound': `ast.expr | None`, 'default_value': `ast.expr | None`} Added in version 3.13.

Instances of `ast.expr` and `ast.stmt` subclasses have `lineno`, `col_offset`, `end_lineno`, and `end_col_offset` attributes. The `lineno` and `end_lineno` are the first and last line numbers of source text span (1-indexed so the first line is line 1) and the `col_offset` and `end_col_offset` are the corresponding UTF-8 byte offsets of the first and last tokens that generated the node. The UTF-8 offset is recorded because the parser uses UTF-8 internally. Note that the end positions are not required by the compiler and are therefor

Changed in version 3.8: Class `ast.Constant` is now used for all constants.

Changed in version 3.9: Simple indices are represented by their value, extended slices are represented as tuples.

Changed in version 3.13: AST node constructors were changed to provide sensible defaults for omitted fields: optional fields now default to `None`, list fields default to an empty list, and fields of type `ast.expr` default to `Load()`. Previously, omitted attributes would not exist on constructed nodes (accessing them raised `AttributeError`).

Changed in version 3.14: The `__repr__()` output of AST nodes includes the values of the node fields.

Deprecated since version 3.8, removed in version 3.14: Previous versions of Python provided the AST classes `ast.Num`, `ast.Str`, `ast.Bytes`, `ast.NameConstant` and `ast.Ellipsis`, which were deprecated in Python 3.8. These classes were removed in Python 3.14, and their functionality has been replaced with `ast.Constant`.

Deprecated since version 3.9: Old classes `ast.Index` and `ast.ExtSlice` are still available, but they will be removed in future Python releases. In the meantime, instantiating them will return an instance of a different class.

Deprecated since version 3.13, will be removed in version 3.15: Previous versions of Python allowed the creation of AST nodes that were missing required fields. Similarly, AST node constructors allowed arbitrary keyword arguments that were set as attributes of the AST node, even if they did not match any of the fields of the AST node. This behavior is deprecated and will be removed in Python 3.15.

■ Nota: Note The descriptions of the specific node classes displayed here were initially adapted from the fantastic Green Tree Snakes project and all its contributors.

Root nodes

class `ast.Module`(`body`, `type_ignores`)

A Python module, as with file input. Node type generated by `ast.parse()` in the default "exec" mode. `body` is a list of the module's `Statements`. `type_ignores` is a list of the module's type ignore comments; see `ast.parse()` for more details. >>> `print(ast.dump(ast.parse('x = 1'), indent=4))` `Module(body=[Assign(targets=[Name(id='x', ctx=Store())], value=Constant(value=1))])`

class `ast.Expression`(`body`)

A single Python expression input. Node type generated by `ast.parse()` when `mode` is "eval". `body` is a single node, one of the expression types. >>> `print(ast.dump(ast.parse('123', mode='eval'), indent=4))` `Expression(body=Constant(value=123))`

class `ast.Interactive`(`body`)

A single interactive input, like in Interactive Mode. Node type generated by `ast.parse()` when `mode` is "single". `body` is a list of statement nodes. >>> `print(ast.dump(ast.parse('x = 1; y =`

```
2', mode='single'), indent=4)) Interactive(body=[Assign(targets=[Name(id='x',
ctx=Store())], value=Constant(value=1)), Assign(targets=[Name(id='y', ctx=Store())], value=Constant(value=2))])
```

class ast.FunctionType(argtypes, returns)

A representation of an old-style type comments for functions, as Python versions prior to 3.5 didn't support PEP 484 annotations. Node type generated by `ast.parse()` when `mode` is `"func_type"`. Such type comments would look like this: `def sum_two_number(a, b): # type: (int, int) -> int` returns a single expression node. `>>> print(ast.dump(ast.parse('(int, str) -> List[int]', mode='func_type'), indent=4))` `FunctionType(argtypes=[Name(id='int', ctx=Load()), Name(id='str', ctx=`

Literals

class ast.Constant(value)

A constant value. The `value` attribute of the `Constant` literal contains the Python object it represents. The values represented can be instances of `str`, `bytes`, `int`, `float`, `complex`, and `bool`, and the constants `None` and `Ellipsis`. `>>> print(ast.dump(ast.parse('123', mode='eval'), indent=4))` `Expression(body=Constant(value=123))`

class ast.FormattedValue(value, conversion, format_spec)

Node representing a single formatting field in an f-string. If the string contains a single formatting field and nothing else the node can be isolated otherwise it appears in `JoinedStr.value` is any expression node (such as a literal, a variable, or a function call). `conversion` is an integer: `-1`: no formatting (ord('a'))! a ASCII formatting (ord('r'))! `repr()` formatting (ord('s'))! `s` string formatting `format_spec` is a `JoinedStr` node representing the formatting of the value, or `None` if no format was spec

class ast.JoinedStr(values)

An f-string, comprising a series of `FormattedValue` and `Constant` nodes. `>>> print(ast.dump(ast.parse('f"sin({a}) is {sin(a):.3}"', mode='eval'), indent=4))` `Expression(body=JoinedStr(values=[Constant(value='sin('), FormattedValue(value=Name(id='a', ctx=Load()), conversion=-1), Constant(value=') is '), FormattedValue(value=Call(func=Name(id='sin', ctx=Load()), args=[Name(id='a', ctx=Load())]), conversion=-1, format_spec=JoinedStr(values=[Constant(value='.3')]))]))`

class ast.TemplateStr(values, /)

Added in version 3.14. Node representing a template string literal, comprising a series of `Interpolation` and `Constant` nodes. These nodes may be any order, and do not need to be interleaved. `>>> expr = ast.parse('"{name} finished {place:ordinal}"', mode='eval') >>> print(ast.dump(expr, indent=4))` `Expression(body=TemplateStr(values=[Interpolation(value=Name(id='name', ctx=Load()), str='name', conversion=-1), Constant(value=' finished '), Interpolation(value=Name(id='place', ctx=Load()), str='place', conversion=-1, for`

class ast.Interpolation(value, str, conversion, format_spec=None)

Added in version 3.14. Node representing a single interpolation field in a template string literal. `value` is any expression node (such as a literal, a variable, or a function call). This has the same meaning as `FormattedValue`. `value.str` is a constant containing the text of the interpolation expression. If `str` is set to `None`, then `value` is used to generate code when calling `ast.unparse()`. This no longer guarantees that the generated code is identical to the original and is intended for code generation. `conversion`

class ast.List(elts, ctx)

class ast.Tuple(elts, ctx)

A list or tuple. `elts` holds a list of nodes representing the elements. `ctx` is `Store` if the container is an assignment target (i.e. `(x, y) = something`), and `Load` otherwise. `>>> print(ast.dump(ast.parse('[1, 2, 3]', mode='eval'), indent=4))` `Expression(body=List(elts=[Constant(value=1), Constant(value=2), Constant(value=3)], ctx=Load())) >>> print(ast.dump(ast.parse('(1, 2, 3)', mode='eval'), indent=4))` `Expression(body=Tuple(elts=[Constant(value=1), Constant(valu`

```
e=2),Constant(value=3)],ctx=Load()))
```

classast.Set(elts)

A set.elts holds a list of nodes representing the set's elements. >>>print(ast.dump(ast.parse('{1, 2, 3}', mode='eval'), indent=4))Expression(body=Set(elts=[Constant(value=1), Constant(value=2), Constant(value=3)]))

classast.Dict(keys, values)

A dictionary.keys and values hold lists of nodes representing the keys and the values respectively, in matching order (what would be returned when calling dictionary.keys() and dictionary.values()). When doing dictionary unpacking using dictionary literals the expression to be expanded goes in the values list, with a None at the corresponding position in keys. >>>print(ast.dump(ast.parse('{\"a\":1, **d}', mode='eval'), indent=4))Expression(body=Dict(keys=[Constant(value='a'), None], values=[Constant(value=1), Name(

Variables

classast.Name(id, ctx)

A variable name.id holds the name as a string, and ctx is one of the following types.

classast.Load

classast.Store

classast.Del

Variable references can be used to load the value of a variable, to assign a new value to it, or to delete it. Variable references are given a context to distinguish these

```
cases.>>>print(ast.dump(ast.parse('a'), indent=4))Module(body=[Expr(value=Name(id='a', ctx=Load()))])>>>print(ast.dump(ast.parse('a = 1'), indent=4))Module(body=[Assign(targets=[Name(id='a', ctx=Store())], value=Constant(value=1))])>>>print(ast.dump(ast.parse('del a'), indent=4))Module(body=[Delete(targets=[Name(id='a', ctx=Del())])])
```

classast.Starred(value, ctx)

A*var variable reference.value holds the variable, typically a Name node. This type must be used when building a Call node with *args. >>>print(ast.dump(ast.parse('a, *b = it'), indent=4))Module(body=[Assign(targets=[Tuple(elts=[Name(id='a', ctx=Store()), Starred(value=Name(id='b', ctx=Store()), ctx=Store())], value=Name(id='it', ctx=Load()))])])

Expressions

classast.Expr(value)

When an expression, such as a function call, appears as a statement by itself with its return value not used or stored, it is wrapped in this container.value holds one of the other nodes in this section, a Constant, a Name, a Lambda, a Yield or YieldFrom node. >>>print(ast.dump(ast.parse('-a'), indent=4))Module(body=[Expr(value=UnaryOp(op=USub(), operand=Name(id='a', ctx=Load())))])

classast.UnaryOp(op, operand)

A unary operation.op is the operator, and operand any expression node.

classast.UAdd

classast.USub

classast.Not

classast.Invert

Unary operator tokens. Note that `not` is a keyword, `invert` is the `~` operator. >>>print(ast.dump(ast.parse('not x', mode='eval'), indent=4))Expression(body=UnaryOp(op=Not(), operand=Name(id='x', ctx=Load())))

class ast.BinOp(left, op, right)

A binary operation (like addition or division). `op` is the operator, and `left` and `right` are any expression nodes. >>>print(ast.dump(ast.parse('x + y', mode='eval'), indent=4))Expression(body=BinOp(left=Name(id='x', ctx=Load()), op=Add(), right=Name(id='y', ctx=Load())))

class ast.Add

class ast.Sub

class ast.Mult

class ast.Div

class ast.FloorDiv

class ast.Mod

class ast.Pow

class ast.LShift

class ast.RShift

class ast.BitOr

class ast.BitXor

class ast.BitAnd

class ast.MatMult

Binary operator tokens.

class ast.BoolOp(op, values)

A boolean operation, `'or'` or `'and'`. `op` is `Or` or `And`. `values` are the values involved. Consecutive operations with the same operator, such as `a or b or c`, are collapsed into one node with several values. This doesn't include `not`, which is a `UnaryOp`. >>>print(ast.dump(ast.parse('x or y', mode='eval'), indent=4))Expression(body=BoolOp(op=Or(), values=[Name(id='x', ctx=Load()), Name(id='y', ctx=Load())]))

class ast.And

class ast.Or

Boolean operator tokens.

class ast.Compare(left, ops, comparators)

A comparison of two or more values. `left` is the first value in the comparison, `ops` the list of operators, and `comparators` the list of values after the first element in the comparison. >>>print(ast.dump(ast.parse('1 <= a < 10', mode='eval'), indent=4))Expression(body=Compare(left=Constant(value=1), ops=[LtE(), Lt()], comparators=[Name(id='a', ctx=Load()), Constant(value=10)]))

class ast.Eq

class ast.NotEq

class ast.Lt

class ast.LtE

class ast.Gt

classast.GtE

classast.Is

classast.IsNot

classast.In

classast.NotIn

Comparison operator tokens.

classast.Call(func,args,keywords)

A function call.`func` is the function, which will often be a `Name` or `Attribute` object. Of the arguments: `args` holds a list of the arguments passed by position. `keywords` holds a list of `keyword` objects representing arguments passed by keyword. The `args` and `keywords` arguments are optional and default to empty lists.

```
>>>print(ast.dump(ast.parse('func(a, b=c, *d, **e)',mode='eval'),indent=4))Expression(body=Call(func=Name(id='func', ctx=Load()),args=[Name(id='a', ctx=Load()),Starred(value=Name(id='d', ctx=Load()))],keywords=[keyword(id='b', value=Name(id='c', ctx=Load()), ctx=Load()),keyword(id='e', value=Name(id='e', ctx=Load()), ctx=Load())],ctx=Load()))
```

classast.keyword(arg,value)

A keyword argument to a function call or class definition. `arg` is a raw string of the parameter name, `value` is a node to pass in.

classast.IfExp(test,body,orelse)

An expression such as `a if b else c`. Each field holds a single node, so in the following example, all three are `Name` nodes.

```
>>>print(ast.dump(ast.parse('a if b else c',mode='eval'),indent=4))Expression(body=IfExp(test=Name(id='b', ctx=Load()),body=Name(id='a', ctx=Load()),orelse=Name(id='c', ctx=Load())))
```

classast.Attribute(value,attr,ctx)

Attribute access, e.g. `d.keys.value` is a node, typically a `Name`. `attr` is a bare string giving the name of the attribute, and `ctx` is `Load`, `Store` or `Del` according to how the attribute is acted on.

```
>>>print(ast.dump(ast.parse('snake.colour',mode='eval'),indent=4))Expression(body=Attribute(value=Name(id='snake', ctx=Load()),attr='colour',ctx=Load()))
```

classast.NamedExpr(target,value)

A named expression. This AST node is produced by the assignment expressions operator (also known as the walrus operator). As opposed to the `Assign` node in which the first argument can be multiple nodes, in this case both `target` and `value` must be single nodes.

```
>>>print(ast.dump(ast.parse('(x := 4)',mode='eval'),indent=4))Expression(body=NamedExpr(target=Name(id='x', ctx=Store()),value=Constant(value=4)))
```

Added in version 3.8.

Subscripting

classast.Subscript(value,slice,ctx)

A subscript, such as `sl[1]`. `value` is the subscripted object (usually sequence or mapping). `slice` is an index, slice or key. It can be a `Tuple` and contain a `Slice`. `ctx` is `Load`, `Store` or `Del` according to the action performed with the subscript.

```
>>>print(ast.dump(ast.parse('l[1:2, 3]',mode='eval'),indent=4))Expression(body=Subscript(value=Name(id='l', ctx=Load()),slice=Tuple(elts=[Slice(lower=Constant(value=1),upper=Constant(value=2)),Constant(value=3)],ctx=Load()),ctx=Load()))
```

classast.Slice(lower,upper,step)

Regular slicing (on the form `lower:upper` or `lower:upper:step`). Can occur only inside the `slice` field of `Subscript`, either directly or as an element of `Tuple`.

```
>>>print(ast.dump(ast.parse('l[1:2]',mode='eval'),indent=4))Expression(body=Subscript(value=Name(id='l', ctx=Load()),slice=Slice(lower=Constant(value=1),upper=Constant(value=2)),ctx=Load(),ctx=Load()))
```



```
body=Subscript(value=Name(id='l',
ctx=Load()),slice=Slice(lower=Constant(value=1),upper=Constant(value=2)),ctx=Load()))
```

Comprehensions

class ast.ListComp(elt, generators)

class ast.SetComp(elt, generators)

class ast.GeneratorExp(elt, generators)

class ast.DictComp(key, value, generators)

List and set comprehensions, generator expressions, and dictionary comprehensions. `elt` (or `key` and `value`) is a single node representing the part that will be evaluated for each item. `generators` is a list of comprehension nodes.

```
>>> print(ast.dump(...ast.parse('[x for x in numbers]', mode='eval', ...indent=4, ...)))
Expression(body=ListComp(elt=Name(id='x', ctx=Load()), generators=[comprehension(target=Name(id='x', ctx=Store()), iter=Name(id='numbers', ctx=Load()), is_async=0)]))
>>> print(ast.dump(...ast.parse('{x:
```

class ast.comprehension(target, iter, ifs, is_async)

One for clause in a comprehension. `target` is the reference to use for each element - typically a `Name` or `Tuple` node. `iter` is the object to iterate over. `ifs` is a list of test expressions: each for clause can have multiple ifs. `is_async` indicates a comprehension is asynchronous (using `async for` instead of `for`). The value is an integer (0 or 1).

```
>>> print(ast.dump(ast.parse('[ord(c) for line in file for c in line]', mode='eval', ...indent=4)))
# Multiple comprehensions in one.
Expression(body=ListComp(elt=Call(func=Name(
```

Statements

class ast.Assign(targets, value, type_comment)

type_comment

An assignment. `targets` is a list of nodes, and `value` is a single node. Multiple nodes in `targets` represents assigning the same value to each. Unpacking is represented by putting a `Tuple` or `List` with in `targets`. `type_comment` is an optional string with the type annotation as a comment.

```
>>> print(ast.dump(ast.parse('a = b = 1', indent=4)))
# Multiple assignment
Module(body=[Assign(targets=[Name(id='a', ctx=Store()), Name(id='b', ctx=Store())], value=Constant(value=1))])
>>> print(ast.dump(ast.parse('a, b = c',
```

`type_comment` is an optional string with the type annotation as a comment.

class ast.AnnAssign(target, annotation, value, simple)

An assignment with a type annotation. `target` is a single node and can be a `Name`, an `Attribute` or a `Subscript`. `annotation` is the annotation, such as a `Constant` or `Name` node. `value` is a single optional node. `simple` is always either 0 (indicating a “complex” target) or 1 (indicating a “simple” target). A “simple” target consists solely of a `Name` node that does not appear between parentheses; all other targets are considered complex. Only simple targets appear in the `__annotations__` dictionary of modules and classes.

```
>>
```

class ast.AugAssign(target, op, value)

Augmented assignment, such as `a += 1`. In the following example, `target` is a `Name` node for `x` (with the `Store` context), `op` is `Add`, and `value` is a `Constant` with value for 1. The `target` attribute cannot be of class `Tuple` or `List`, unlike the `targets` of `Assign`.

```
>>> print(ast.dump(ast.parse('x += 2', indent=4)))
Module(body=[AugAssign(target=Name(id='x', ctx=Store()), op=Add(), value=Constant(value=2))])
```

class ast.Raise(exc, cause)

Araisestatement.excis the exception object to be raised, normally aCallorName, orNonefor a standaloneraise.causeis the optional part foryinaraisexfromy.>>>print(ast.dump(ast.parse('raise x from y'),indent=4))Module(body=[Raise(exc=Name(id='x', ctx=Load()),cause=Name(id='y', ctx=Load()))])

classast.Assert(test,msg)

An assertion.testholds the condition, such as aComparenode.msgholds the failure message.>>>print(ast.dump(ast.parse('assert x,y'),indent=4))Module(body=[Assert(test=Name(id='x', ctx=Load()),msg=Name(id='y', ctx=Load()))])

classast.Delete(targets)

Represents adelstatement.targetsis a list of nodes, such asName,AttributeorSubscriptnodes.>>>print(ast.dump(ast.parse('del x,y,z'),indent=4))Module(body=[Delete(targets=[Name(id='x', ctx=Del()),Name(id='y', ctx=Del()),Name(id='z', ctx=Del())])])

classast.Pass

Apasstatement.>>>print(ast.dump(ast.parse('pass'),indent=4))Module(body=[Pass()])

classast.TypeAlias(name,type_params,value)

Atype aliascreated through thetypestatement.nameis the name of the alias,type_paramsis a list of type parameters, andvalueis the value of the type alias.>>>print(ast.dump(ast.parse('type Alias = int'),indent=4))Module(body=[TypeAlias(name=Name(id='Alias', ctx=Store()),value=Name(id='int', ctx=Load()))])Added in version 3.12.

Other statements which are only applicable inside functions or loops are described in other sections.

Imports

classast.Import(names)

An import statement.namesis a list ofaliasnodes.>>>print(ast.dump(ast.parse('import x,y,z'),indent=4))Module(body=[Import(names=[alias(name='x'),alias(name='y'),alias(name='z')])])

classast.ImportFrom(module,names,level)

Representsfromximporty.moduleis a raw string of the 'from' name, without any leading dots, orNonefor statements such asfrom.importfoo.levelis an integer holding the level of the relative import (0 means absolute import).>>>print(ast.dump(ast.parse('from y import x,y,z'),indent=4))Module(body=[ImportFrom(module='y',names=[alias(name='x'),alias(name='y'),alias(name='z')],level=0)])

classast.alias(name,asname)

Both parameters are raw strings of the names.asnamecan beNoneif the regular name is to be used.>>>print(ast.dump(ast.parse('from ..foo.bar import a as b, c'),indent=4))Module(body=[ImportFrom(module='foo.bar',names=[alias(name='a', asname='b'),alias(name='c')],level=2)])

Control flow

■ Nota: NoteOptional clauses such aselseare stored as an empty list if they're not present.

classast.If(test,body,orelse)

Anifstatement.testholds a single node, such as aComparenode.bodyandorelseeach hold a list of nodes.elifclauses don't have a special representation in the AST, but rather appear as extralfnodes within theorelsesection of the previous one.>>>print(ast.dump(ast.parse('""...if x:.....elif y:.....else:.....""'),indent=4))Module(body=[If(test=Name(id='x', ctx=Load()),body=[Expr(value=Constant(value=Ellipsis))],orelse=[If(test=Name(id='y',

ctx=Load()),body=[Expr(value=Constant(value=Ellipsis

class ast.For(target,iter,body,orelse,type_comment)

type_comment

A for loop. target holds the variable(s) the loop assigns to, as a single Name, Tuple, List, Attribute or Subscript node. iter holds the item to be looped over, again as a single node. body and orelse contain lists of nodes to execute. Those in orelse are executed if the loop finishes normally, rather than via a break statement. type_comment is an optional string with the type annotation as a comment. >>> print(ast.dump(ast.parse("""...for x in y:.....else:....."""), indent=4)) Module(body=[For(target= type_comment is an optional string with the type annotation as a comment.

class ast.While(test,body,orelse)

A while loop. test holds the condition, such as a Compare node. >>> print(ast.dump(ast.parse("""...while x:.....else:....."""), indent=4)) Module(body=[While(test=Name(id='x', ctx=Load()),body=[Expr(value=Constant(value=Ellipsis))],orelse=[Expr(value=Constant(value=Ellipsis))]))

class ast.Break

class ast.Continue

The break and continue statements. >>> print(ast.dump(ast.parse("""...for a in b:...if a > 5:...break...else:...continue....."""), indent=4)) Module(body=[For(target=Name(id='a', ctx=Store()),iter=Name(id='b', ctx=Load()),body=[If(test=Compare(left=Name(id='a', ctx=Load()),ops=[>],comparators=[Constant(value=5)]),body=[Break()],orelse=[Continue()])])])

class ast.Try(body,handlers,orelse,finalbody)

try blocks. All attributes are list of nodes to execute, except for handlers, which is a list of ExceptHandler nodes. >>> print(ast.dump(ast.parse("""...try:.....except Exception:.....except OtherException as e:.....else:.....finally:....."""), indent=4)) Module(body=[Try(body=[Expr(value=Constant(value=Ellipsis))],handlers=[ExceptHandler(type=Name(id='Exception', ctx=Load()),body=[Expr(value=Constant(value=Ellipsis))],ExceptHandler(type=Name(id='OtherException', ctx=Load()),name='e

class ast.TryStar(body,handlers,orelse,finalbody)

try blocks which are followed by except* clauses. The attributes are the same as for Try but the ExceptHandler nodes in handlers are interpreted as except* blocks rather than except. >>> print(ast.dump(ast.parse("""...try:.....except* Exception:....."""), indent=4)) Module(body=[Try Star(body=[Expr(value=Constant(value=Ellipsis))],handlers=[ExceptHandler(type=Name(id='Exception', ctx=Load()),body=[Expr(value=Constant(value=Ellipsis))])])]) Added in version 3.11.

class ast.ExceptHandler(type,name,body)

A single except clause. type is the exception type it will match, typically a Name node (or None for a catch-all except: clause). name is a raw string for the name to hold the exception, or None if the clause doesn't have a foo. body is a list of nodes. >>> print(ast.dump(ast.parse("""...try:...a + 1...except TypeError:...pass..."""), indent=4)) Module(body=[Try(body=[Expr(value=BinOp(left=Name(id='a', ctx=Load()),op=Add(),right=Constant(value=1))],handlers=[ExceptHandler(type=Name(id='TypeError', ctx=Load()),body

class ast.With(items,body,type_comment)

type_comment

A with block. items is a list of withitem nodes representing the context managers, and body is the indented block inside the context. type_comment is an optional string with the type annotation as a comment.

`type_comment` is an optional string with the type annotation as a comment.

class ast.withitem(context_expr, optional_vars)

A single context manager in a `with` block. `context_expr` is the context manager, often a `Call` node. `optional_vars` is a `Name`, `Tuple` or `List` for the `as` foo part, or `None` if that isn't used. >>> print(ast.dump(ast.parse("""\n...with a as b, c as d:\n...something(b, d)...\n"""), indent=4)) Module(body=[With(items=[withitem(context_expr=Name(id='a', ctx=Load()), optional_vars=Name(id='b', ctx=Store())), withitem(context_expr=Name(id='c', ctx=Load()), optional_vars=Name(id='d', ctx=Store()))], body=[Expr(value=Call(func=Name(id='some

Pattern matching

class ast.Match(subject, cases)

A `match` statement. `subject` holds the subject of the match (the object that is being matched against the cases) and `cases` contains an iterable of `match_case` nodes with the different cases. Added in version 3.10.

class ast.match_case(pattern, guard, body)

A single case pattern in a `match` statement. `pattern` contains the match pattern that the subject will be matched against. Note that the `AST` nodes produced for patterns differ from those produced for expressions, even when they share the same syntax. The `guard` attribute contains an expression that will be evaluated if the pattern matches the subject. `body` contains a list of nodes to execute if the pattern matches and the result of evaluating the guard expression is `true`. >>> print(ast.dump(ast.parse("""\n...matc

class ast.MatchValue(value)

A match literal or value pattern that compares by equality. `value` is an expression node. Permitted value nodes are restricted as described in the match statement documentation. This pattern succeeds if the match subject is equal to the evaluated value. >>> print(ast.dump(ast.parse("""\n...match x:\n...case\n\"Relevant\":\n.....\n""\"), indent=4)) Module(body=[Match(subject=Name(id='x', ctx=Load()), cases=[match_case(pattern=MatchValue(value=Constant(value='Relevant')), body=[Expr(value=Constant(value=Ellipsis))

class ast.MatchSingleton(value)

A match literal pattern that compares by identity. `value` is the singleton to be compared against: `None`, `True`, or `False`. This pattern succeeds if the match subject is the given constant. >>> print(ast.dump(ast.parse("""\n...match x:\n...case\nNone:\n.....\n""\"), indent=4)) Module(body=[Match(subject=Name(id='x', ctx=Load()), cases=[match_case(pattern=MatchSingleton(value=None), body=[Expr(value=Constant(value=Ellipsis))]]))]) Added in version 3.10.

class ast.MatchSequence(patterns)

A match sequence pattern. `patterns` contains the patterns to be matched against the subject elements if the subject is a sequence. Matches a variable length sequence if one of the subpatterns is a `MatchStar` node, otherwise matches a fixed length sequence. >>> print(ast.dump(ast.parse("""\n...match x:\n...case\n[1,\n2]:\n.....\n""\"), indent=4)) Module(body=[Match(subject=Name(id='x', ctx=Load()), cases=[match_case(pattern=MatchSequence(patterns=[MatchValue(value=Constant(value=1)), MatchValue(value=Constant(value=

class ast.MatchStar(name)

Matches the rest of the sequence in a variable length match sequence pattern. If `name` is not `None`, a list containing the remaining sequence elements is bound to that name if the overall sequence pattern is successful. >>> print(ast.dump(ast.parse("""\n...match x:\n...case\n[1, 2, *rest]:\n.....\n\"[*]:\n.....\n""\"), indent=4)) Module(body=[Match(subject=Name(id='x', ctx=Load()), cases=[match_case(pattern=MatchSequence(patterns=[MatchValue(value=Constant(value=1)), MatchValue(value=Constant(value=2)), Match

class ast.MatchMapping(keys, patterns, rest)

A match mapping pattern.keys is a sequence of expression nodes.patterns is a corresponding sequence of pattern nodes.rest is an optional name that can be specified to capture the remaining mapping elements. Permitted key expressions are restricted as described in the match statement documentation. This pattern succeeds if the subject is a mapping, all evaluated key expressions are present in the mapping, and the value corresponding to each key matches the corresponding subpattern. If rest is not None,

class ast.MatchClass(cls, patterns, kwd_attrs, kwd_patterns)

A match class pattern.cls is an expression giving the nominal class to be matched.patterns is a sequence of pattern nodes to be matched against the class defined sequence of pattern matching attributes.kwd_attr is a sequence of additional attributes to be matched (specified as keyword arguments in the class pattern),kwd_patterns are the corresponding patterns (specified as keyword values in the class pattern). This pattern succeeds if the subject is an instance of the nominated class, all positional

class ast.MatchAs(pattern, name)

A match “as-pattern”, capture pattern or wildcard pattern.pattern contains the match pattern that the subject will be matched against. If the pattern is None, the node represents a capture pattern (i.e. a bare name) and will always succeed. Then name attribute contains the name that will be bound if the pattern is successful. If name is None, pattern must also be None and the node represents the wildcard pattern.>>>print(ast.dump(ast.parse("""...match x:...case [x] as y:.....case _:....."""), indent=4))

class ast.MatchOr(patterns)

A match “or-pattern”. An or-pattern matches each of its subpatterns in turn to the subject, until one succeeds. The or-pattern is then deemed to succeed. If none of the subpatterns succeed the or-pattern fails. The patterns attribute contains a list of match pattern nodes that will be matched against the subject.>>>print(ast.dump(ast.parse("""...match x:...case [x] | (y):....."""), indent=4))Module(body=[Match(subject=Name(id='x', ctx=Load()), cases=[match_case(pattern=MatchOr(patterns=[MatchSeq

Type annotations

class ast.TypeIgnore(lineno, tag)

A #type: ignore comment located at lineno.tag is the optional tag specified by the form #type: ignore <tag>.>>>print(ast.dump(ast.parse('x = 1 # type: ignore', type_comments=True), indent=4))Module(body=[Assign(targets=[Name(id='x', ctx=Store())], value=Constant(value=1)), type_ignores=[TypeIgnore(lineno=1, tag='')])>>>print(ast.dump(ast.parse('x: bool = 1 # type: ignore[assignment]', type_comments=True), indent=4))Module(body=[AnnAssign(target=Name(id='x', ctx=Store()), annotation=Name(id='bool', ctx=Load())

Type parameters

Type parameters can exist on classes, functions, and type aliases.

class ast.TypeVar(name, bound, default_value)

Atyping.TypeVar.name is the name of the type variable.bound is the bound or constraints, if any. If bound is a Tuple, it represents constraints; otherwise it represents the bound.default_value is the default value; if the TypeVar has no default, this attribute will be set to None.>>>print(ast.dump(ast.parse("type Alias[T: int = bool] = list[T]", indent=4))Module(body=[TypeAlias(name=Name(id='Alias', ctx=Store()), type_params=[TypeVar(name='T', bound=Name(id='int', ctx=Load()), default_value=Name(id='bool',

class ast.ParamSpec(name, default_value)

Atyping.ParamSpec.name is the name of the parameter specification.default_value is the default value; if the ParamSpec has no default, this attribute will be set to None.>>>print(ast.dump(ast.parse('type Alias[*P] = [int,

```
str]] = Callable[P, int]"),indent=4))Module(body=[TypeAlias(name=Name(id='Alias',
ctx=Store()),type_params=[ParamSpec(name='P',default_value=List(elts=[Name(id='int',
ctx=Load()),Name(id='str', ctx=Load())],ctx=Load()))],value=Subscript(value=Name(id='Callable',
ctx=Load()),slice=Tu
```

class ast.TypeVarTuple(name, default_value)

Atyping.TypeVarTuple.name is the name of the type variable tuple.default_value is the default value; if theTypeVarTuple has no default, this attribute will be set toNone.>>>print(ast.dump(ast.parse("type Alias[*Ts = ()] = tuple[*Ts]",indent=4))Module(body=[TypeAlias(name=Name(id='Alias', ctx=Store()),type_params=[TypeVarTuple(name='Ts',default_value=Tuple(ctx=Load()))],value=Subscript(value=Name(id='tuple', ctx=Load()),slice=Tuple(elts=[Starred(value=Name(id='Ts', ctx=Load()),ctx=Load())],ctx=Load

Function and class definitions

class ast.FunctionDef(name, args, body, decorator_list, returns, type_comment, type_params)

type_comment

A function definition.name is a raw string of the function name.args is an arguments node.body is the list of nodes inside the function.decorator_list is the list of decorators to be applied, stored outermost first (i.e. the first in the list will be applied last).returns is the return annotation.type_params is a list of type parameters.type_comment type_comment is an optional string with the type annotation as a comment.Changed in version 3.12: Added type_params.

type_comment is an optional string with the type annotation as a comment.

class ast.Lambda(args, body)

lambda is a minimal function definition that can be used inside an expression. Unlike FunctionDef, body holds a single node.>>>print(ast.dump(ast.parse('lambda x,y: ...',indent=4))Module(body=[Expr(value=Lambda(args=arguments(args=[arg(arg='x'),arg(arg='y')]),body=Constant(value=Ellipsis))))])

class ast.arguments(posonlyargs, args, vararg, kwonlyargs, kw_defaults, kwarg, defaults)

The arguments for a function.posonlyargs, args and kwonlyargs are lists of arg nodes.vararg and kwarg are single arg nodes, referring to the *args, **kwargs parameters.kw_defaults is a list of default values for keyword-only arguments. If one is None, the corresponding argument is required.defaults is a list of default values for arguments that can be passed positionally. If there are fewer defaults, they correspond to the last n arguments.

class ast.arg(arg, annotation, type_comment)

type_comment

A single argument in a list.arg is a raw string of the argument name; annotation is its annotation, such as a Name node.type_comment type_comment is an optional string with the type annotation as a comment>>>print(ast.dump(ast.parse("""\...@decorator1...@decorator2...def f(a: 'annotation', b=1, c=2, *d, e, f=3, **g) -> 'return annotation':...pass...""",indent=4))Module(body=[FunctionDef(name='f',args=arguments(args=[arg(arg='a',annotation=Constant(value='annotation')),arg(arg='b'),arg(arg='c')],vararg

type_comment is an optional string with the type annotation as a comment

class ast.Return(value)

A return statement.>>>print(ast.dump(ast.parse('return 4',indent=4))Module(body=[Return(value=Constant(value=4))]))

class ast.Yield(value)

class ast.YieldFrom(value)

A `yield` or `yield from` expression. Because these are expressions, they must be wrapped in an `Expr` node if the value sent back is not used. `>>> print(ast.dump(ast.parse('yield x'), indent=4))`
`Module(body=[Expr(value=Yield(value=Name(id='x', ctx=Load()))]))`
`>>> print(ast.dump(ast.parse('yield from x'), indent=4))`
`Module(body=[Expr(value=YieldFrom(value=Name(id='x', ctx=Load()))]))`

class `ast.Global(names)`

class `ast.Nonlocal(names)`

`global` and `nonlocal` statements. `names` is a list of raw strings. `>>> print(ast.dump(ast.parse('global x,y,z'), indent=4))`
`Module(body=[Global(names=['x','y','z'])])`
`>>> print(ast.dump(ast.parse('nonlocal x,y,z'), indent=4))`
`Module(body=[Nonlocal(names=['x','y','z'])])`

class `ast.ClassDef(name,bases,keywords,body,decorator_list,type_params)`

A class definition. `name` is a raw string for the class name. `bases` is a list of nodes for explicitly specified base classes. `keywords` is a list of keyword nodes, principally for 'metaclass'. Other keywords will be passed to the metaclass, as per PEP 3115. `body` is a list of nodes representing the code within the class definition. `decorator_list` is a list of nodes, as in `FunctionDef`. `type_params` is a list of type parameters. `>>> print(ast.dump(ast.parse("""\n...@decorator1...\n...@decorator2...\nclass Foo(base1, base2, metac`

Async and await

class `ast.AsyncFunctionDef(name,args,body,decorator_list,returns,type_comment,type_params)`

An `async def` function definition. Has the same fields as `FunctionDef`. Changed in version 3.12: Added `type_params`.

class `ast.Await(value)`

An `await` expression. `value` is what it waits for. Only valid in the body of an `AsyncFunctionDef`.

```
>>> print(ast.dump(ast.parse("""\n... async def f():\n...     await other_func()\n... """), indent=4))
Module(
  body=[
    AsyncFunctionDef(
      name='f',
      args=arguments(),
      body=[
        Expr(
          value=Await(
            value=Call(
              func=Name(id='other_func', ctx=Load()))))])])
```

class `ast.AsyncFor(target,iter,body,orelse,type_comment)`

class `ast.AsyncWith(items,body,type_comment)`

`async for` loops and `async with` context managers. They have the same fields as `For` and `With`, respectively. Only valid in the body of an `AsyncFunctionDef`.

■ Nota: Note When a string is parsed by `ast.parse()`, operator nodes (subclasses of `ast.operator`, `ast.unaryop`, `ast.cmpop`, `ast.boolop` and `ast.expr_context`) on the returned tree will be singletons. Changes to one will be reflected in all other occurrences of the same value (for example, `ast.Add`).

ast helpers

Apart from the node classes, the `ast` module defines these utility functions and classes for traversing abstract syntax trees:

`ast.parse(source, filename='<unknown>', mode='exec', *, type_comments=False, feature_version=None, optimize=-1)`

Parse the source into an AST node. Equivalent to `compile(source, filename, mode, flags=FLAGS_VALUE, optimize=optimize)`, where `FLAGS_VALUE` is `ast.PyCF_ONLY_AST` if `optimize <= 0` and `ast.PyCF_OPTIMIZED_AST` otherwise. If `type_comments=True` is given, the parser is modified to check and return type comments as specified by PEP 484 and PEP 526. This is equivalent to adding `ast.PyCF_TYPE_COMMENTS` to the flags passed to `compile()`. This will report syntax errors for misplaced type comments. Without this flag, type comments will

`ast.unparse(ast_obj)`

Unparse an `ast.ASTObject` and generate a string with code that would produce an equivalent `ast.ASTObject` if parsed back with `ast.parse()`. Warning: The produced code string will not necessarily be equal to the original code that generated the `ast.ASTObject` (without any compiler optimizations, such as constant tuples/frozensets). Warning: Trying to unparse a highly complex expression would result in `RecursionError`. Added in version 3.9.

`ast.literal_eval(node_or_string)`

Evaluate an expression node or a string containing only a Python literal or container display. The string or node provided may only consist of the following Python literal structures: strings, bytes, numbers, tuples, lists, dicts, sets, booleans, `None` and `Ellipsis`. This can be used for evaluating strings containing Python values without the need to parse the values oneself. It is not capable of evaluating arbitrarily complex expressions, for example involving operators or indexing. This function has

`ast.get_docstring(node, clean=True)`

Return the docstring of the given `node` (which must be a `FunctionDef`, `AsyncFunctionDef`, `ClassDef`, or `Module` node), or `None` if it has no docstring. If `clean` is `True`, clean up the docstring's indentation with `inspect.cleandoc()`. Changed in version 3.5: `AsyncFunctionDef` is now supported.

`ast.get_source_segment(source, node, *, padded=False)`

Get source code segment of the `source` that generated `node`. If some location information (`lineno`, `end_lineno`, `col_offset`, or `end_col_offset`) is missing, return `None`. If `padded` is `True`, the first line of a multi-line statement will be padded with spaces to match its original position. Added in version 3.8.

`ast.fix_missing_locations(node)`

When you compile a node tree with `compile()`, the compiler expects `lineno` and `col_offset` attributes for every node that supports them. This is rather tedious to fill in for generated nodes, so this helper adds these attributes recursively where not already set, by setting them to the values of the parent node. It works recursively starting at `node`.

`ast.increment_lineno(node, n=1)`

Increment the line number and end line number of each node in the tree starting at `node` by `n`. This is useful to “move code” to a different location in a file.

`ast.copy_location(new_node, old_node)`

Copy source location (`lineno`, `col_offset`, `end_lineno`, and `end_col_offset`) from `old_node` to `new_node` if possible, and return `new_node`.

`ast.iter_fields(node)`

Yield a tuple of (`fieldname`, `value`) for each field in `node`. `_fields` that is present on `node`.

ast.iter_child_nodes(node)

Yield all direct child nodes of node, that is, all fields that are nodes and all items of fields that are lists of nodes.

ast.walk(node)

Recursively yield all descendant nodes in the tree starting at node (including node itself), in no specified order. This is useful if you only want to modify nodes in place and don't care about the context.

class ast.NodeVisitor

visit(node)

generic_visit(node)

visit_Constant(node)

A node visitor base class that walks the abstract syntax tree and calls a visitor function for every node found. This function may return a value which is forwarded by the visit() method. This class is meant to be subclassed, with the subclass adding visitor methods. visit(node) Visit a node. The default implementation calls the method called self.visit_classname where classname is the name of the node class, or generic_visit() if that method doesn't exist. generic_visit(node) This visitor calls visit() on a

Visit a node. The default implementation calls the method called self.visit_classname where classname is the name of the node class, or generic_visit() if that method doesn't exist.

This visitor calls visit() on all children of the node. Note that child nodes of nodes that have a custom visitor method won't be visited unless the visitor calls generic_visit() or visits them itself.

Handles all constant nodes.

class ast.NodeTransformer

A NodeVisitor subclass that walks the abstract syntax tree and allows modification of nodes. The NodeTransformer will walk the AST and use the return value of the visitor methods to replace or remove the old node. If the return value of the visitor method is None, the node will be removed from its location, otherwise it is replaced with the return value. The return value may be the original node in which case no replacement takes place. Here is an example transformer that rewrites all occurrences of

ast.dump(node, annotate_fields=True, include_attributes=False, *, indent=None, show_empty=False)

Return a formatted dump of the tree in node. This is mainly useful for debugging purposes. If annotate_fields is true (by default), the returned string will show the names and the values for fields. If annotate_fields is false, the result string will be more compact by omitting unambiguous field names. Attributes such as line numbers and column offsets are not dumped by default. If this is wanted, include_attributes can be set to true. If indent is a non-negative integer or string, then the tree will

Compiler flags

The following flags may be passed to compile() in order to change effects on the compilation of a program:

ast.PyCF_ALLOW_TOP_LEVEL_AWAIT

Enables support for top-level await, async for, async with and async comprehensions. Added in version 3.8.

ast.PyCF_ONLY_AST

Generates and returns an abstract syntax tree instead of returning a compiled code object.

ast.PyCF_OPTIMIZED_AST

The returned AST is optimized according to the `optimize` argument in `compile()` or `ast.parse()`. Added in version 3.13.

ast.PyCF_TYPE_COMMENTS

Enables support for PEP 484 and PEP 526 style type comments (`#type:<type>`, `#type:ignore<stuff>`). Added in version 3.8.

ast.compare(a,b,/,*,compare_attributes=False)

Recursively compares two ASTs. `compare_attributes` affects whether AST attributes are considered in the comparison. If `compare_attributes` is `False` (default), then attributes are ignored. Otherwise they must all be equal. This option is useful to check whether the ASTs are structurally equal but differ in whitespace or similar details. Attributes include line numbers and column offsets. Added in version 3.14.

Command-line usage

Added in version 3.9.

The `ast` module can be executed as a script from the command line. It is as simple as:

```
python -m ast [-m <mode>] [-a] [infile]
```

The following options are accepted:

-h,--help

Show the help message and exit.

-m<mode>

--mode<mode>

Specify what kind of code must be compiled, like the `mode` argument in `parse()`.

--no-type-comments

Don't parse type comments.

-a,--include-attributes

Include attributes such as line numbers and column offsets.

-i<indent>

--indent<indent>

Indentation of nodes in AST (number of spaces).

--feature-version<version>

Python version in the format 3.x (for example, 3.10). Defaults to the current version of the interpreter. Added in version 3.14.

-O<level>

--optimize<level>

Optimization level for parser. Defaults to no optimization. Added in version 3.14.

--show-empty

Show empty lists and fields that are `None`. Defaults to not showing empty objects. Added in version 3.14.

If `infile` is specified its contents are parsed to AST and dumped to `stdout`. Otherwise, the content is read from `stdin`.

See also

Green Tree Snakes, an external documentation resource, has good details on working with Python ASTs.

ASTTokens annotates Python ASTs with the positions of tokens and text in the source code that generated them. This is helpful for tools that make source code transformations.

leoAst.py unifies the token-based and parse-tree-based views of python programs by inserting two-way links between tokens and ast nodes.

LibCST parses code as a Concrete Syntax Tree that looks like an ast tree and keeps all formatting details. It's useful for building automated refactoring (codemod) applications and linters.

Parso is a Python parser that supports error recovery and round-trip parsing for different Python versions (in multiple Python versions). Parso is also able to list multiple syntax errors in your Python file.

symtable— Access to the compiler's symbol tables

Source code: `Lib/symtable.py`

Symbol tables are generated by the compiler from AST just before bytecode is generated. The symbol table is responsible for calculating the scope of every identifier in the code. `symtable` provides an interface to examine these tables.

Generating Symbol Tables

`symtable.symtable(code, filename, compile_type)`

Return the `toplevelSymbolTable` for the Python source code. `filename` is the name of the file containing the code. `compile_type` is like the `mode` argument to `compile()`.

Examining Symbol Tables

`class symtable.SymbolTableType`

`MODULE="module"`

`FUNCTION="function"`

`CLASS="class"`

`ANNOTATION="annotation"`

`TYPE_ALIAS="typealias"`

`TYPE_PARAMETERS="typeparameters"`

`TYPE_VARIABLE="typevariable"`

An enumeration indicating the type of a `SymbolTable` object. `MODULE="module"` Used for the symbol table of a module. `FUNCTION="function"` Used for the symbol table of a function. `CLASS="class"` Used for the symbol table of a class. The following members refer to different flavors of annotation scopes. `ANNOTATION="annotation"` Used for annotations if from `__future__` import annotations is active. `TYPE_ALIAS="typealias"` Used for the symbol table of type constructions. `TYPE_PARAMETERS="typeparameters"` Used for the symbol table

Used for the symbol table of a module.

Used for the symbol table of a function.

Used for the symbol table of a class.

Used for annotations if from `__future__` import annotations is active.

Used for the symbol table of type constructions.

Used for the symbol table of generic functions or generic classes.

Used for the symbol table of the bound, the constraint tuple or the default value of a single type variable in the formal sense, i.e., a `TypeVar`, a `TypeVarTuple` or a `ParamSpec` object (the latter two do not support a bound or a constraint tuple).

`class symtable.SymbolTable`

`get_type()`

get_id()

get_name()

get_lineno()

is_optimized()

is_nested()

has_children()

get_identifiers()

lookup(name)

get_symbols()

get_children()

A namespace table for a block. The constructor is not public.
get_type() Return the type of the symbol table. Possible values are members of the `SymbolTableType` enumeration. Changed in version 3.12: Added 'annotation', 'TypeVarbound', 'typealias', and 'typeparameter' as possible return values. Changed in version 3.13: Return values are members of the `SymbolTableType` enumeration. The exact values of the returned string may change in the future, and thus, it is recommended to use `SymbolTableType` members instead of

Return the type of the symbol table. Possible values are members of the `SymbolTableType` enumeration. Changed in version 3.12: Added 'annotation', 'TypeVarbound', 'typealias', and 'typeparameter' as possible return values. Changed in version 3.13: Return values are members of the `SymbolTableType` enumeration. The exact values of the returned string may change in the future, and thus, it is recommended to use `SymbolTableType` members instead of hard-coded strings.

Return the table's identifier.

Return the table's name. This is the name of the class if the table is for a class, the name of the function if the table is for a function, or 'top' if the table is global (`get_type()` returns 'module'). For type parameter scopes (which are used for generic classes, functions, and type aliases), it is the name of the underlying class, function, or type alias. For type alias scopes, it is the name of the type alias. For `TypeVarbound` scopes, it is the name of the `TypeVar`.

Return the number of the first line in the block this table represents.

Return `True` if the locals in this table can be optimized.

Return `True` if the block is a nested class or function.

Return `True` if the block has nested namespaces within it. These can be obtained with `get_children()`.

Return a view object containing the names of symbols in the table. See the documentation of view objects.

Lookup `name` in the table and return a `Symbol` instance.

Return a list of `Symbol` instances for names in the table.

Return a list of the nested symbol tables.

class `symtable`.Function

get_parameters()

get_locals()

get_globals()

get_nonlocals()

get_frees()

A namespace for a function or method. This class inherits from `SymbolTable`. `get_parameters()` Return a tuple containing names of parameters to this function. `get_locals()` Return a tuple containing names of locals in this function. `get_globals()` Return a tuple containing names of globals in this function. `get_nonlocals()` Return a tuple containing names of explicitly declared nonlocals in this function. `get_frees()` Return a tuple containing names of free (closure) variables in this function.

Return a tuple containing names of parameters to this function.

Return a tuple containing names of locals in this function.

Return a tuple containing names of globals in this function.

Return a tuple containing names of explicitly declared nonlocals in this function.

Return a tuple containing names of free (closure) variables in this function.

class `symtable.Class`

get_methods()

A namespace of a class. This class inherits from `SymbolTable`. `get_methods()` Return a tuple containing the names of method-like functions declared in the class. Here, the term 'method' designates any function defined in the class body via `def` or `async def`. Functions defined in a deeper scope (e.g., in an inner class) are not picked up by `get_methods()`. For example:

```
>>> import symtable
>>> st = symtable.symtable("""...def outer(): pass...class A:...def f():...def w(): pass...def g(self): pass...@classmethod
```

Return a tuple containing the names of method-like functions declared in the class. Here, the term 'method' designates any function defined in the class body via `def` or `async def`. Functions defined in a deeper scope (e.g., in an inner class) are not picked up by `get_methods()`. For example:

```
>>> import symtable
>>> st = symtable.symtable("""...def outer(): pass...class A:...def f():...def w():
pass...def g(self): pass...@classmethod...async def h(cls): pass...global outer...def outer(self): pass...""",
```

class `symtable.Symbol`

get_name()

is_referenced()

is_imported()

is_parameter()

is_type_parameter()

is_global()

is_nonlocal()

is_declared_global()

is_local()

is_annotated()

is_free()

is_free_class()

is_assigned()

is_comp_iter()

is_comp_cell()

is_namespace()

get_namespaces()

get_namespace()

An entry in a `SymTable` corresponding to an identifier in the source. The constructor is not public.
`get_name()` Return the symbol's name.
`is_referenced()` Return `True` if the symbol is used in its block.
`is_imported()` Return `True` if the symbol is created from an import statement.
`is_parameter()` Return `True` if the symbol is a parameter.
`is_type_parameter()` Return `True` if the symbol is a type parameter. Added in version 3.14.
`is_global()` Return `True` if the symbol is global.
`is_nonlocal()` Return `True` if the symbol is nonlocal

Return the symbol's name.

Return `True` if the symbol is used in its block.

Return `True` if the symbol is created from an import statement.

Return `True` if the symbol is a parameter.

Return `True` if the symbol is a type parameter. Added in version 3.14.

Return `True` if the symbol is global.

Return `True` if the symbol is nonlocal.

Return `True` if the symbol is declared global with a global statement.

Return `True` if the symbol is local to its block.

Return `True` if the symbol is annotated. Added in version 3.6.

Return `True` if the symbol is referenced in its block, but not assigned to.

Return `True` if a class-scoped symbol is free from the perspective of a method. Consider the following example:

```
deff():x=1# function-scoped
class C:x=2# class-scoped
def method(self):return x
```


In this example, the class-scoped symbol `x` is considered to be free from the perspective of `C.method`, thereby allowing the latter to return `1` at runtime and not `2`. Added in version 3.14.

Return `True` if the symbol is assigned to in its block.

Return `True` if the symbol is a comprehension iteration variable. Added in version 3.14.

Return `True` if the symbol is a cell in an inlined comprehension. Added in version 3.14.

Return `True` if name binding introduces new namespace. If the name is used as the target of a function or class statement, this will be `True`. For example:

```
>>> table = symtable.symtable("def some_func():
pass", "string", "exec")
>>> table.lookup("some_func").is_namespace()
True
```


Note that a single name can be bound to multiple objects. If the result is `True`, the name may also be bound to other objects, like an int or list, that does not introduce a new namespace.

Return a list of namespaces bound to this name.

Return the namespace bound to this name. If more than one or no namespace is bound to this name, a `ValueError` is raised.

Command-Line Usage

Added in version 3.13.

The `symtable` module can be executed as a script from the command line.

```
python -m symtable [infile...]
```

Symbol tables are generated for the specified Python source files and dumped to stdout. If no input file is specified, the content is read from stdin.

token— Constants used with Python parse trees

Source code: `Lib/token.py`

This module provides constants which represent the numeric values of leaf nodes of the parse tree (terminal tokens). Refer to the file `Grammar/Tokens` in the Python distribution for the definitions of the names in the context of the language grammar. The specific numeric values which the names map to may change between Python versions.

The module also provides a mapping from numeric codes to names and some functions. The functions mirror definitions in the Python C header files.

Note that a token's value may depend on tokenizer options. For example, `a"+"` token may be reported as either `PLUS` or `OP`, or `a"match"` token may be either `NAME` or `SOFT_KEYWORD`.

token.tok_name

Dictionary mapping the numeric values of the constants defined in this module back to name strings, allowing more human-readable representation of parse trees to be generated.

token.ISTERMINAL(x)

Return `True` for terminal token values.

token.ISNONTERMINAL(x)

Return `True` for non-terminal token values.

token.ISEOF(x)

Return `True` if `x` is the marker indicating the end of input.

The token constants are:

token.NAME

Token value that indicates an identifier or keyword.

token.NUMBER

Token value that indicates a numeric literal

token.STRING

Token value that indicates a string or byte literal, excluding formatted string literals. The token string is not interpreted: it includes the surrounding quotation marks and the prefix (if given); backslashes are included literally, without processing escape sequences.

token.OP

A generic token value that indicates an operator or delimiter. CPython implementation detail: This value is only reported by the `tokenize` module. Internally, the tokenizer uses exact token types instead.

token.COMMENT

Token value used to indicate a comment. The parser ignores `COMMENT` tokens.

token.NEWLINE

Token value that indicates the end of a logical line.

token.NL

Token value used to indicate a non-terminating newline. `NL` tokens are generated when a logical line of code is continued over multiple physical lines. The parser ignores `NL` tokens.

token.INDENT

Token value used at the beginning of a logical line to indicate the start of an indented block.

token.DEDENT

Token value used at the beginning of a logical line to indicate the end of an indented block.

token.FSTRING_START

Token value used to indicate the beginning of an f-string literal. CPython implementation detail: The token string includes the prefix and the opening quote(s), but none of the contents of the literal.

token.FSTRING_MIDDLE

Token value used for literal text inside an f-string literal, including format specifications. CPython implementation detail: Replacement fields (that is, the non-literal parts of f-strings) use the same tokens as other expressions, and are delimited by `LBRACE`, `RBRACE`, `EXCLAMATION` and `COLON` tokens.

token.FSTRING_END

Token value used to indicate the end of an f-string. CPython implementation detail: The token string contains the closing quote(s).

token.TSTRING_START

Token value used to indicate the beginning of a template string literal. CPython implementation detail: The token string includes the prefix and the opening quote(s), but none of the contents of the literal. Added in version 3.14.

token.TSTRING_MIDDLE

Token value used for literal text inside a template string literal including format specifications. CPython implementation detail: Replacement fields (that is, the non-literal parts of t-strings) use the same tokens as other expressions, and are delimited by `LBRACE`, `RBRACE`, `EXCLAMATION` and `COLON` tokens. Added in version 3.14.

token.TSTRING_END

Token value used to indicate the end of a template string literal. CPython implementation detail: The token string contains the closing quote(s). Added in version 3.14.

token.ENDMARKER

Token value that indicates the end of input. Used in top-level grammar rules.

token.ENCODING

Token value that indicates the encoding used to decode the source bytes into text. The first token returned by `tokenize.tokenize()` will always be an `ENCODING` token. CPython implementation detail: This token type isn't used by the C tokenizer but is needed for the `tokenize` module.

The following token types are not produced by the `tokenize` module, and are defined for special uses in the tokenizer or parser:

token.TYPE_IGNORE

Token value indicating that a type ignore comment was recognized. Such tokens are produced instead of regular `COMMENT` tokens only with the `PyCF_TYPE_COMMENTS` flag.

token.TYPE_COMMENT

Token value indicating that a type comment was recognized. Such tokens are produced instead of regular `COMMENT` tokens only with the `PyCF_TYPE_COMMENTS` flag.

token.SOFT_KEYWORD

Token value indicating a soft keyword. The tokenizer never produces this value. To check for a soft keyword, pass a NAME token's string to `keyword.issoftkeyword()`.

token.ERROR_TOKEN

Token value used to indicate wrong input. The `tokenize` module generally indicates errors by raising exceptions instead of emitting this token. It can also emit tokens such as `OP` or `NAME` with strings that are later rejected by the parser.

The remaining tokens represent specific operators and delimiters. (The `tokenize` module reports these as `OP`; see `exact_type` in the `tokenize` documentation for details.)

```
Token | Value | token. | LPAR | "(" | token. | RPAR | ")" | token. | LSQB | "[" | token. | RSQB | "]" | token. | COLON | ":" | token. | COMMA | "," | token. | SEMI | ";" | token. | PLUS | "+" | token. | MINUS | "-" | token. | STAR | "*" | token. | SLASH | "/" | token. | VBAR | "|" | token. | AMPER | "&" | token. | LESS | "<" | token. | GREATER | ">" | token. | EQUAL | "=" | token. | DOT | "." | token. | PERCENT | "%" | token. | LBRACE | "{" | token. | RBRACE | "}" | token. | EQEQUAL | "==" | token. | NOTEQUAL | "!=" | token. | LESSEQUAL | "<=" | token. | GREATEREQUAL | ">=" | token. | TILDE | "~" | token. | CIRCUMFLEX | "^" | token. | LEFTSHIFT | "<<" | token. | RIGHTSHIFT | ">>" | token. | DOUBLESTAR | "**" | token. | PLUSEQUAL | "+=" | token. | MINEQUAL | "-=" | token. | STAREQUAL | "*="
```

The following non-token constants are provided:

token.N_TOKENS

The number of token types defined in this module.

token.EXACT_TOKEN_TYPES

A dictionary mapping the string representation of a token to its numeric code. Added in version 3.8.

Changed in version 3.5: Added `AWAIT` and `ASYNCTOKENS`.

Changed in version 3.7: Added `COMMENT`, `NL` and `ENCODING` tokens.

Changed in version 3.7: Removed `AWAIT` and `ASYNCTOKENS`. “`async`” and “`await`” are now tokenized as `NAME` tokens.

Changed in version 3.8: Added `TYPE_COMMENT`, `TYPE_IGNORE`, `COLONEQUAL`.

Added `AWAIT` and `ASYNCTOKENS` back (they're needed to support parsing older Python versions for `ast.parse()` with `feature_version` set to 6 or lower).

Changed in version 3.12: Added `EXCLAMATION`.

Changed in version 3.13: Removed `AWAIT` and `ASYNCTOKENS` again.

keyword— Testing for Python keywords

Source code: `Lib/keyword.py`

This module allows a Python program to determine if a string is a keyword or soft keyword.

keyword.iskeyword(s)

Return `True` if `s` is a Python keyword.

keyword.kwlist

Sequence containing all the keywords defined for the interpreter. If any keywords are defined to only be active when particular `__future__` statements are in effect, these will be included as well.

keyword.issoftkeyword(s)

Return `True` if `s` is a Python soft keyword. Added in version 3.9.

keyword.softkwlist

Sequence containing all the soft keywords defined for the interpreter. If any soft keywords are defined to only be active when particular `__future__` statements are in effect, these will be included as well. Added in version 3.9.

tokenize—Tokenizer for Python source

Source code: `Lib/tokenize.py`

The `tokenize` module provides a lexical scanner for Python source code, implemented in Python. The scanner in this module returns comments as tokens as well, making it useful for implementing “pretty-printers”, including colorizers for on-screen displays.

To simplify token stream handling, `operator` and `delimitertokens` and `Ellipsis` are returned using the generic `OPtoken` type. The exact type can be determined by checking the `exact_type` property on the named tuple returned from `tokenize.tokenize()`.

■■ Advertencia: WarningNote that the functions in this module are only designed to parse syntactically valid Python code (code that does not raise when parsed using `ast.parse()`). The behavior of the functions in this module is undefined when providing invalid Python code and it can change at any point.

Tokenizing Input

The primary entry point is a generator:

`tokenize.tokenize(readline)`

The `tokenize()` generator requires one argument, `readline`, which must be a callable object which provides the same interface as the `io.IOBase.readline()` method of file objects. Each call to the function should return one line of input as bytes. The generator produces 5-tuples with these members: the token type; the token string; a 2-tuple (`srow`, `scol`) of ints specifying the row and column where the token begins in the source; a 2-tuple (`erow`, `ecol`) of ints specifying the row and column where the token ends

`tokenize.generate_tokens(readline)`

Tokenize a source reading unicode strings instead of bytes. Like `tokenize()`, the `readline` argument is a callable returning a single line of input. However, `generate_tokens()` expects `readline` to return a `str` object rather than bytes. The result is an iterator yielding named tuples, exactly like `tokenize()`. It does not yield an `ENCODING` token.

All constants from the `token` module are also exported from `tokenize`.

Another function is provided to reverse the tokenization process. This is useful for creating tools that tokenize a script, modify the token stream, and write back the modified script.

`tokenize.untokenize(iterable)`

Converts tokens back into Python source code. The `iterable` must return sequences with at least two elements, the token type and the token string. Any additional sequence elements are ignored. The result is guaranteed to tokenize back to match the input so that the conversion is lossless and round-trips are assured. The guarantee applies only to the token type and token string as the spacing between tokens (column positions) may change. It returns bytes, encoded using the `ENCODING` token, which is the

`tokenize()` needs to detect the encoding of source files it tokenizes. The function it uses to do this is available:

`tokenize.detect_encoding(readline)`

The `detect_encoding()` function is used to detect the encoding that should be used to decode a Python source file. It requires one argument, `readline`, in the same way as the `tokenize()` generator. It will call `readline` a maximum of twice, and return the encoding used (as a string) and a list of any lines (not decoded from bytes) it has read in. It detects the encoding from the presence of a UTF-8 BOM or an encoding cookie as specified in PEP 263. If both a BOM and a cookie are present, but disagree, a `Syn`

tokenize.open(filename)

Open a file in read only mode using the encoding detected by `detect_encoding()`. Added in version 3.2.

exception tokenize.TokenError

Raised when either a docstring or expression that may be split over several lines is not completed anywhere in the file, for example: `"""Beginning of docstring or: [1,2,3`

Command-Line Usage

Added in version 3.3.

The `tokenize` module can be executed as a script from the command line. It is as simple as:

```
python -m tokenize [-e] [filename.py]
```

The following options are accepted:

-h, --help

show this help message and exit

-e, --exact

display token names using the exact type

If `filename.py` is specified its contents are tokenized to stdout. Otherwise, tokenization is performed on stdin.

Examples

Example of a script rewriter that transforms float literals into Decimal objects:

```
from tokenize import tokenize, untokenize, NUMBER, STRING, NAME, OP
from io import BytesIO
```

```
def decistmt(s):
    """Substitute Decimals for floats in a string of statements.

    >>> from decimal import Decimal
    >>> s = 'print(+21.3e-5*-.1234/81.7)'
    >>> decistmt(s)
    "print (+Decimal ('21.3e-5')*-Decimal ('.1234')/Decimal ('81.7'))"
```

The format of the exponent is inherited from the platform C library. Known cases are "e-007" (Windows) and "e-07" (not Windows). Since we're only showing 12 digits, and the 13th isn't close to 5, the rest of the output should be platform-independent.

```
>>> exec(s) #doctest: +ELLIPSIS
-3.21716034272e-0...7
```

Output from calculations with Decimal should be identical across all platforms.

```
>>> exec(decistmt(s))
-3.217160342717258261933904529E-7
"""

result = []
g = tokenize(BytesIO(s.encode('utf-8')).readline) # tokenize the string
for toknum, tokval, _, _, _ in g:
    if toknum == NUMBER and '.' in tokval: # replace NUMBER tokens
        result.extend([
```

```

        (NAME, 'Decimal'),
        (OP, '('),
        (STRING, repr(tokval)),
        (OP, ')')
    ])
else:
    result.append((toknum, tokval))
return untokenize(result).decode('utf-8')

```

Example of tokenizing from the command line. The script:

```

def say_hello():
    print("Hello, World!")

```

```
say_hello()
```

will be tokenized to the following output where the first column is the range of the line/column coordinates where the token is found, the second column is the name of the token, and the final column is the value of the token (if any)

```

$ python -m tokenize hello.py
0,0-0,0:      ENCODING      'utf-8'
1,0-1,3:      NAME          'def'
1,4-1,13:     NAME          'say_hello'
1,13-1,14:    OP            '('
1,14-1,15:    OP            ')'
1,15-1,16:    OP            ':'
1,16-1,17:    NEWLINE      '\n'
2,0-2,4:      INDENT       '    '
2,4-2,9:      NAME          'print'
2,9-2,10:     OP            '('
2,10-2,25:    STRING        '"Hello, World!"'
2,25-2,26:    OP            ')'
2,26-2,27:    NEWLINE      '\n'
3,0-3,1:      NL           '\n'
4,0-4,0:      DEDENT       ''
4,0-4,9:      NAME          'say_hello'
4,9-4,10:     OP            '('
4,10-4,11:    OP            ')'
4,11-4,12:    NEWLINE      '\n'
5,0-5,0:      ENDMARKER    ''

```

The exact token type names can be displayed using the -e option:

```

$ python -m tokenize -e hello.py
0,0-0,0:      ENCODING      'utf-8'
1,0-1,3:      NAME          'def'
1,4-1,13:     NAME          'say_hello'
1,13-1,14:    LPAR         '('
1,14-1,15:    RPAR         ')'
1,15-1,16:    COLON        ':'
1,16-1,17:    NEWLINE      '\n'
2,0-2,4:      INDENT       '    '
2,4-2,9:      NAME          'print'
2,9-2,10:     LPAR         '('
2,10-2,25:    STRING        '"Hello, World!"'
2,25-2,26:    RPAR         ')'
2,26-2,27:    NEWLINE      '\n'
3,0-3,1:      NL           '\n'
4,0-4,0:      DEDENT       ''
4,0-4,9:      NAME          'say_hello'
4,9-4,10:     LPAR         '('
4,10-4,11:    RPAR         ')'
4,11-4,12:    NEWLINE      '\n'
5,0-5,0:      ENDMARKER    ''

```

Example of tokenizing a file programmatically, reading unicode strings instead of bytes with `generate_tokens()`:

```
import tokenize

with tokenize.open('hello.py') as f:
    tokens = tokenize.generate_tokens(f.readline)
    for token in tokens:
        print(token)
```

Or reading bytes directly with `tokenize()`:

```
import tokenize

with open('hello.py', 'rb') as f:
    tokens = tokenize.tokenize(f.readline)
    for token in tokens:
        print(token)
```

tabnanny— Detection of ambiguous indentation

Source code: `Lib/tabnanny.py`

For the time being this module is intended to be called as a script. However it is possible to import it into an IDE and use the `functioncheck()` described below.

■ Nota: NoteThe API provided by this module is likely to change in future releases; such changes may not be backward compatible.

`tabnanny.check(file_or_dir)`

If `file_or_dir` is a directory and not a symbolic link, then recursively descend the directory tree named `file_or_dir`, checking all .py files along the way. If `file_or_dir` is an ordinary Python source file, it is checked for whitespace related problems. The diagnostic messages are written to standard output using the `print()` function.

`tabnanny.verbose`

Flag indicating whether to print verbose messages. This is incremented by the `-v` option if called as a script.

`tabnanny.filename_only`

Flag indicating whether to print only the filenames of files containing whitespace related problems. This is set to true by the `-q` option if called as a script.

`exceptiontabnanny.NannyNag`

Raised by `process_tokens()` if detecting an ambiguous indent. Captured and handled in `check()`.

`tabnanny.process_tokens(tokens)`

This function is used by `check()` to process tokens generated by the `tokenize` module.

See also

Module `tokenize`

Lexical scanner for Python source code.

pyclbr— Python module browser support

Source code: `Lib/pyclbr.py`

The `pyclbr` module provides limited information about the functions, classes, and methods defined in a Python-coded module. The information is sufficient to implement a module browser. The information is extracted from the Python source code rather than by importing the module, so this module is safe to use with untrusted code. This restriction makes it impossible to use this module with modules not implemented in Python, including all standard and optional extension modules.

`pyclbr.readmodule(module,path=None)`

Return a dictionary mapping module-level class names to class descriptors. If possible, descriptors for imported base classes are included. Parameter `module` is a string with the name of the module to read; it may be the name of a module within a package. If given, `path` is a sequence of directory paths prepended to `sys.path`, which is used to locate the module source code. This function is the original interface and is only kept for back compatibility. It returns a filtered version of the following.

`pyclbr.readmodule_ex(module,path=None)`

Return a dictionary-based tree containing a function or class descriptors for each function and class defined in the module with a `for class statement`. The returned dictionary maps module-level function and class names to their descriptors. Nested objects are entered into the children dictionary of their parent. As with `readmodule`, `module` names the module to be read and `path` is prepended to `sys.path`. If the module being read is a package, the returned dictionary has a key `'__path__'` whose value is

Added in version 3.7: Descriptors for nested definitions. They are accessed through the new `children` attribute. Each has a new `parent` attribute.

The descriptors returned by these functions are instances of `Function` and `Class` classes. Users are not expected to create instances of these classes.

Function Objects

`class pyclbr.Function`

`file`

`module`

`name`

`lineno`

`parent`

`children`

`is_async`

Class `Function` instances describe functions defined by `def` statements. They have the following attributes: `fileName` of the file in which the function is defined. `module` The name of the module defining the function described. `name` The name of the function. `lineno` The line number in the file where the definition starts. `parent` For top-level functions, `None`. For nested functions, the parent. Added in version 3.7. `children` A dictionary mapping names to descriptors for nested functions and classes. Added in version

Name of the file in which the function is defined.

The name of the module defining the function described.

The name of the function.

The line number in the file where the definition starts.

For top-level functions, None. For nested functions, the parent. Added in version 3.7.

Adictionary mapping names to descriptors for nested functions and classes. Added in version 3.7.

True for functions that are defined with the async prefix, False otherwise. Added in version 3.10.

Class Objects

class `pyclbr.Class`

file

module

name

lineno

parent

children

super

methods

`Class` instances describe classes defined by class statements. They have the same attributes as `Function`s and two more. `file` Name of the file in which the class is defined. `module` The name of the module defining the class described. `name` The name of the class. `lineno` The line number in the file where the definition starts. `parent` For top-level classes, None. For nested classes, the parent. Added in version 3.7. `children` A dictionary mapping names to descriptors for nested functions and classes. Added in ver

Name of the file in which the class is defined.

The name of the module defining the class described.

The name of the class.

The line number in the file where the definition starts.

For top-level classes, None. For nested classes, the parent. Added in version 3.7.

A dictionary mapping names to descriptors for nested functions and classes. Added in version 3.7.

A list of `Class` objects which describe the immediate base classes of the class being described. Classes which are named as superclasses but which are not discoverable by `readmodule_ex()` are listed as a string with the class name instead of as `Class` objects.

A dictionary mapping method names to line numbers. This can be derived from the newer `children` dictionary, but remains for back-compatibility.

py_compile— Compile Python source files

Source code: Lib/py_compile.py

The `py_compile` module provides a function to generate a byte-code file from a source file, and another function used when the module source file is invoked as a script.

Though not often needed, this function can be useful when installing modules for shared use, especially if some of the users may not have permission to write the byte-code cache files in the directory containing the source code.

exception `py_compile.PyCompileError`

Exception raised when an error occurs while attempting to compile the file.

`py_compile.compile(file, cfile=None, dfile=None, doraise=False, optimize=-1, invalidation_mode=PyInvalidationMode.TIMESTAMP, quiet=0)`

Compile a source file to byte-code and write out the byte-code cache file. The source code is loaded from the file named `file`. The byte-code is written to `cfile`, which defaults to the PEP 3147/PEP 488 path, ending in `.pyc`. For example, if `file` is `foo/bar/baz.py`, `cfile` will default to `foo/bar/__pycache__/baz.cpython-32.pyc` for Python 3.2. If `dfile` is specified, it is used instead of `cfile` as the name of the source file from which source lines are obtained for display in exception tracebacks. If `doraise` is true,

class `py_compile.PyInvalidationMode`

`TIMESTAMP`

`CHECKED_HASH`

`UNCHECKED_HASH`

An enumeration of possible methods the interpreter can use to determine whether a bytecode file is up to date with a source file. The `.pycfile` indicates the desired invalidation mode in its header. See `Cached bytecode invalidation` for more information on how Python invalidates `.pycfiles` at runtime. Added in version 3.7. `TIMESTAMP` The `.pycfile` includes the timestamp and size of the source file, which Python will compare against the metadata of the source file at runtime to determine if the `.pycfile` needs

The `.pycfile` includes the timestamp and size of the source file, which Python will compare against the metadata of the source file at runtime to determine if the `.pycfile` needs to be regenerated.

The `.pycfile` includes a hash of the source file content, which Python will compare against the source at runtime to determine if the `.pycfile` needs to be regenerated.

Like `CHECKED_HASH`, the `.pycfile` includes a hash of the source file content. However, Python will at runtime assume the `.pycfile` is up to date and not validate the `.pyc` against the source file at all. This option is useful when the `.pycs` are kept up to date by some system external to Python like a build system.

Command-Line Interface

This module can be invoked as a script to compile several source files. The files named in `filenames` are compiled and the resulting bytecode is cached in the normal manner. This program does not search a directory structure to locate source files; it only compiles files named explicitly. The exit status is nonzero if one of the files could not be compiled.

`<file>...<fileN>`

-

Positional arguments are files to compile. If-is the only parameter, the list of files is taken from standard input.

-q,--quiet

Suppress errors output.

Changed in version 3.2:Added support for-.

Changed in version 3.10:Added support for-q.

See also

Modulecompileall

Utilities to compile all Python source files in a directory tree.

compileall— Byte-compile Python libraries

Source code: `Lib/compileall.py`

This module provides some utility functions to support installing Python libraries. These functions compile Python source files in a directory tree. This module can be used to create the cached byte-code files at library installation time, which makes them available for use even by users who don't have write permission to the library directories.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Command-line use

This module can work as a script (using `python -m compileall`) to compile Python sources.

directory...

file...

Positional arguments are files to compile or directories that contain source files, traversed recursively. If no argument is given, behave as if the command line was `-l<directories from sys.path>`.

-l

Do not recurse into subdirectories, only compile source code files directly contained in the named or implied directories.

-f

Force rebuild even if timestamps are up-to-date.

-q

Do not print the list of files compiled. If passed once, error messages will still be printed. If passed twice (`-qq`), all output is suppressed.

-ddestdir

Directory prepended to the path to each file being compiled. This will appear in compilation time tracebacks, and is also compiled in to the byte-code file, where it will be used in tracebacks and other messages in cases where the source file does not exist at the time the byte-code file is executed.

-sstrip_prefix

Remove the given prefix from paths recorded in the .pyc files. Paths are made relative to the prefix. This option can be used with `-p` but not with `-d`.

-ppprepend_prefix

Prepend the given prefix to paths recorded in the .pyc files. Use `-p` to make the paths absolute. This option can be used with `-s` but not with `-d`.

-xregex

regex is used to search the full path to each file considered for compilation, and if the regex produces a match, the file is skipped.

-ilist

Read the filelist and add each line that it contains to the list of files and directories to compile. If list is -, read lines from stdin.

-b

Write the byte-code files to their legacy locations and names, which may overwrite byte-code files created by another version of Python. The default is to write files to their PEP 3147 locations and names, which allows byte-code files from multiple versions of Python to coexist.

-r

Control the maximum recursion level for subdirectories. If this is given, then -l option will not be taken into account. `python -m compileall <directory> -r 0` is equivalent to `python -m compileall <directory> -l`.

-jN

Use `N` workers to compile the files within the given directory. If 0 is used, then the result of `os.process_cpu_count()` will be used.

--invalidation-mode[timestamp|checked-hash|unchecked-hash]

Control how the generated byte-code files are invalidated at runtime. The `timestamp` value, means that .pyc files with the source timestamp and size embedded will be generated. The `checked-hash` and `unchecked-hash` values cause hash-based pycs to be generated. Hash-based pycs embed a hash of the source file contents rather than a timestamp. See `Cached bytecode invalidation` for more information on how Python validates bytecode cache files at runtime. The default is `timestamp` if the `SOURCE_DATE_EPOCH` environment variable

-o level

Compile with the given optimization level. May be used multiple times to compile for multiple levels at a time (for example, `compileall -o1 -o2`).

-edir

Ignore symlinks pointing outside the given directory.

--hardlink-dupes

If two .pyc files with different optimization level have the same content, use hard links to consolidate duplicate files.

Changed in version 3.2: Added the `-i`, `-b` and `-h` options.

Changed in version 3.5: Added the `-j`, `-r`, and `-q` options. `-q` option was changed to a multilevel value. `-b` will always produce a byte-code file ending in .pyc, never .pyo.

Changed in version 3.7: Added the `--invalidation-mode` option.

Changed in version 3.9: Added the `-s`, `-p`, `-e` and `--hardlink-dupes` options. Raised the default recursion limit from 10 to `sys.getrecursionlimit()`. Added the possibility to specify the `-o` option multiple times.

There is no command-line option to control the optimization level used by the `compile()` function, because the Python interpreter itself already provides the option: `python -O -m compileall`.

Similarly, the `compile()` function respects the `sys.pycache_prefix` setting. The generated bytecode cache will only be useful if `compile()` is run with the same `sys.pycache_prefix` (if any) that will be used at runtime.

Public functions

`compileall.compile_dir(dir, maxlevels=sys.getrecursionlimit(), ddir=None, force=False, rx=None, quiet=0, legacy=False, optimize=-1, workers=1, invalidation_mode=None, *, stripdir=None, prependdir=None, limit_sl_dest=None, hardlink_dupes=False)`

Recursively descend the directory tree named bydir, compiling all.pyfiles along the way. Return a true value if all the files compiled successfully, and a false value otherwise. Themaxlevelsparameter is used to limit the depth of the recursion; it defaults tosys.getrecursionlimit().Ifddiris given, it is prepended to the path to each file being compiled for use in compilation time tracebacks, and is also compiled in to the byte-code file, where it will be used in tracebacks and other messages in c

compileall.compile_file(fullname,ddir=None,force=False,rx=None,quiet=0,legacy=False,optimize=-1,invalidation_mode=None,*,stripdir=None,prependdir=None,limit_sl_dest=None,hardlink_dupes=False)

Compile the file with pathfullname. Return a true value if the file compiled successfully, and a false value otherwise.Ifddiris given, it is prepended to the path to the file being compiled for use in compilation time tracebacks, and is also compiled in to the byte-code file, where it will be used in tracebacks and other messages in cases where the source file does not exist at the time the byte-code file is executed.Ifrxis given, itssearchmethod is passed the full path name to the file being co

compileall.compile_path(skip_curdir=True,maxlevels=0,force=False,quiet=0,legacy=False,optimize=-1,invalidation_mode=None)

Byte-compile all the.pyfiles found alongsys.path. Return a true value if all the files compiled successfully, and a false value otherwise.Ifskip_curdiris true (the default), the current directory is not included in the search. All other parameters are passed to thecompile_dir()function. Note that unlike the other compile functions,maxlevelsdefaults to0.Changed in version 3.2:Added thelegacyandoptimizeparameter.Changed in version 3.5:quietparameter was changed to a multilevel value.Changed in v

To force a recompile of all the.pyfiles in theLib/subdirectory and all its subdirectories:

```
import compileall

compileall.compile_dir('Lib/', force=True)

# Perform same compilation, excluding files in .svn directories.
import re
compileall.compile_dir('Lib/', rx=re.compile(r'[/\\][.]svn'), force=True)

# pathlib.Path objects can also be used.
import pathlib
compileall.compile_dir(pathlib.Path('Lib/'), force=True)
```

See also

Modulepy_compile

Byte-compile a single source file.

dis— Disassembler for Python bytecode

Source code: `Lib/dis.py`

The `dis` module supports the analysis of CPython bytecode by disassembling it. The CPython bytecode which this module takes as an input is defined in the file `Include/opcode.h` and used by the compiler and the interpreter.

CPython implementation detail: Bytecode is an implementation detail of the CPython interpreter. No guarantees are made that bytecode will not be added, removed, or changed between versions of Python. Use of this module should not be considered to work across Python VMs or Python releases.

Changed in version 3.6: Use 2 bytes for each instruction. Previously the number of bytes varied by instruction.

Changed in version 3.10: The argument of jump, exception handling and loop instructions is now the instruction offset rather than the byte offset.

Changed in version 3.11: Some instructions are accompanied by one or more inline cache entries, which take the form of `CACHE` instructions. These instructions are hidden by default, but can be shown by passing `show_caches=True` to `anydisutility`. Furthermore, the interpreter now adapts the bytecode to specialize it for different runtime conditions. The adaptive bytecode can be shown by passing `adaptive=True`.

Changed in version 3.12: The argument of a jump is the offset of the target instruction relative to the instruction that appears immediately after the jump instruction's `CACHE` entries.

As a consequence, the presence of the `CACHE` instructions is transparent for forward jumps but needs to be taken into account when reasoning about backward jumps.

Changed in version 3.13: The output shows logical labels rather than instruction offsets for jump targets and exception handlers. The `-O` command line option and the `show_offsets` argument were added.

Changed in version 3.14: The `-P` command-line option and the `show_positions` argument were added.

The `-S` command-line option is added.

Example: Given the function `myfunc()`:

```
def myfunc(alist):  
    return len(alist)
```

the following command can be used to display the disassembly of `myfunc()`:

```
>>> dis.dis(myfunc)  
2           RESUME                     0  
  
3           LOAD_GLOBAL                1 (len + NULL)  
           LOAD_FAST_BORROW            0 (alist)  
           CALL                        1  
           RETURN_VALUE
```

(The “2” is a line number).

Command-line interface

The `dis` module can be invoked as a script from the command line:

```
python -m dis [-h] [-C] [-O] [-P] [-S] [infile]
```

The following options are accepted:

-h, --help

Display usage and exit.

-C,--show-caches

Show inline caches. Added in version 3.13.

-O,--show-offsets

Show offsets of instructions. Added in version 3.13.

-P,--show-positions

Show positions of instructions in the source code. Added in version 3.14.

-S,--specialized

Show specialized bytecode. Added in version 3.14.

If `infile` is specified, its disassembled code will be written to stdout. Otherwise, disassembly is performed on compiled source code received from stdin.

Bytecode analysis

Added in version 3.4.

The bytecode analysis API allows pieces of Python code to be wrapped in a `Bytecode` object that provides easy access to details of the compiled code.

`class dis.Bytecode(x, *, first_line=None, current_offset=None, show_caches=False, adaptive=False, show_offsets=False, show_positions=False)`

`classmethod from_traceback(tb, *, show_caches=False)`

`codeobj`

`first_line`

`dis()`

`info()`

Analyse the bytecode corresponding to a function, generator, asynchronous generator, coroutine, method, string of source code, or a code object (as returned by `compile()`). This is a convenience wrapper around many of the functions listed below, most notably `get_instructions()`, as iterating over a `Bytecode` instance yields the bytecode operations as `Instruction` instances. If `first_line` is not `None`, it indicates the line number that should be reported for the first source line in the disassembled code. Other

Construct a `Bytecode` instance from the given traceback, setting `current_offset` to the instruction responsible for the exception.

The compiled code object.

The first source line of the code object (if available)

Return a formatted view of the bytecode operations (the same as printed by `dis.dis()`, but returned as a multi-line string).

Return a formatted multi-line string with detailed information about the code object, like `code_info()`.

Example:

```
>>> bytecode = dis.Bytecode(myfunc)
>>> for instr in bytecode:
...     print(instr.opname)
...
RESUME
LOAD_GLOBAL
```

LOAD_FAST_BORROW
CALL
RETURN_VALUE

Analysis functions

The `dis` module also defines the following analysis functions that convert the input directly to the desired output. They can be useful if only a single operation is being performed, so the intermediate analysis object isn't useful:

`dis.code_info(x)`

Return a formatted multi-line string with detailed code object information for the supplied function, generator, asynchronous generator, coroutine, method, source code string or code object. Note that the exact contents of code info strings are highly implementation dependent and they may change arbitrarily across Python VMs or Python releases. Added in version 3.2. Changed in version 3.7: This can now handle coroutine and asynchronous generator objects.

`dis.show_code(x,*,file=None)`

Print detailed code object information for the supplied function, method, source code string or code object to file (or `sys.stdout` if file is not specified). This is a convenient shorthand for `print(code_info(x),file=file)`, intended for interactive exploration at the interpreter prompt. Added in version 3.2. Changed in version 3.4: Added `file` parameter.

`dis.dis(x=None,*,file=None,depth=None,show_caches=False,adaptive=False,show_offsets=False,show_positions=False)`

Disassemble the `x` object. `x` can denote either a module, a class, a method, a function, a generator, an asynchronous generator, a coroutine, a code object, a string of source code or a byte sequence of raw bytecode. For a module, it disassembles all functions. For a class, it disassembles all methods (including class and static methods). For a code object or sequence of raw bytecode, it prints one line per bytecode instruction. It also recursively disassembles nested code objects. These can include `g`

`dis.distb(tb=None,*,file=None,show_caches=False,adaptive=False,show_offset=False,show_positions=False)`

Disassemble the top-of-stack function of a traceback, using the last traceback if none was passed. The instruction causing the exception is indicated. The disassembly is written as text to the supplied `file` argument if provided and to `sys.stdout` otherwise. Changed in version 3.4: Added `file` parameter. Changed in version 3.11: Added the `show_caches` and `adaptive` parameters. Changed in version 3.13: Added the `show_offsets` parameter. Changed in version 3.14: Added the `show_positions` parameter.

`dis.disassemble(code,lasti=-1,*,file=None,show_caches=False,adaptive=False,show_offsets=False,show_positions=False)`

`dis.disco(code,lasti=-1,*,file=None,show_caches=False,adaptive=False,show_offsets=False,show_positions=False)`

Disassemble a code object, indicating the last instruction if `lasti` was provided. The output is divided in the following columns: the source code location of the instruction. Complete location information is shown if `show_positions` is true. Otherwise (the default) only the line number is displayed. the current instruction, indicated as `-->`, a labelled instruction, indicated with `>>`, the address of the instruction, the operation code name, operation parameters, and interpretation of the parameters in parenthesis.

`dis.get_instructions(x,*,first_line=None,show_caches=False,adaptive=False)`

Return an iterator over the instructions in the supplied function, method, source code string or code object. The iterator generates a series of `Instruction` named tuples giving the details of each operation in the supplied

code.If `first_line` is not `None`, it indicates the line number that should be reported for the first source line in the disassembled code. Otherwise, the source line information (if any) is taken directly from the disassembled code object.The `adaptive` parameter works as it does in `dis()`

dis.findlinestarts(code)

This generator function uses the `co_lines()` method of the code object `code` to find the offsets which are starts of lines in the source code. They are generated as `(offset,lineno)` pairs.Changed in version 3.6:Line numbers can be decreasing. Before, they were always increasing.Changed in version 3.10:The PEP 626 `co_lines()` method is used instead of the `co_firstlineno` and `co_lnotab` attributes of the code object.Changed in version 3.13:Line numbers can be `None` for bytecode that does not map to source lines.

dis.findlabels(code)

Detect all offsets in the raw compiled bytecode string `code` which are jump targets, and return a list of these offsets.

dis.stack_effect(opcode,oparg=None,*,jump=None)

Compute the stack effect of `opcode` with argument `oparg`.If the code has a jump target and `jump` is `True`, `stack_effect()` will return the stack effect of jumping. If `jump` is `False`, it will return the stack effect of not jumping. And if `jump` is `None` (default), it will return the maximal stack effect of both cases.Added in version 3.4.Changed in version 3.8:Added `jump` parameter.Changed in version 3.13:If `oparg` is omitted (or `None`), the stack effect is now returned for `oparg=0`. Previously this was an error for opcodes tha

Python Bytecode Instructions

The `get_instructions()` function and `Bytecode` class provide details of bytecode instructions as `Instruction` instances:

class dis.Instruction

opcode

opname

baseopcode

baseopname

arg

oparg

argval

argrepr

offset

start_offset

cache_offset

end_offset

starts_line

line_number

is_jump_target

jump_target

positions

cache_info

Details for a bytecode operation
opcode numeric code for operation, corresponding to the opcode values listed below and the bytecode values in the `Opcode` collections.
opname human readable name for operation
baseopcode numeric code for the base operation if operation is specialized; otherwise equal to `opcode`
baseopname human readable name for the base operation if operation is specialized; otherwise equal to `opname`
arg numeric argument to operation (if any), otherwise `None`
argalias for `arg` if `arg` is a resolved argument value

numeric code for operation, corresponding to the opcode values listed below and the bytecode values in the `Opcode` collections.

human readable name for operation

numeric code for the base operation if operation is specialized; otherwise equal to `opcode`

human readable name for the base operation if operation is specialized; otherwise equal to `opname`

numeric argument to operation (if any), otherwise `None`

alias for `arg`

resolved arg value (if any), otherwise `None`

human readable description of operation argument (if any), otherwise an empty string.

start index of operation within bytecode sequence

start index of operation within bytecode sequence, including prefixed `EXTENDED_ARG` operations if present; otherwise equal to `offset`

start index of the cache entries following the operation

end index of the cache entries following the operation

`True` if this opcode starts a source line, otherwise `False`

source line number associated with this opcode (if any), otherwise `None`

`True` if other code jumps to here, otherwise `False`

bytecode index of the jump target if this is a jump operation, otherwise `None`

`dis.Positions` object holding the start and end locations that are covered by this instruction.

Information about the cache entries of this instruction, as triplets of the form `(name, size, data)`, where `name` and `size` describe the cache format and `data` is the contents of the cache.
`cache_info` is `None` if the instruction does not have caches.

class dis.Positions

lineno

end_lineno

col_offset

end_col_offset

In case the information is not available, some fields might be `None`.
`lineno`, `end_lineno`, `col_offset`, `end_col_offset` Added in version 3.11.

The Python compiler currently generates the following bytecode instructions.

General instructions

In the following, We will refer to the interpreter stack as `STACK` and describe operations on it as if it was a Python list. The top of the stack corresponds to `STACK[-1]` in this language.

NOP

Do nothing code. Used as a placeholder by the bytecode optimizer, and to generate line tracing events.

NOT_TAKEN

Do nothing code. Used by the interpreter to record `BRANCH_LEFT` and `BRANCH_RIGHT` events for `sys.monitoring`. Added in version 3.14.

POP_ITER

Removes the iterator from the top of the stack. Added in version 3.14.

POP_TOP

Removes the top-of-stack item: `STACK.pop()`

END_FOR

Removes the top-of-stack item. Equivalent to `POP_TOP`. Used to clean up at the end of loops, hence the name. Added in version 3.12.

END_SEND

Implements `del STACK[-2]`. Used to clean up when a generator exits. Added in version 3.12.

COPY(i)

Push the *i*-th item to the top of the stack without removing it from its original location: `assert i > 0; STACK.append(STACK[-i])` Added in version 3.11.

SWAP(i)

Swap the top of the stack with the *i*-th element: `STACK[-i], STACK[-1] = STACK[-1], STACK[-i]` Added in version 3.11.

CACHE

Rather than being an actual instruction, this opcode is used to mark extra space for the interpreter to cache useful data directly in the bytecode itself. It is automatically hidden by all disutilities, but can be viewed with `show_caches=True`. Logically, this space is part of the preceding instruction. Many opcodes expect to be followed by an exact number of caches, and will instruct the interpreter to skip over them at runtime. Populated caches can look like arbitrary instructions, so great care sh

Unary operations

Unary operations take the top of the stack, apply the operation, and push the result back on the stack.

UNARY_NEGATIVE

Implements `STACK[-1] = -STACK[-1]`.

UNARY_NOT

Implements `STACK[-1] = not STACK[-1]`. Changed in version 3.13: This instruction now requires an exact boolean operand.

UNARY_INVERT

Implements `STACK[-1] = ~STACK[-1]`.

GET_ITER

Implements `STACK[-1] = iter(STACK[-1])`.

GET_YIELD_FROM_ITER

If `STACK[-1]` is a generator, iterator or coroutine object it is left as is. Otherwise, implements `STACK[-1]=iter(STACK[-1])`. Added in version 3.5.

TO_BOOL

Implements `STACK[-1]=bool(STACK[-1])`. Added in version 3.13.

Binary and in-place operations

Binary operations remove the top two items from the stack (`STACK[-1]` and `STACK[-2]`). They perform the operation, then put the result back on the stack.

In-place operations are like binary operations, but the operation is done in-place when `STACK[-2]` supports it, and the resulting `STACK[-1]` may be (but does not have to be) the original `STACK[-2]`.

BINARY_OP(op)

Implements the binary and in-place operators (depending on the value of `op`): `rhs=STACK.pop()` `lhs=STACK.pop()` `STACK.append(lhsoprhs)` Added in version 3.11. Changed in version 3.14: With `oparg :NB_SUBSCR`, implements binary subscript (replaces opcode `BINARY_SUBSCR`)

STORE_SUBSCR

Implements: `key=STACK.pop()` `container=STACK.pop()` `value=STACK.pop()` `container[key]=value`

DELETE_SUBSCR

Implements: `key=STACK.pop()` `container=STACK.pop()` `del container[key]`

BINARY_SLICE

Implements: `end=STACK.pop()` `start=STACK.pop()` `container=STACK.pop()` `STACK.append(container[start:end])` Added in version 3.12.

STORE_SLICE

Implements: `end=STACK.pop()` `start=STACK.pop()` `container=STACK.pop()` `value=STACK.pop()` `container[start:end]=value` Added in version 3.12.

Coroutine opcodes

GET_AWAITABLE(where)

Implements `STACK[-1]=get_awaitable(STACK[-1])`, where `get_awaitable(o)` returns `o` if `o` is a coroutine object or a generator object with the `CO_ITERABLE_COROUTINE` flag, or resolves `o.__await__`. If the `where` operand is nonzero, it indicates where the instruction occurs: 1: After a call to `__aenter__` 2: After a call to `__aexit__` Added in version 3.5. Changed in version 3.11: Previously, this instruction did not have an `oparg`.

GET_AITER

Implements `STACK[-1]=STACK[-1].__aiter__()`. Added in version 3.5. Changed in version 3.7: Returning awaitable objects from `__aiter__` is no longer supported.

GET_ANEXT

Implement `STACK.append(get_awaitable(STACK[-1].__anext__()))` to the stack. See `GET_AWAITABLE` for details about `get_awaitable`. Added in version 3.5.

END_ASYNC_FOR

Terminates an `async for` loop. Handles an exception raised when awaiting a next item. The stack contains the `async iterable` in `STACK[-2]` and the raised exception in `STACK[-1]`. Both are popped. If the exception is not `StopAsyncIteration`, it is re-raised. Added in version 3.8. Changed in version 3.11: Exception representation on the stack now consist of one, not three, items.

CLEANUP_THROW

Handles an exception raised during `athrow()` or `close()` call through the current frame. If `STACK[-1]` is an instance of `StopIteration`, pop three values from the stack and push its `value` member. Otherwise, re-raise `STACK[-1]`. Added in version 3.12.

Miscellaneous opcodes

SET_ADD(i)

Implements: `item=STACK.pop(); set.add(STACK[-i], item)` Used to implement set comprehensions.

LIST_APPEND(i)

Implements: `item=STACK.pop(); list.append(STACK[-i], item)` Used to implement list comprehensions.

MAP_ADD(i)

Implements: `value=STACK.pop(); key=STACK.pop(); dict.__setitem__(STACK[-i], key, value)` Used to implement dict comprehensions. Added in version 3.1. Changed in version 3.8: Map value is `STACK[-1]` and map key is `STACK[-2]`. Before, those were reversed.

For all of the `SET_ADD`, `LIST_APPEND` and `MAP_ADD` instructions, while the added value or key/value pair is popped off, the container object remains on the stack so that it is available for further iterations of the loop.

RETURN_VALUE

Returns with `STACK[-1]` to the caller of the function.

YIELD_VALUE

Yields `STACK.pop()` from a generator. Changed in version 3.11: `oparg` set to be the stack depth. Changed in version 3.12: `oparg` set to be the exception block depth, for efficient closing of generators. Changed in version 3.13: `oparg` is 1 if this instruction is part of a `yield-from` or `await`, and 0 otherwise.

SETUP_ANNOTATIONS

Checks whether `__annotations__` is defined in `locals()`, if not it is set up to an empty dict. This opcode is only emitted if a class or module body contains variable annotations statically. Added in version 3.6.

POP_EXCEPT

Pops a value from the stack, which is used to restore the exception state. Changed in version 3.11: Exception representation on the stack now consist of one, not three, items.

RERAISE

Re-raises the exception currently on top of the stack. If `oparg` is non-zero, pops an additional value from the stack which is used to set `self_lasti` of the current frame. Added in version 3.9. Changed in version 3.11: Exception representation on the stack now consist of one, not three, items.

PUSH_EXC_INFO

Pops a value from the stack. Pushes the current exception to the top of the stack. Pushes the value originally popped back to the stack. Used in exception handlers. Added in version 3.11.

CHECK_EXC_MATCH

Performs exception matching for `except`. Tests whether the `STACK[-2]` is an exception matching `STACK[-1]`. Pops `STACK[-1]` and pushes the boolean result of the test. Added in version 3.11.

CHECK_EG_MATCH

Performs exception matching for `except*`. Applies `split(STACK[-1])` on the exception group representing `STACK[-2]`. In case of a match, pops two items from the stack and pushes the non-matching

subgroup (None in case of full match) followed by the matching subgroup. When there is no match, pops one item (the match type) and pushes None. Added in version 3.11.

WITH_EXCEPT_START

Calls the function in position 4 on the stack with arguments (type, val, tb) representing the exception at the top of the stack. Used to implement the `callcontext_manager.__exit__(*exc_info())` when an exception has occurred in `awithstatement`. Added in version 3.9. Changed in version 3.11: The `__exit__` function is in position 4 of the stack rather than 7. Exception representation on the stack now consist of one, not three, items.

LOAD_COMMON_CONSTANT

Pushes a common constant onto the stack. The interpreter contains a hardcoded list of constants supported by this instruction. Used by the `assertstatement` to load `AssertionError`. Added in version 3.14.

LOAD_BUILD_CLASS

Pushes `builtins.__build_class__()` onto the stack. It is later called to construct a class.

GET_LEN

Perform `STACK.append(len(STACK[-1]))`. Used in `matchstatements` where comparison with structure of pattern is needed. Added in version 3.10.

MATCH_MAPPING

If `STACK[-1]` is an instance of `collections.abc.Mapping` (or, more technically: if it has the `Py_TPFLAGS_MAPPING` flag set in its `tp_flags`), push `True` onto the stack. Otherwise, push `False`. Added in version 3.10.

MATCH_SEQUENCE

If `STACK[-1]` is an instance of `collections.abc.Sequence` and is not an instance of `str/bytes/bytearray` (or, more technically: if it has the `Py_TPFLAGS_SEQUENCE` flag set in its `tp_flags`), push `True` onto the stack. Otherwise, push `False`. Added in version 3.10.

MATCH_KEYS

`STACK[-1]` is a tuple of mapping keys, and `STACK[-2]` is the match subject. If `STACK[-2]` contains all of the keys in `STACK[-1]`, push a tuple containing the corresponding values. Otherwise, push `None`. Added in version 3.10. Changed in version 3.11: Previously, this instruction also pushed a boolean value indicating success (`True`) or failure (`False`).

STORE_NAME(namei)

Implements `name=STACK.pop().namei` is the index of `name` in the `attributeco_names` of the code object. The compiler tries to use `STORE_FAST` or `STORE_GLOBAL` if possible.

DELETE_NAME(namei)

Implements `del name`, where `namei` is the index into `co_names` attribute of the code object.

UNPACK_SEQUENCE(count)

Unpacks `STACK[-1]` into `count` individual values, which are put onto the stack right-to-left. Require there to be exactly `count` values: `assert(len(STACK[-1])==count) STACK.extend(STACK.pop()[::-count-1:-1])`

UNPACK_EX(counts)

Implements assignment with a starred target: Unpacks an iterable in `STACK[-1]` into individual values, where the total number of values can be smaller than the number of items in the iterable: one of the new values will be a list of all leftover items. The number of values before and after the list value is limited to 255. The number of values before the list value is encoded in the argument of the opcode. The number of values after the list if any

is encoded using an `EXTENDED_ARG`. As a consequence, t

STORE_ATTR(namei)

Implements: `obj=STACK.pop()value=STACK.pop()obj.name=value` where `namei` is the index of `name` in `co_names` of the code object.

DELETE_ATTR(namei)

Implements: `obj=STACK.pop()del obj.name` where `namei` is the index of `name` in `co_names` of the code object.

STORE_GLOBAL(namei)

Works as `STORE_NAME`, but stores the name as a global.

DELETE_GLOBAL(namei)

Works as `DELETE_NAME`, but deletes a global name.

LOAD_CONST(consti)

Pushes `co_consts[consti]` onto the stack.

LOAD_SMALL_INT(i)

Pushes the integer `i` onto the stack. `i` must be in `range(256)`. Added in version 3.14.

LOAD_NAME(namei)

Pushes the value associated with `co_names[namei]` onto the stack. The name is looked up within the locals, then the globals, then the builtins.

LOAD_LOCALS

Pushes a reference to the locals dictionary onto the stack. This is used to prepare namespace dictionaries for `LOAD_FROM_DICT_OR_DEREF` and `LOAD_FROM_DICT_OR_GLOBALS`. Added in version 3.12.

LOAD_FROM_DICT_OR_GLOBALS(i)

Pops a mapping off the stack and looks up the value for `co_names[namei]`. If the name is not found there, looks it up in the globals and then the builtins, similar to `LOAD_GLOBAL`. This is used for loading global variables in annotation scopes within class bodies. Added in version 3.12.

BUILD_TEMPLATE

Constructs a new `Template` instance from a tuple of strings and a tuple of interpolations and pushes the resulting object onto the stack: `interpolations=STACK.pop()strings=STACK.pop()STACK.append(_build_template(strings,interpolations))`. Added in version 3.14.

BUILD_INTERPOLATION(format)

Constructs a new `Interpolation` instance from a value and its source expression and pushes the resulting object onto the stack. If no conversion or format specification is present, `format` is set to `2`. If the low bit of `format` is set, it indicates that the interpolation contains a format specification. If `format >> 2` is non-zero, it indicates that the interpolation contains a conversion. The value of `format >> 2` is the conversion type (0 for no conversion, 1 for `s`, 2 for `r`, and 3 for `a`): `conversion=format >> 2 if format & 1: form`

BUILD_TUPLE(count)

Creates a tuple consuming `count` items from the stack, and pushes the resulting tuple onto the stack: `if count==0: value=() else: value=tuple(STACK[-count:]) STACK=STACK[:-count] STACK.append(value)`

BUILD_LIST(count)

Works as `BUILD_TUPLE`, but creates a list.

BUILD_SET(count)

Works as **BUILD_TUPLE**, but creates a set.

BUILD_MAP(count)

Pushes a new dictionary object onto the stack. Pops $2 * \text{count}$ items so that the dictionary holds count entries: $\{ \dots, \text{STACK}[-4]:\text{STACK}[-3], \text{STACK}[-2]:\text{STACK}[-1] \}$. Changed in version 3.5: The dictionary is created from stack items instead of creating an empty dictionary pre-sized to hold count items.

BUILD_STRING(count)

Concatenates count strings from the stack and pushes the resulting string onto the stack. Added in version 3.6.

LIST_EXTEND(i)

Implements: $\text{seq} = \text{STACK.pop}()$ $\text{list.extend}(\text{STACK}[-i], \text{seq})$ Used to build lists. Added in version 3.9.

SET_UPDATE(i)

Implements: $\text{seq} = \text{STACK.pop}()$ $\text{set.update}(\text{STACK}[-i], \text{seq})$ Used to build sets. Added in version 3.9.

DICT_UPDATE(i)

Implements: $\text{map} = \text{STACK.pop}()$ $\text{dict.update}(\text{STACK}[-i], \text{map})$ Used to build dicts. Added in version 3.9.

DICT_MERGE(i)

Like **DICT_UPDATE** but raises an exception for duplicate keys. Added in version 3.9.

LOAD_ATTR(namei)

If the low bit of namei is not set, this replaces $\text{STACK}[-1]$ with $\text{getattr}(\text{STACK}[-1], \text{co_names}[\text{namei} >> 1])$. If the low bit of namei is set, this will attempt to load a method named $\text{co_names}[\text{namei} >> 1]$ from the $\text{STACK}[-1]$ object. $\text{STACK}[-1]$ is popped. This bytecode distinguishes two cases: if $\text{STACK}[-1]$ has a method with the correct name, the bytecode pushes the unbound method and $\text{STACK}[-1]$. $\text{STACK}[-1]$ will be used as the first argument (self) by **CALL** or **CALL_KW** when calling the unbound method. Otherwise, **NULL** and the object returned by getattr .

LOAD_SUPER_ATTR(namei)

This opcode implements **super()**, both in its zero-argument and two-argument forms (e.g. **super().method()**, **super().attr** and **super(cls, self).method()**, **super(cls, self).attr**). It pops three values from the stack (from top of stack down): self: the first argument to the current method; cls: the class within which the current method was defined; the global **super**. With respect to its argument, it works similarly to **LOAD_ATTR**, except that namei is shifted left by 2 bits instead of 1. The low bit of namei signals to attempt a

COMPARE_OP(opname)

Performs a Boolean operation. The operation name can be found in **cmp_op** [$\text{opname} >> 5$]. If the fifth-lowest bit of opname is set ($\text{opname} \& 16$), the result should be coerced to **bool**. Changed in version 3.13: The fifth-lowest bit of the **oparg** now indicates a forced conversion to **bool**.

IS_OP(invert)

Performs **is** comparison, or **isnot** if invert is 1. Added in version 3.9.

CONTAINS_OP(invert)

Performs **in** comparison, or **notin** if invert is 1. Added in version 3.9.

IMPORT_NAME(namei)

Imports the module $\text{co_names}[\text{namei}]$. $\text{STACK}[-1]$ and $\text{STACK}[-2]$ are popped and provide the fromlist and level arguments of **__import__()**. The module object is pushed onto the stack. The current namespace is not affected: for a proper import statement, a subsequent **STORE_FAST** instruction modifies the

namespace.

IMPORT_FROM(namei)

Loads the attribute `co_names[namei]` from the module found in `STACK[-1]`. The resulting object is pushed onto the stack, to be subsequently stored by a `STORE_FAST` instruction.

JUMP_FORWARD(delta)

Increments bytecode counter by `delta`.

JUMP_BACKWARD(delta)

Decrements bytecode counter by `delta`. Checks for interrupts. Added in version 3.11.

JUMP_BACKWARD_NO_INTERRUPT(delta)

Decrements bytecode counter by `delta`. Does not check for interrupts. Added in version 3.11.

POP_JUMP_IF_TRUE(delta)

If `STACK[-1]` is true, increments the bytecode counter by `delta`. `STACK[-1]` is popped. Changed in version 3.11: The oparg is now a relative delta rather than an absolute target. This opcode is a pseudo-instruction, replaced in final bytecode by the directed versions (forward/backward). Changed in version 3.12: This is no longer a pseudo-instruction. Changed in version 3.13: This instruction now requires an exact bool operand.

POP_JUMP_IF_FALSE(delta)

If `STACK[-1]` is false, increments the bytecode counter by `delta`. `STACK[-1]` is popped. Changed in version 3.11: The oparg is now a relative delta rather than an absolute target. This opcode is a pseudo-instruction, replaced in final bytecode by the directed versions (forward/backward). Changed in version 3.12: This is no longer a pseudo-instruction. Changed in version 3.13: This instruction now requires an exact bool operand.

POP_JUMP_IF_NOT_NONE(delta)

If `STACK[-1]` is not None, increments the bytecode counter by `delta`. `STACK[-1]` is popped. Added in version 3.11. Changed in version 3.12: This is no longer a pseudo-instruction.

POP_JUMP_IF_NONE(delta)

If `STACK[-1]` is None, increments the bytecode counter by `delta`. `STACK[-1]` is popped. Added in version 3.11. Changed in version 3.12: This is no longer a pseudo-instruction.

FOR_ITER(delta)

`STACK[-1]` is an iterator. Call its `__next__()` method. If this yields a new value, push it on the stack (leaving the iterator below it). If the iterator indicates it is exhausted then the byte code counter is incremented by `delta`. Changed in version 3.12: Up until 3.11 the iterator was popped when it was exhausted.

LOAD_GLOBAL(namei)

Loads the global named `co_names[namei >> 1]` onto the stack. Changed in version 3.11: If the low bit of `namei` is set, then a `NULL` is pushed to the stack before the global variable.

LOAD_FAST(var_num)

Pushes a reference to the local `co_varnames[var_num]` onto the stack. Changed in version 3.12: This opcode is now only used in situations where the local variable is guaranteed to be initialized. It cannot raise `UnboundLocalError`.

LOAD_FAST_BORROW(var_num)

Pushes a borrowed reference to the local `co_varnames[var_num]` onto the stack. Added in version 3.14.

LOAD_FAST_LOAD_FAST(var_nums)

Pushes references `toco_varnames[var_nums>>4]` and `co_varnames[var_nums&15]` onto the stack. Added in version 3.13.

LOAD_FAST_BORROW_LOAD_FAST_BORROW(var_nums)

Pushes borrowed references `toco_varnames[var_nums>>4]` and `co_varnames[var_nums&15]` onto the stack. Added in version 3.14.

LOAD_FAST_CHECK(var_num)

Pushes a reference to the local `co_varnames[var_num]` onto the stack, raising an `UnboundLocalError` if the local variable has not been initialized. Added in version 3.12.

LOAD_FAST_AND_CLEAR(var_num)

Pushes a reference to the local `co_varnames[var_num]` onto the stack (or pushes `NULL` onto the stack if the local variable has not been initialized) and sets `co_varnames[var_num]` to `NULL`. Added in version 3.12.

STORE_FAST(var_num)

Stores `STACK.pop()` into the local `co_varnames[var_num]`.

STORE_FAST_STORE_FAST(var_nums)

Stores `STACK[-1]` into `co_varnames[var_nums>>4]` and `STACK[-2]` into `co_varnames[var_nums&15]`. Added in version 3.13.

STORE_FAST_LOAD_FAST(var_nums)

Stores `STACK.pop()` into the local `co_varnames[var_nums>>4]` and pushes a reference to the local `co_varnames[var_nums&15]` onto the stack. Added in version 3.13.

DELETE_FAST(var_num)

Deletes local `co_varnames[var_num]`.

MAKE_CELL(i)

Creates a new cell in slot `i`. If that slot is nonempty then that value is stored into the new cell. Added in version 3.11.

LOAD_DEREF(i)

Loads the cell contained in slot `i` of the “fast locals” storage. Pushes a reference to the object the cell contains on the stack. Changed in version 3.11: `i` is no longer offset by the length of `co_varnames`.

LOAD_FROM_DICT_OR_DEREF(i)

Pops a mapping off the stack and looks up the name associated with slot `i` of the “fast locals” storage in this mapping. If the name is not found there, loads it from the cell contained in slot `i`, similar to `LOAD_DEREF`. This is used for loading closure variables in class bodies (which previously used `LOAD_CLASSDEREF`) and in annotation scopes within class bodies. Added in version 3.12.

STORE_DEREF(i)

Stores `STACK.pop()` into the cell contained in slot `i` of the “fast locals” storage. Changed in version 3.11: `i` is no longer offset by the length of `co_varnames`.

DELETE_DEREF(i)

Empties the cell contained in slot `i` of the “fast locals” storage. Used by the `del` statement. Added in version 3.2. Changed in version 3.11: `i` is no longer offset by the length of `co_varnames`.

COPY_FREE_VARS(n)

Copies then free (closure) variables from the closure into the frame. Removes the need for special code on the caller's side when calling closures. Added in version 3.11.

RAISE_VARARGS(argc)

Raises an exception using one of the 3 forms of the raise statement, depending on the value of `argc`: 0: raise(re-raise previous exception) 1: raise(`STACK[-1]`)(raise exception instance or type at `STACK[-1]`) 2: raise(`STACK[-2]` from `STACK[-1]`)(raise exception instance or type at `STACK[-2]` with `__cause__` set to `STACK[-1]`)

CALL(argc)

Calls a callable object with the number of arguments specified by `argc`. On the stack are (in ascending order): The callable itself for `NULL`. The remaining positional arguments `argc` is the total of the positional arguments, excluding self. `CALL` pops all arguments and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object. Added in version 3.11. Changed in version 3.13: The callable now always appears at the same position on the

CALL_KW(argc)

Calls a callable object with the number of arguments specified by `argc`, including one or more named arguments. On the stack are (in ascending order): The callable itself for `NULL`. The remaining positional arguments The named arguments A tuple of keyword argument names `argc` is the total of the positional and named arguments, excluding self. The length of the tuple of keyword argument names is the number of named arguments. `CALL_KW` pops all arguments, the keyword names, and the callable object off the stack, calls t

CALL_FUNCTION_EX(flags)

Calls a callable object with variable set of positional and keyword arguments. If the lowest bit of `flags` is set, the top of the stack contains a mapping object containing additional keyword arguments. Before the callable is called, the mapping object and iterable object are each “unpacked” and their contents passed in as keyword and positional arguments respectively. `CALL_FUNCTION_EX` pops all arguments and the callable object off the stack, calls the callable object with those arguments, and pushes

PUSH_NULL

Pushes a `NULL` to the stack. Used in the call sequence to match the `NULL` pushed by `LOAD_METHOD` for non-method calls. Added in version 3.11.

MAKE_FUNCTION

Pushes a new function object on the stack built from the code object at `STACK[-1]`. Changed in version 3.10: Flag value `0x04` is a tuple of strings instead of dictionary Changed in version 3.11: Qualified name at `STACK[-1]` was removed. Changed in version 3.13: Extra function attributes on the stack, signaled by `oparg` flags, were removed. They now use `SET_FUNCTION_ATTRIBUTE`.

SET_FUNCTION_ATTRIBUTE(flag)

Sets an attribute on a function object. Expects the function at `STACK[-1]` and the attribute value to set at `STACK[-2]`; consumes both and leaves the function at `STACK[-1]`. The flag determines which attribute to set: `0x01` a tuple of default values for positional-only and positional-or-keyword parameters in positional order `0x02` a dictionary of keyword-only parameters' default values `0x04` a tuple of strings containing parameters' annotations `0x08` a tuple containing cells for free variables, making a closure `0x1`

BUILD_SLICE(argc)

Pushes a slice object on the stack. `argc` must be 2 or 3. If it is 2, implements: `end=STACK.pop()` `start=STACK.pop()` `STACK.append(slice(start,end))` if it is 3, implements: `step=STACK.pop()` `end=STACK.pop()` `start=STACK.pop()` `STACK.append(slice(start,end,step))` See the `slice()` built-in function for more information.

EXTENDED_ARG(ext)

Prefixes any opcode which has an argument too big to fit into the default one byte. `ext` holds an additional byte which act as higher bits in the argument. For each opcode, at most three prefixal `EXTENDED_ARG` are allowed, forming an argument from two-byte to four-byte.

CONVERT_VALUE(oparg)

Convert value to a string, depending

`onoparg:value=STACK.pop()result=func(value)STACK.append(result)`
`oparg==1: callstr()`
`onvalueoparg==2: callrepr()`
`onvalueoparg==3: callascii()`
`onvalue` Used for implementing formatted string literals (f-strings). Added in version 3.13.

FORMAT_SIMPLE

Formats the value on top of stack: `value=STACK.pop()result=value.__format__("")STACK.append(result)` Used for implementing formatted string literals (f-strings). Added in version 3.13.

FORMAT_WITH_SPEC

Formats the given value with the given format

`spec:spec=STACK.pop()value=STACK.pop()result=value.__format__(spec)STACK.append(result)` Used for implementing formatted string literals (f-strings). Added in version 3.13.

MATCH_CLASS(count)

`STACK[-1]` is a tuple of keyword attribute names, `STACK[-2]` is the class being matched against, and `STACK[-3]` is the match subject. `count` is the number of positional sub-patterns. `PopSTACK[-1],STACK[-2], andSTACK[-3]`. If `STACK[-3]` is an instance of `STACK[-2]` and has the positional and keyword attributes required by `count` and `STACK[-1]`, push a tuple of extracted attributes. Otherwise, push `None`. Added in version 3.10. Changed in version 3.11: Previously, this instruction also pushed a boolean value indicating success

RESUME(context)

A no-op. Performs internal tracing, debugging and optimization checks. The `context` operand consists of two parts. The lowest two bits indicate where the `RESUME` occurs: 0 The start of a function, which is neither a generator, coroutine nor an async generator 1 After `await expression` 2 After `await from expression` 3 After `await expression` The next bit is 1 if the `RESUME` is at except-depth 1, and 0 otherwise. Added in version 3.11. Changed in version 3.13: The `oparg` value changed to include information about except-depth

RETURN_GENERATOR

Create a generator, coroutine, or async generator from the current frame. Used as first opcode of in code object for the above mentioned callables. Clear the current frame and return the newly created generator. Added in version 3.11.

SEND(delta)

Equivalent to `STACK[-1]=STACK[-2].send(STACK[-1])`. Used in `yield from` and `await` statements. If the call raises `StopIteration`, pop the top value from the stack, push the exception's `value` attribute, and increment the bytecode counter by `delta`. Added in version 3.11.

HAVE_ARGUMENT

This is not really an opcode. It identifies the dividing line between opcodes in the range `[0,255]` which don't use their argument and those that do (`<HAVE_ARGUMENT` and `>=HAVE_ARGUMENT`, respectively). If your application uses pseudo instructions or specialized instructions, use the `hasarg` collection instead. Changed in version 3.6: Now every instruction has an argument, but opcodes `<HAVE_ARGUMENT` ignore it. Before, only opcodes `>=HAVE_ARGUMENT` had an argument. Changed in version 3.12: Pseudo instructions were

CALL_INTRINSIC_1

Calls an intrinsic function with one argument. Passes `STACK[-1]` as the argument and sets `STACK[-1]` to the result. Used to implement functionality that is not performance critical. The operand determines which intrinsic function is called: `OperandDescription` `INTRINSIC_1_INVALID` Not valid `INTRINSIC_PRINT` Prints the argument to standard out. Used in the REPL. `INTRINSIC_IMPORT_STAR` Performs `import*` for the named module. `INTRINSIC_STOPITERATION_ERROR` Extracts the return value from a `StopIteration` exception. `INTRINSIC_A`

CALL_INTRINSIC_2

Calls an intrinsic function with two arguments. Used to implement functionality that is not performance critical: `arg2=STACK.pop()` `arg1=STACK.pop()` `result=intrinsic2(arg1,arg2)` `STACK.append(result)` The operand determines which intrinsic function is called: `OperandDescription` `INTRINSIC_2_INVALID` Not valid `INTRINSIC_PREP_RERAISE_STAR` Calculates the `ExceptionGroup` to raise from `try-except*`. `INTRINSIC_TYPEVAR_WITH_BOUND` Creates a `typing.TypeVar` with a bound. `INTRINSIC_TYPEVAR_WITH_CONSTRAINTS` Creates a `typing.TypeVar`

LOAD_SPECIAL

Performs special method lookup on `STACK[-1]`. If `type(STACK[-1]).__xxx__` is a method, leave `type(STACK[-1]).__xxx__`; `STACK[-1]` on the stack. If `type(STACK[-1]).__xxx__` is not a method, leave `STACK[-1].__xxx__`; `NULL` on the stack. Added in version 3.14.

Pseudo-instructions

These opcodes do not appear in Python bytecode. They are used by the compiler but are replaced by real opcodes or removed before bytecode is generated.

SETUP_FINALLY(target)

Set up an exception handler for the following code block. If an exception occurs, the value stack level is restored to its current state and control is transferred to the exception handler at `target`.

SETUP_CLEANUP(target)

Like `SETUP_FINALLY`, but in case of an exception also pushes the last instruction (`lasti`) to the stack so that `RERAISE` can restore it. If an exception occurs, the value stack level and the last instruction on the frame are restored to their current state, and control is transferred to the exception handler at `target`.

SETUP_WITH(target)

Like `SETUP_CLEANUP`, but in case of an exception one more item is popped from the stack before control is transferred to the exception handler at `target`. This variant is used in `with` and `async with` constructs, which push the return value of the context manager's `__enter__()` or `__aenter__()` to the stack.

POP_BLOCK

Marks the end of the code block associated with the last `SETUP_FINALLY`, `SETUP_CLEANUP` or `SETUP_WITH`.

LOAD_CONST_IMMORTAL(consti)

Works as `LOAD_CONST`, but is more efficient for immortal objects.

JUMP

JUMP_NO_INTERRUPT

Undirected relative jump instructions which are replaced by their directed (forward/backward) counterparts by the assembler.

JUMP_IF_TRUE

JUMP_IF_FALSE

Conditional jumps which do not impact the stack. Replaced by the sequence `COPY1, TO_BOOL, POP_JUMP_IF_TRUE/FALSE`.

LOAD_CLOSURE(i)

Pushes a reference to the cell contained in slot `i` of the “fast locals” storage. Note that `LOAD_CLOSURE` is replaced with `LOAD_FAST` in the assembler. Changed in version 3.13: This opcode is now a pseudo-instruction.

Opcode collections

These collections are provided for automatic introspection of bytecode instructions:

Changed in version 3.12: The collections now contain pseudo instructions and instrumented instructions as well. These are opcodes with values $\geq \text{MIN_PSEUDO_OPCODE}$ and $\geq \text{MIN_INSTRUMENTED_OPCODE}$.

dis.opname

Sequence of operation names, indexable using the bytecode.

dis.opmap

Dictionary mapping operation names to bytecodes.

dis.cmp_op

Sequence of all compare operation names.

dis.hasarg

Sequence of bytecodes that use their argument. Added in version 3.12.

dis.hasconst

Sequence of bytecodes that access a constant.

dis.hasfree

Sequence of bytecodes that access a free (closure) variable. ‘free’ in this context refers to names in the current scope that are referenced by inner scopes or names in outer scopes that are referenced from this scope. It does not include references to global or builtin scopes.

dis.hasname

Sequence of bytecodes that access an attribute by name.

dis.hasjump

Sequence of bytecodes that have a jump target. All jumps are relative. Added in version 3.13.

dis.haslocal

Sequence of bytecodes that access a local variable.

dis.hascompare

Sequence of bytecodes of Boolean operations.

dis.hasexc

Sequence of bytecodes that set an exception handler. Added in version 3.12.

dis.hasjrel

Sequence of bytecodes that have a relative jump target. Deprecated since version 3.13: All jumps are now relative. Use `hasjump`.

dis.hasjabs

Sequence of bytecodes that have an absolute jump target.Deprecated since version 3.13:All jumps are now relative. This list is empty.

pickletools— Tools for pickle developers

Source code: `Lib/pickletools.py`

This module contains various constants relating to the intimate details of the `pickle` module, some lengthy comments about the implementation, and a few useful functions for analyzing pickled data. The contents of this module are useful for Python core developers who are working on the pickle; ordinary users of the `pickle` module probably won't find the `pickletools` module relevant.

Command-line usage

Added in version 3.2.

When invoked from the command line, `python-mpickletools` will disassemble the contents of one or more pickle files. Note that if you want to see the Python object stored in the pickle rather than the details of pickle format, you may want to use `mpickle` instead. However, when the pickle file that you want to examine comes from an untrusted source, `mpickletools` is a safer option because it does not execute pickle bytecode.

For example, with a tuple `(1,2)` pickled in file `x.pickle`:

```
$ python -m pickle x.pickle
(1, 2)
```

```
$ python -m pickletools x.pickle
0: \x80 PROTO      3
2: K    BININT1    1
4: K    BININT1    2
6: \x86 TUPLE2
7: q    BININPUT   0
9: .    STOP
highest protocol among opcodes = 2
```

Command-line options

-a,--annotate

Annotate each line with a short opcode description.

-o,--output=<file>

Name of a file where the output should be written.

-l,--indentlevel=<num>

The number of blanks by which to indent a new MARK level.

-m,--memo

When multiple objects are disassembled, preserve memo between disassemblies.

-p,--preamble=<preamble>

When more than one pickle file is specified, print given preamble before each disassembly.

pickle_file

A pickle file to read, or to indicate reading from standard input.

Programmatic interface

pickletools.dis(pickle,out=None,memo=None,indentlevel=4,annotate=0)

Outputs a symbolic disassembly of the pickle to the file-like object `out`, defaulting to `sys.stdout`. `pickle` can be a string or a file-like object. `memo` can be a Python dictionary that will be used as the pickle's memo; it can be used to perform disassemblies across multiple pickles created by the same pickler. Successive levels, indicated by `MARK` opcodes in the stream, are indented by `indentlevel` spaces. If a nonzero value is given to `annotate`, each opcode in the output is annotated with a short description.

pickletools.genops(pickle)

Provides an iterator over all of the opcodes in a pickle, returning a sequence of `(opcode,arg,pos)` triples. `opcode` is an instance of an `OpcodeInfo` class; `arg` is the decoded value, as a Python object, of the opcode's argument; `pos` is the position at which this opcode is located. `pickle` can be a string or a file-like object.

pickletools.optimize(picklestring)

Returns a new equivalent pickle string after eliminating unused `PUT` opcodes. The optimized pickle is shorter, takes less transmission time, requires less storage space, and unpickles more efficiently.

MS Windows Specific Services

This chapter describes modules that are only available on MS Windows platforms.

- `msvcrt`— Useful routines from the MS VC++ runtime
File Operations
Console I/O
Other Functions
- `winreg`— Windows registry access
Functions
Constants
HKEY_* Constants
Access Rights
64-bit Specific
Value Types
Registry Handle Objects
- `winsound`— Sound-playing interface for Windows

msvcrt— Useful routines from the MS VC++ runtime

These functions provide access to some useful capabilities on Windows platforms. Some higher-level modules use these functions to build the Windows implementations of their services. For example, the `getpass` module uses this in the implementation of the `getpass()` function.

Further documentation on these functions can be found in the Platform API documentation.

The module implements both the normal and wide char variants of the console I/O api. The normal API deals only with ASCII characters and is of limited use for internationalized applications. The wide char API should be used where ever possible.

Availability: Windows.

Changed in version 3.3: Operations in this module now raise `OSError` where `IOError` was raised.

File Operations

msvcrt.locking(fd,mode,nbytes)

Lock part of a file based on file descriptor `fd` from the C runtime. Raises `OSError` on failure. The locked region of the file extends from the current file position for `nbytes` bytes, and may continue beyond the end of the file. `mode` must be one of the `LK_*` constants listed below. Multiple regions in a file may be locked at the same time, but may not overlap. Adjacent regions are not merged; they must be unlocked individually. Raises an auditing event `msvcrt.locking` with arguments `fd,mode,nbytes`.

msvcrt.LK_LOCK

msvcrt.LK_RLCK

Locks the specified bytes. If the bytes cannot be locked, the program immediately tries again after 1 second. If, after 10 attempts, the bytes cannot be locked, `OSError` is raised.

msvcrt.LK_NBLCK

msvcrt.LK_NBLCK

Locks the specified bytes. If the bytes cannot be locked, `OSError` is raised.

msvcrt.LK_UNLCK

Unlocks the specified bytes, which must have been previously locked.

msvcrt.setmode(fd,flags)

Set the line-end translation mode for the file descriptor `fd`. To set it to text mode, `flags` should be `os.O_TEXT`; for binary, it should be `os.O_BINARY`.

msvcrt.open_osfhandle(handle,flags)

Create a C runtime file descriptor from the file handle `handle`. The `flags` parameter should be a bitwise OR of `os.O_APPEND`, `os.O_RDONLY`, `os.O_TEXT` and `os.O_NOINHERIT`. The returned file descriptor may be used as a parameter to `os.fdopen()` to create a file object. The file descriptor is inheritable by default. Pass `os.O_NOINHERIT` flag to make it non inheritable. Raises an auditing event `msvcrt.open_osfhandle` with arguments `handle,flags`.

msvcrt.get_osfhandle(fd)

Return the file handle for the file descriptor `fd`. Raises `OSError` if `fd` is not recognized. Raises an auditing event `msvcrt.get_osfhandle` with argument `fd`.

Console I/O

msvcrt.kbhit()

Returns a nonzero value if a keypress is waiting to be read. Otherwise, return 0.

msvcrt.getch()

Read a keypress and return the resulting character as a byte string. Nothing is echoed to the console. This call will block if a keypress is not already available, but will not wait for Enter to be pressed. If the pressed key was a special function key, this will return '\000' or '\xe0'; the next call will return the keycode. The Control-C keypress cannot be read with this function.

msvcrt.getwch()

Wide char variant of getch(), returning a Unicode value.

msvcrt.getche()

Similar to getch(), but the keypress will be echoed if it represents a printable character.

msvcrt.getwche()

Wide char variant of getche(), returning a Unicode value.

msvcrt.putch(char)

Print the byte string `char` to the console without buffering.

msvcrt.putwch(unicode_char)

Wide char variant of putch(), accepting a Unicode value.

msvcrt.ungetch(char)

Cause the byte string `char` to be “pushed back” into the console buffer; it will be the next character read by getch() or getche().

msvcrt.ungetwch(unicode_char)

Wide char variant of ungetch(), accepting a Unicode value.

Other Functions

msvcrt.heapmin()

Force the `malloc()` heap to clean itself up and return unused blocks to the operating system. On failure, this raises `OSError`.

msvcrt.set_error_mode(mode)

Changes the location where the C runtime writes an error message for an error that might end the program. `mode` must be one of the `OUT_*` constants listed below or `REPORT_ERRMODE`. Returns the old setting or -1 if an error occurs. Only available in debug build of Python.

msvcrt.OUT_TO_DEFAULT

Error sink is determined by the app's type. Only available in debug build of Python.

msvcrt.OUT_TO_STDERR

Error sink is a standard error. Only available in debug build of Python.

msvcrt.OUT_TO_MSGBOX

Error sink is a message box. Only available in debug build of Python.

msvcrt.REPORT_ERRMODE

Report the current error mode value. Only available in debug build of Python.

msvcrt.CrtSetReportMode(type,mode)

Specifies the destination or destinations for a specific report type generated by `_CrtDbgReport()` in the MS VC++ runtime. `type` must be one of the `CRT_*` constants listed below. `mode` must be one of the `CRTDBG_*` constants listed below. Only available in debug build of Python.

msvcrt.CrtSetReportFile(type,file)

After you use `CrtSetReportMode()` to specify `CRTDBG_MODE_FILE`, you can specify the file handle to receive the message text. `type` must be one of the `CRT_*` constants listed below. `file` should be the file handle you want specified. Only available in debug build of Python.

msvcrt.CRT_WARN

Warnings, messages, and information that doesn't need immediate attention.

msvcrt.CRT_ERROR

Errors, unrecoverable problems, and issues that require immediate attention.

msvcrt.CRT_ASSERT

Assertion failures.

msvcrt.CRTDBG_MODE_DEBUG

Writes the message to the debugger's output window.

msvcrt.CRTDBG_MODE_FILE

Writes the message to a user-supplied file handle. `CrtSetReportFile()` should be called to define the specific file or stream to use as the destination.

msvcrt.CRTDBG_MODE_WNDW

Creates a message box to display the message along with the `Abort`, `Retry`, and `Ignore` buttons.

msvcrt.CRTDBG_REPORT_MODE

Returns current mode for the specified type.

msvcrt.CRT_ASSEMBLY_VERSION

The CRT Assembly version, from the `crtassem.h` header file.

msvcrt.VC_ASSEMBLY_PUBLICKEYTOKEN

The VC Assembly public key token, from the `crtassem.h` header file.

msvcrt.LIBRARIES_ASSEMBLY_NAME_PREFIX

The Libraries Assembly name prefix, from the `crtassem.h` header file.

winreg— Windows registry access

These functions expose the Windows registry API to Python. Instead of using an integer as the registry handle, a `handle` object is used to ensure that the handles are closed correctly, even if the programmer neglects to explicitly close them.

Availability: Windows.

Changed in version 3.3: Several functions in this module used to raise a `WindowsError`, which is now an alias of `OSError`.

Functions

This module offers the following functions:

winreg.CloseKey(hkey)

Closes a previously opened registry key. The `hkey` argument specifies a previously opened key. Note if `hkey` is not closed using this method (or via `hkey.Close()`), it is closed when the `hkey` object is destroyed by Python.

winreg.ConnectRegistry(computer_name, key)

Establishes a connection to a predefined registry handle on another computer, and returns a `handle` object. `computer_name` is the name of the remote computer, of the form `r"\\computername"`. If `None`, the local computer is used. `key` is the predefined handle to connect to. The return value is the handle of the opened key. If the function fails, an `OSError` exception is raised. Raises an auditing event `winreg.ConnectRegistry` with arguments `computer_name, key`. Changed in version 3.3: See above.

winreg.CreateKey(key, sub_key)

Creates or opens the specified key, returning a `handle` object. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that names the key this method opens or creates. If `key` is one of the predefined keys, `sub_key` may be `None`. In that case, the handle returned is the same key handle passed in to the function. If the key already exists, this function opens the existing key. The return value is the handle of the opened key. If the function fails, an `OSError` exception is raised. R

winreg.CreateKeyEx(key, sub_key, reserved=0, access=KEY_WRITE)

Creates or opens the specified key, returning a `handle` object. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that names the key this method opens or creates. `reserved` is a reserved integer, and must be zero. The default is zero. `access` is an integer that specifies an access mask that describes the desired security access for the key. Default is `KEY_WRITE`. See `Access Rights` for other allowed values. If `key` is one of the predefined keys, `sub_key` may be `None`. In that cas

winreg.DeleteKey(key, sub_key)

Deletes the specified key. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that must be a subkey of the key identified by the `key` parameter. This value must not be `None`, and the key may not have subkeys. This method can not delete keys with subkeys. If the method succeeds, the entire key, including all of its values, is removed. If the method fails, an `OSError` exception is raised. Raises an auditing event `winreg.DeleteKey` with arguments `key, sub_key, access`. Changed in ve

winreg.DeleteKeyEx(key, sub_key, access=KEY_WOW64_64KEY, reserved=0)

Deletes the specified key. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that must be a subkey of the key identified by the `key` parameter. This value must not be `None`, and the key may not have subkeys. `reserved` is a reserved integer, and must be zero. The default is zero. `access` is an integer

that specifies an access mask that describes the desired security access for the key. Default is `KEY_WOW64_64KEY`. On 32-bit Windows, the `WOW64` constants are ignored. See `AccessRights`.

winreg.DeleteValue(key,value)

Removes a named value from a registry key. `key` is an already open key, or one of the predefined `HKEY_*` constants. `value` is a string that identifies the value to remove. Raises an auditing event `winreg.DeleteValue` with arguments `key,value`.

winreg.EnumKey(key,index)

Enumerates subkeys of an open registry key, returning a string. `key` is an already open key, or one of the predefined `HKEY_*` constants. `index` is an integer that identifies the index of the key to retrieve. The function retrieves the name of one subkey each time it is called. It is typically called repeatedly until an `OSError` exception is raised, indicating, no more values are available. Raises an auditing event `winreg.EnumKey` with arguments `key,index`. Changed in version 3.3: See above.

winreg.EnumValue(key,index)

Enumerates values of an open registry key, returning a tuple. `key` is an already open key, or one of the predefined `HKEY_*` constants. `index` is an integer that identifies the index of the value to retrieve. The function retrieves the name of one subkey each time it is called. It is typically called repeatedly, until an `OSError` exception is raised, indicating no more values. The result is a tuple of 3 items: `IndexMeaning` A string that identifies the value name 1 An object that holds the value data, and whose `t`

winreg.ExpandEnvironmentStrings(str)

Expands environment variable placeholders `%NAME%` in strings like `REG_EXPAND_SZ`:>>> `ExpandEnvironmentStrings('%windir%') 'C:\\Windows'` Raises an auditing event `winreg.ExpandEnvironmentStrings` with argument `str`.

winreg.FlushKey(key)

Writes all the attributes of a key to the registry. `key` is an already open key, or one of the predefined `HKEY_*` constants. It is not necessary to call `FlushKey()` to change a key. Registry changes are flushed to disk by the registry using its lazy flusher. Registry changes are also flushed to disk at system shutdown. Unlike `CloseKey()`, the `FlushKey()` method returns only when all the data has been written to the registry. An application should only call `FlushKey()` if it requires absolute certainty that reg

winreg.LoadKey(key,sub_key,file_name)

Creates a subkey under the specified key and stores registration information from a specified file into that subkey. `key` is a handle returned by `ConnectRegistry()` or one of the constants `SHKEY_USERS` or `HKEY_LOCAL_MACHINE`. `sub_key` is a string that identifies the subkey to load. `file_name` is the name of the file to load registry data from. This file must have been created with the `SaveKey()` function. Under the file allocation table (FAT) file system, the filename may not have an extension. A call to `LoadKey()` fail

winreg.OpenKey(key,sub_key,reserved=0,access=KEY_READ)

winreg.OpenKeyEx(key,sub_key,reserved=0,access=KEY_READ)

Opens the specified key, returning a handle object. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that identifies the sub_key to open. `reserved` is a reserved integer, and must be zero. The default is zero. `access` is an integer that specifies an access mask that describes the desired security access for the key. Default is `KEY_READ`. See `Access Rights` for other allowed values. The result is a new handle to the specified key. If the function fails, `OSError` is raised. R

winreg.QueryInfoKey(key)

Returns information about a key, as a tuple. `key` is an already open key, or one of the predefined `HKEY_*` constants. The result is a tuple of 3 items: `IndexMeaning` 0 An integer giving the number of sub keys this key has. 1 An integer giving the number of values this key has. 2 An integer giving when the key was last modified (if available) as 100's of nanoseconds since Jan 1, 1601. Raises an `auditing event` `winreg.QueryInfoKey` with argument `key`.

winreg.QueryValue(key,sub_key)

Retrieves the unnamed value for a key, as a string. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that holds the name of the subkey with which the value is associated. If this parameter is `None` or empty, the function retrieves the value set by the `SetValue()` method for the key identified by `key`. Values in the registry have name, type, and data components. This method retrieves the data for a key's first value that has a `NULL` name. But the underlying API call does

winreg.QueryValueEx(key,value_name)

Retrieves the type and data for a specified value name associated with an open registry key. `key` is an already open key, or one of the predefined `HKEY_*` constants. `value_name` is a string indicating the value to query. The result is a tuple of 2 items: `IndexMeaning` 0 The value of the registry item. 1 An integer giving the registry type for this value (see table in docs for `SetValueEx()`). Raises an `auditing event` `winreg.QueryValue` with arguments `key`, `sub_key`, `value_name`.

winreg.SaveKey(key,file_name)

Saves the specified key, and all its subkeys to the specified file. `key` is an already open key, or one of the predefined `HKEY_*` constants. `file_name` is the name of the file to save registry data to. This file cannot already exist. If this filename includes an extension, it cannot be used on file allocation table (FAT) file systems by the `LoadKey()` method. If `key` represents a key on a remote computer, the path described by `file_name` is relative to the remote computer. The caller of this method must possess

winreg.SetValue(key,sub_key,type,value)

Associates a value with a specified key. `key` is an already open key, or one of the predefined `HKEY_*` constants. `sub_key` is a string that names the subkey with which the value is associated. `type` is an integer that specifies the type of the data. Currently this must be `REG_SZ`, meaning only strings are supported. Use the `SetValueEx()` function for support for other data types. `value` is a string that specifies the new value. If the key specified by the `sub_key` parameter does not exist, the `SetValue` function creat

winreg.SetValueEx(key,value_name,reserved,type,value)

Stores data in the value field of an open registry key. `key` is an already open key, or one of the predefined `HKEY_*` constants. `value_name` is a string that names the subkey with which the value is associated. `reserved` can be anything – zero is always passed to the API. `type` is an integer that specifies the type of the data. See `Value Types` for the available types. `value` is a string that specifies the new value. This method can also set additional value and type information for the specified key. The key ident

winreg.DisableReflectionKey(key)

Disables registry reflection for 32-bit processes running on a 64-bit operating system. `key` is an already open key, or one of the predefined `HKEY_*` constants. Will generally raise `NotImplementedError` if executed on a 32-bit operating system. If the key is not on the reflection list, the function succeeds but has no effect. Disabling reflection for a key does not affect reflection of any subkeys. Raises an `auditing event` `winreg.DisableReflectionKey` with argument `key`.

winreg.EnableReflectionKey(key)

Restores registry reflection for the specified disabled key. `key` is an already open key, or one of the predefined `HKEY_*` constants. Will generally raise `NotImplementedError` if executed on a 32-bit operating system. Restoring reflection for a key does not affect reflection of any subkeys. Raises an `auditing`

eventwinreg.EnableReflectionKeywith argumentkey.

winreg.QueryReflectionKey(key)

Determines the reflection state for the specified key.keyis an already open key, or one of the predefinedHKEY_* constants.ReturnsTrueif reflection is disabled.Will generally raiseNotImplementedErrorif executed on a 32-bit operating system.Raises anauditing eventwinreg.QueryReflectionKeywith argumentkey.

Constants

The following constants are defined for use in manywinregfunctions.

HKEY_* Constants

winreg.HKEY_CLASSES_ROOT

Registry entries subordinate to this key define types (or classes) of documents and the properties associated with those types. Shell and COM applications use the information stored under this key.

winreg.HKEY_CURRENT_USER

Registry entries subordinate to this key define the preferences of the current user. These preferences include the settings of environment variables, data about program groups, colors, printers, network connections, and application preferences.

winreg.HKEY_LOCAL_MACHINE

Registry entries subordinate to this key define the physical state of the computer, including data about the bus type, system memory, and installed hardware and software.

winreg.HKEY_USERS

Registry entries subordinate to this key define the default user configuration for new users on the local computer and the user configuration for the current user.

winreg.HKEY_PERFORMANCE_DATA

Registry entries subordinate to this key allow you to access performance data. The data is not actually stored in the registry; the registry functions cause the system to collect the data from its source.

winreg.HKEY_CURRENT_CONFIG

Contains information about the current hardware profile of the local computer system.

winreg.HKEY_DYN_DATA

This key is not used in versions of Windows after 98.

Access Rights

For more information, seeRegistry Key Security and Access.

winreg.KEY_ALL_ACCESS

Combines the STANDARD_RIGHTS_REQUIRED,KEY_QUERY_VALUE,KEY_SET_VALUE,KEY_CREATE_SUB_KEY,KEY_ENUMERATE_SUB_KEYS,KEY_NOTIFY, andKEY_CREATE_LINKaccess rights.

winreg.KEY_WRITE

Combines the STANDARD_RIGHTS_WRITE,KEY_SET_VALUE, andKEY_CREATE_SUB_KEYaccess rights.

winreg.KEY_READ

Combines the STANDARD_RIGHTS_READ, KEY_QUERY_VALUE, KEY_ENUMERATE_SUB_KEYS, and KEY_NOTIFY values.

winreg.KEY_EXECUTE

Equivalent to KEY_READ.

winreg.KEY_QUERY_VALUE

Required to query the values of a registry key.

winreg.KEY_SET_VALUE

Required to create, delete, or set a registry value.

winreg.KEY_CREATE_SUB_KEY

Required to create a subkey of a registry key.

winreg.KEY_ENUMERATE_SUB_KEYS

Required to enumerate the subkeys of a registry key.

winreg.KEY_NOTIFY

Required to request change notifications for a registry key or for subkeys of a registry key.

winreg.KEY_CREATE_LINK

Reserved for system use.

64-bit Specific

For more information, see [Accessing an Alternate Registry View](#).

winreg.KEY_WOW64_64KEY

Indicates that an application on 64-bit Windows should operate on the 64-bit registry view. On 32-bit Windows, this constant is ignored.

winreg.KEY_WOW64_32KEY

Indicates that an application on 64-bit Windows should operate on the 32-bit registry view. On 32-bit Windows, this constant is ignored.

Value Types

For more information, see [Registry Value Types](#).

winreg.REG_BINARY

Binary data in any form.

winreg.REG_DWORD

32-bit number.

winreg.REG_DWORD_LITTLE_ENDIAN

A 32-bit number in little-endian format. Equivalent to REG_DWORD.

winreg.REG_DWORD_BIG_ENDIAN

A 32-bit number in big-endian format.

winreg.REG_EXPAND_SZ

Null-terminated string containing references to environment variables (%PATH%).

winreg.REG_LINK

A Unicode symbolic link.

winreg.REG_MULTI_SZ

A sequence of null-terminated strings, terminated by two null characters. (Python handles this termination automatically.)

winreg.REG_NONE

No defined value type.

winreg.REG_QWORD

A 64-bit number. Added in version 3.6.

winreg.REG_QWORD_LITTLE_ENDIAN

A 64-bit number in little-endian format. Equivalent to REG_QWORD. Added in version 3.6.

winreg.REG_RESOURCE_LIST

A device-driver resource list.

winreg.REG_FULL_RESOURCE_DESCRIPTOR

A hardware setting.

winreg.REG_RESOURCE_REQUIREMENTS_LIST

A hardware resource list.

winreg.REG_SZ

A null-terminated string.

Registry Handle Objects

This object wraps a Windows HKEY object, automatically closing it when the object is destroyed. To guarantee cleanup, you can call either the `Close()` method on the object, or the `CloseKey()` function.

All registry functions in this module return one of these objects.

All registry functions in this module which accept a handle object also accept an integer, however, use of the handle object is encouraged.

Handle objects provide semantics for `__bool__()`— thus

```
if handle:
    print("Yes")
```

will print `Yes` if the handle is currently valid (has not been closed or detached).

Handle objects can be converted to an integer (e.g., using the built-in `int()` function), in which case the underlying Windows handle value is returned. You can also use the `Detach()` method to return the integer handle, and also disconnect the Windows handle from the handle object.

PyHKEY.Close()

Closes the underlying Windows handle. If the handle is already closed, no error is raised.

PyHKEY.Detach()

Detaches the Windows handle from the handle object. The result is an integer that holds the value of the handle before it is detached. If the handle is already detached or closed, this will return zero. After calling this

function, the handle is effectively invalidated, but the handle is not closed. You would call this function when you need the underlying Win32 handle to exist beyond the lifetime of the handle object. Raises an `auditing` event `winreg.PyHKEY.Detach` with argument `key`.

`PyHKEY.__enter__()`

`PyHKEY.__exit__(*exc_info)`

The `HKEY` object implements `__enter__()` and `__exit__()` and thus supports the context protocol for the `with` statement: `with OpenKey(HKEY_LOCAL_MACHINE, "foo") as key: ... # work with key` will automatically close `key` when control leaves the `with` block.

winsound— Sound-playing interface for Windows

The `winsound` module provides access to the basic sound-playing machinery provided by Windows platforms. It includes functions and several constants.

Availability: Windows.

winsound.Beep(frequency,duration)

Beep the PC's speaker. The `frequency` parameter specifies frequency, in hertz, of the sound, and must be in the range 37 through 32,767. The `duration` parameter specifies the number of milliseconds the sound should last. If the system is not able to beep the speaker, `RuntimeError` is raised.

winsound.PlaySound(sound,flags)

Call the underlying `PlaySound()` function from the Platform API. The `sound` parameter may be a filename, a system sound alias, audio data as a bytes-like object, or `None`. Its interpretation depends on the value of `flags`, which can be a bitwise ORed combination of the constants described below. If the `sound` parameter is `None`, any currently playing waveform sound is stopped. If the system indicates an error, `RuntimeError` is raised.

winsound.MessageBeep(type=MB_OK)

Call the underlying `MessageBeep()` function from the Platform API. This plays a sound as specified in the registry. The `type` argument specifies which sound to play; possible values are `-1`, `MB_ICONASTERISK`, `MB_ICONEXCLAMATION`, `MB_ICONHAND`, `MB_ICONQUESTION`, and `MB_OK`, all described below. The value `-1` produces a "simple beep"; this is the final fallback if a sound cannot be played otherwise. If the system indicates an error, `RuntimeError` is raised.

winsound.SND_FILENAME

The `sound` parameter is the name of a WAV file. Do not use with `SND_ALIAS`.

winsound.SND_ALIAS

The `sound` parameter is a sound association name from the registry. If the registry contains no such name, play the system default sound unless `SND_NODEFAULT` is also specified. If no default sound is registered, raise `RuntimeError`. Do not use with `SND_FILENAME`. All Win32 systems support at least the following; most systems support many more: `PlaySound()` name Corresponding Control Panel Sound
`name` 'SystemAsterisk' Asterisk 'SystemExclamation' Exclamation 'SystemExit' Exit Windows 'SystemHand' Critical Stop 'SystemQ'

winsound.SND_LOOP

Play the sound repeatedly. The `SND_ASYNC` flag must also be used to avoid blocking. Cannot be used with `SND_MEMORY`.

winsound.SND_MEMORY

The `sound` parameter to `PlaySound()` is a memory image of a WAV file, as a bytes-like object. Note This module does not support playing from a memory image asynchronously, so a combination of this flag and `SND_ASYNC` will raise `RuntimeError`.

winsound.SND_PURGE

Stop playing all instances of the specified sound. Note This flag is not supported on modern Windows platforms.

winsound.SND_ASYNC

Return immediately, allowing sounds to play asynchronously.

winsound.SND_NODEFAULT

If the specified sound cannot be found, do not play the system default sound.

winsound.SND_NOSTOP

Do not interrupt sounds currently playing.

winsound.SND_NOWAIT

Return immediately if the sound driver is busy. **Note** This flag is not supported on modern Windows platforms.

winsound.SND_APPLICATION

The `sound` parameter is an application-specific alias in the registry. This flag can be combined with the `SND_ALIAS` flag to specify an application-defined sound alias.

winsound.SND_SENTRY

Triggers a `SoundSentry` event when the sound is played. Added in version 3.14.

winsound.SND_SYNC

The sound is played synchronously. This is the default behavior. Added in version 3.14.

winsound.SND_SYSTEM

Assign the sound to the audio session for system notification sounds. Added in version 3.14.

winsound.MB_ICONASTERISK

Play the `SystemDefaultSound`.

winsound.MB_ICONEXCLAMATION

Play the `SystemExclamationSound`.

winsound.MB_ICONHAND

Play the `SystemHandSound`.

winsound.MB_ICONQUESTION

Play the `SystemQuestionSound`.

winsound.MB_OK

Play the `SystemDefaultSound`.

winsound.MB_ICONERROR

Play the `SystemHandSound`. Added in version 3.14.

winsound.MB_ICONINFORMATION

Play the `SystemDefaultSound`. Added in version 3.14.

winsound.MB_ICONSTOP

Play the `SystemHandSound`. Added in version 3.14.

winsound.MB_ICONWARNING

Play the `SystemExclamationSound`. Added in version 3.14.

Unix-specific services

The modules described in this chapter provide interfaces to features that are unique to the Unix operating system, or in some cases to some or many variants of it. Here's an overview:

- `shlex`— Simple lexical analysis
`shlex` Objects
Parsing Rules
Improved Compatibility with Shells
- `posix`— The most common POSIX system calls
Large File Support
Notable Module Contents
- `pwd`— The password database
- `grp`— The group database
- `termios`— POSIX style tty control
Example
- `tty`— Terminal control functions
- `pty`— Pseudo-terminal utilities
Example
- `fcntl`— The `fcntl` and `ioctl` system calls
- `resource`— Resource usage information
Resource Limits
Resource Usage
- `syslog`— Unix syslog library routines
Examples
Simple example

shlex— Simple lexical analysis

Source code: `Lib/shlex.py`

The `shlex` class makes it easy to write lexical analyzers for simple syntaxes resembling that of the Unix shell. This will often be useful for writing minilanguages, (for example, in run control files for Python applications) or for parsing quoted strings.

The `shlex` module defines the following functions:

shlex.split(s, comments=False, posix=True)

Split the string using shell-like syntax. If `comments` is `False` (the default), the parsing of comments in the given string will be disabled (setting the `commenters` attribute of the `shlex` instance to the empty string). This function operates in POSIX mode by default, but uses non-POSIX mode if the `posix` argument is false. Changed in version 3.12: Passing `None` for `s` argument now raises an exception, rather than reading `sys.stdin`.

shlex.join(split_command)

Concatenate the tokens of the list `split_command` and return a string. This function is the inverse of `split()`.>>>from shlex import join>>>print(join(['echo', '-n', 'Multiple words']))echo -n 'Multiple words' The returned value is shell-escaped to protect against injection vulnerabilities (see `quote()`). Added in version 3.8.

shlex.quote(s)

Return a shell-escaped version of the string. The returned value is a string that can safely be used as one token in a shell command line, for cases where you cannot use a list. Warning: The `shlex` module is only designed for Unix shells. The `quote()` function is not guaranteed to be correct on non-POSIX compliant shells or shells from other operating systems such as Windows. Executing commands quoted by this module on such shells can open up the possibility of a command injection vulnerability. Consider

The `shlex` module defines the following class:

class shlex.shlex(instream=None, infile=None, posix=False, punctuation_chars=False)

A `shlex` instance or subclass instance is a lexical analyzer object. The initialization argument, if present, specifies where to read characters from. It must be a file-/stream-like object with `read()` and `readline()` methods, or a string. If no argument is given, input will be taken from `sys.stdin`. The second optional argument is a filename string, which sets the initial value of the `infile` attribute. If the `instream` argument is omitted or equal to `sys.stdin`, this second argument defaults to `"stdin"`. The

See also

Module configparser

Parser for configuration files similar to the Windows `.ini` files.

shlex Objects

A `shlex` instance has the following methods:

shlex.get_token()

Return a token. If tokens have been stacked using `push_token()`, pop a token off the stack. Otherwise, read one from the input stream. If reading encounters an immediate end-of-file, `eof` is returned (the empty string `""`) in non-POSIX mode, and `None` in POSIX mode).

shlex.push_token(str)

Push the argument onto the token stack.

shlex.read_token()

Read a raw token. Ignore the pushback stack, and do not interpret source requests. (This is not ordinarily a useful entry point, and is documented here only for the sake of completeness.)

shlex.sourcehook(filename)

When `shlex` detects a source request (see `source` below) this method is given the following token as argument, and expected to return a tuple consisting of a filename and an open file-like object. Normally, this method first strips any quotes off the argument. If the result is an absolute pathname, or there was no previous source request in effect, or the previous source was a stream (such as `sys.stdin`), the result is left alone. Otherwise, if the result is a relative pathname, the directory part of `t`

shlex.push_source(newstream,newfile=None)

Push an input source stream onto the input stack. If the filename argument is specified it will later be available for use in error messages. This is the same method used internally by the `sourcehook()` method.

shlex.pop_source()

Pop the last-pushed input source from the input stack. This is the same method used internally when the lexer reaches EOF on a stacked input stream.

shlex.error_leader(infile=None,lineno=None)

This method generates an error message leader in the format of a Unix C compiler error label; the format is `"%s",line%d:",` where the `%s` is replaced with the name of the current source file and the `%d` with the current input line number (the optional arguments can be used to override these). This convenience is provided to encourage `shlex` users to generate error messages in the standard, parseable format understood by Emacs and other Unix tools.

Instances of `shlex` subclasses have some public instance variables which either control lexical analysis or can be used for debugging:

shlex.commenters

The string of characters that are recognized as comment beginners. All characters from the comment beginner to end of line are ignored. Includes just `#` by default.

shlex.wordchars

The string of characters that will accumulate into multi-character tokens. By default, includes all ASCII alphanumerics and underscore. In POSIX mode, the accented characters in the Latin-1 set are also included. If `punctuation_chars` is not empty, the characters `~./*?=:`, which can appear in filename specifications and command line parameters, will also be included in this attribute, and any characters which appear in `punctuation_chars` will be removed from `wordchars` if they are present there. If `white`

shlex.whitespace

Characters that will be considered whitespace and skipped. Whitespace bounds tokens. By default, includes space, tab, linefeed and carriage-return.

shlex.escape

Characters that will be considered as escape. This will be only used in POSIX mode, and includes just `\` by default.

shlex.quotes

Characters that will be considered string quotes. The token accumulates until the same quote is encountered again (thus, different quote types protect each other as in the shell.) By default, includes ASCII single and

double quotes.

shlex.escapedquotes

Characters in quotes that will interpret escape characters defined in `escape`. This is only used in POSIX mode, and includes just `'\"'` by default.

shlex.whitespace_split

If `True`, tokens will only be split in whitespaces. This is useful, for example, for parsing command lines with `shlex`, getting tokens in a similar way to shell arguments. When used in combination with `punctuation_chars`, tokens will be split on whitespace in addition to those characters. Changed in version 3.8: The `punctuation_chars` attribute was made compatible with the `whitespace_split` attribute.

shlex.infile

The name of the current input file, as initially set at class instantiation time or stacked by later source requests. It may be useful to examine this when constructing error messages.

shlex.instream

The input stream from which this `shlex` instance is reading characters.

shlex.source

This attribute is `None` by default. If you assign a string to it, that string will be recognized as a lexical-level inclusion request similar to the `source` keyword in various shells. That is, the immediately following token will be opened as a filename and input will be taken from that stream until EOF, at which point the `close()` method of that stream will be called and the input source will again become the original input stream. Source requests may be stacked any number of levels deep.

shlex.debug

If this attribute is numeric and 1 or more, a `shlex` instance will print verbose progress output on its behavior. If you need to use this, you can read the module source code to learn the details.

shlex.lineno

Source line number (count of newlines seen so far plus one).

shlex.token

The token buffer. It may be useful to examine this when catching exceptions.

shlex.eof

Token used to determine end of file. This will be set to the empty string (`''`), in non-POSIX mode, and to `None` in POSIX mode.

shlex.punctuation_chars

A read-only property. Characters that will be considered punctuation. Runs of punctuation characters will be returned as a single token. However, note that no semantic validity checking will be performed: for example, `'>>>'` could be returned as a token, even though it may not be recognised as such by shells. Added in version 3.6.

Parsing Rules

When operating in non-POSIX mode, `shlex` will try to obey the following rules.

- Quote characters are not recognized within words (Do "Not" Separate is parsed as the single word Do "Not" Separate);

- Escape characters are not recognized;
- Enclosing characters in quotes preserve the literal value of all characters within the quotes;
- Closing quotes separate words ("Do"Separate is parsed as "Do"andSeparate);
- If `whitespace_split` is `False`, any character not declared to be a word character, whitespace, or a quote will be returned as a single-character token. If it is `True`, `shlex` will only split words in whitespaces;
- EOF is signaled with an empty string ("");
- It's not possible to parse empty strings, even if quoted.

When operating in POSIX mode, `shlex` will try to obey the following parsing rules.

- Quotes are stripped out, and do not separate words ("Do" "Not" "Separate" is parsed as the single word `DoNotSeparate`);
- Non-quoted escape characters (e.g. `\`) preserve the literal value of the next character that follows;
- Enclosing characters in quotes which are not part of escaped quotes (e.g. `""`) preserve the literal value of all characters within the quotes;
- Enclosing characters in quotes which are part of escaped quotes (e.g. `""`) preserves the literal value of all characters within the quotes, with the exception of the characters mentioned in `escape`. The escape characters retain their special meaning only when followed by the quote in use, or the escape character itself. Otherwise the escape character will be considered a normal character.
- EOF is signaled with a `None` value;
- Quoted empty strings (") are allowed.

Improved Compatibility with Shells

Added in version 3.6.

The `shlex` class provides compatibility with the parsing performed by common Unix shells like `bash`, `dash`, and `sh`. To take advantage of this compatibility, specify the `punctuation_chars` argument in the constructor. This defaults to `False`, which preserves pre-3.6 behaviour. However, if it is set to `True`, then parsing of the characters `();<>|&` is changed: any run of these characters is returned as a single token. While this is short of a full parser for shells (which would be out of scope for the standard library, given the multiplicity of shells out there), it does allow you to perform processing of command lines more easily than you could otherwise. To illustrate, you can see the difference in the following snippet:

```
>>> import shlex
>>> text = "a && b; c && d || e; f >'abc'; (def \"ghi\")"
>>> s = shlex.shlex(text, posix=True)
>>> s.whitespace_split = True
>>> list(s)
['a', '&&', 'b;', 'c', '&&', 'd', '||', 'e;', 'f', '>abc;', '(def', 'ghi)']
>>> s = shlex.shlex(text, posix=True, punctuation_chars=True)
>>> s.whitespace_split = True
>>> list(s)
['a', '&&', 'b', ';', 'c', '&&', 'd', '||', 'e', ';', 'f', '>', 'abc', ';',
('(', 'def', 'ghi', ')']
```

Of course, tokens will be returned which are not valid for shells, and you'll need to implement your own error checks on the returned tokens.

Instead of passing `True` as the value for the `punctuation_chars` parameter, you can pass a string with specific characters, which will be used to determine which characters constitute punctuation. For example:

```
>>> import shlex
>>> s = shlex.shlex("a && b || c", punctuation_chars="|")
>>> list(s)
['a', '&', '&', 'b', '||', 'c']
```

■ Nota: NoteWhen `punctuation_chars` is specified, the `wordchars` attribute is augmented with the characters `~-./*?`. That is because these characters can appear in file names (including wildcards) and command-line arguments (e.g. `--color=auto`). Hence: `>>>import shlex>>>s=shlex.shlex('~ /a && b-c --color=auto || d *.py?',...punctuation_chars=True)>>>list(s) ['~/a', '&&', 'b-c', '--color=auto', '||', 'd', '*.py?']` However, to match the shell as closely as possible, it is recommended to always use `posix and whitespace_split` when using `punctuation_chars`, which will negate `wordchars` entirely.

For best effect, `punctuation_charss` should be set in conjunction with `posix=True`. (Note that `posix=False` is the default for `shlex`.)

posix— The most common POSIX system calls

This module provides access to operating system functionality that is standardized by the C Standard and the POSIX standard (a thinly disguised Unix interface).

Availability: Unix.

Do not import this module directly. Instead, import the module `os`, which provides a portable version of this interface. On Unix, the `os` module provides a superset of the `posix` interface. On non-Unix operating systems the `posix` module is not available, but a subset is always available through the `os` interface. Once `os` is imported, there is no performance penalty in using it instead of `posix`. In addition, `os` provides some additional functionality, such as automatically calling `putenv()` when an entry in `os.environ` is changed.

Errors are reported as exceptions; the usual exceptions are given for type errors, while errors reported by the system calls raise `OSError`.

Large File Support

Several operating systems (including AIX and Solaris) provide support for files that are larger than 2 GiB from a C programming model where `int` and `long` are 32-bit values. This is typically accomplished by defining the relevant size and offset types as 64-bit values. Such files are sometimes referred to as large files.

Large file support is enabled in Python when the size of an `off_t` is larger than a `long` and the `long` is at least as large as an `off_t`. It may be necessary to configure and compile Python with certain compiler flags to enable this mode. For example, with Solaris 2.6 and 2.7 you need to do something like:

```
CFLAGS="-getconf LFS_CFLAGS" OPT="-g -O2 $CFLAGS" \
./configure
```

On large-file-capable Linux systems, this might work:

```
CFLAGS='-D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64' OPT="-g -O2 $CFLAGS" \
./configure
```

Notable Module Contents

In addition to many functions described in the `os` module documentation, `posix` defines the following data item:

posix.environ

A dictionary representing the string environment at the time the interpreter was started. Keys and values are bytes on Unix and str on Windows. For example, `environ[b'HOME']` (`environ['HOME']` on Windows) is the pathname of your home directory, equivalent to `getenv("HOME")` in C. Modifying this dictionary does not affect the string environment passed on by `execv()`, `popen()` or `system()`; if you need to change the environment, pass `environ` to `execve()` or add variable assignments and export statements to the command

pwd— The password database

This module provides access to the Unix user account and password database. It is available on all Unix versions.

Availability: Unix, not WASI, not iOS.

Password database entries are reported as a tuple-like object, whose attributes correspond to the members of the `passwd` structure (Attribute field below, see `<pwd.h>`):

Index	Attribute	Meaning
0	<code>pw_name</code>	Login name
1	<code>pw_passwd</code>	Optional encrypted password
2	<code>pw_uid</code>	Numerical user ID
3	<code>pw_gid</code>	Numerical group ID
4	<code>pw_gecos</code>	User name or comment field
5	<code>pw_dir</code>	User home directory
6	<code>pw_shell</code>	User command interpreter

The uid and gid items are integers, all others are strings. `KeyError` is raised if the entry asked for cannot be found.

■ **Nota:** Note! In traditional Unix the field `pw_passwd` usually contains a password encrypted with a DES derived algorithm. However most modern unices use a so-called shadow password system. On those unices the `pw_passwd` field only contains an asterisk (*) or the letter 'x' where the encrypted password is stored in a file `/etc/shadow` which is not world readable. Whether the `pw_passwd` field contains anything useful is system-dependent.

It defines the following items:

pwd.getpwuid(uid)

Return the password database entry for the given numeric user ID.

pwd.getpwnam(name)

Return the password database entry for the given user name.

pwd.getpwall()

Return a list of all available password database entries, in arbitrary order.

See also

Module grp

An interface to the group database, similar to this.

grp— The group database

This module provides access to the Unix group database. It is available on all Unix versions.

Availability: Unix, not WASI, not Android, not iOS.

Group database entries are reported as a tuple-like object, whose attributes correspond to the members of the `groupstructure` (Attribute field below, see `<grp.h>`):

Index	Attribute	Meaning	0	gr_name	the name of the group	1	gr_passwd	the (encrypted) group password; often empty	2	gr_gid	the numerical group ID	3	gr_mem	all the group member's user names
-------	-----------	---------	---	---------	-----------------------	---	-----------	---	---	--------	------------------------	---	--------	-----------------------------------

The `gid` is an integer, name and password are strings, and the member list is a list of strings. (Note that most users are not explicitly listed as members of the group they are in according to the password database. Check both databases to get complete membership information. Also note that `agr_name` that starts with `a+` or `-` is likely to be a YP/NIS reference and may not be accessible via `getgrnam()` or `getgrgid()`.)

It defines the following items:

grp.getgrgid(id)

Return the group database entry for the given numeric group ID. `KeyError` is raised if the entry asked for cannot be found. Changed in version 3.10: `TypeError` is raised for non-integer arguments like floats or strings.

grp.getgrnam(name)

Return the group database entry for the given group name. `KeyError` is raised if the entry asked for cannot be found.

grp.getgrall()

Return a list of all available group entries, in arbitrary order.

See also

Module pwd

An interface to the user database, similar to this.

termios— POSIX style tty control

This module provides an interface to the POSIX calls for tty I/O control. For a complete description of these calls, see `termios(3)` Unix manual page. It is only available for those Unix versions that support POSIX `termios` style tty I/O control configured during installation.

Availability: Unix.

All functions in this module take a file descriptor `fd` as their first argument. This can be an integer file descriptor, such as returned by `sys.stdin.fileno()`, or a file object, such as `sys.stdin` itself.

This module also defines all the constants needed to work with the functions provided here; these have the same name as their counterparts in C. Please refer to your system documentation for more information on using these terminal control interfaces.

The module defines the following functions:

termios.tcgetattr(fd)

Return a list containing the tty attributes for file descriptor `fd`, as follows: `[iflag, oflag, cflag, lflag, ispeed, ospeed, cc]` where `cc` is a list of the tty special characters (each a string of length 1, except the items with indices `VMIN` and `VTIME`, which are integers when these fields are defined). The interpretation of the flags and the speeds as well as the indexing in the `cc` array must be done using the symbolic constants defined in the `termios` module.

termios.tcsetattr(fd, when, attributes)

termios.TCSANOW

termios.TCSADRAIN

termios.TCSAFLUSH

Set the tty attributes for file descriptor `fd` from the `attributes`, which is a list like the one returned by `tcgetattr()`. The `when` argument determines when the attributes are changed: `termios.TCSANOW` Change attributes immediately. `termios.TCSADRAIN` Change attributes after transmitting all queued output. `termios.TCSAFLUSH` Change attributes after transmitting all queued output and discarding all queued input.

Change attributes immediately.

Change attributes after transmitting all queued output.

Change attributes after transmitting all queued output and discarding all queued input.

termios.tcsendbreak(fd, duration)

Send a break on file descriptor `fd`. A zero `duration` sends a break for 0.25–0.5 seconds; a non-zero `duration` has a system dependent meaning.

termios.tcdrain(fd)

Wait until all output written to file descriptor `fd` has been transmitted.

termios.tcflush(fd, queue)

Discard queued data on file descriptor `fd`. The `queue` selector specifies which queue: `TCIFLUSH` for the input queue, `TCOFLUSH` for the output queue, or `TCIOFLUSH` for both queues.

termios.tcflow(fd, action)

Suspend or resume input or output on file descriptor `fd`. The `action` argument can be `TCOOFF` to suspend output, `TCOON` to restart output, `TCIOFF` to suspend input, or `TCION` to restart input.

`termios.tcgetwinsize(fd)`

Return a tuple `(ws_row, ws_col)` containing the tty window size for file descriptor `fd`. Requires `termios.TIOCGWINSZ` or `termios.TIOCGSIZE`. Added in version 3.11.

`termios.tcsetwinsize(fd, winsize)`

Set the tty window size for file descriptor `fd` from `winsize`, which is a two-item tuple `(ws_row, ws_col)` like the one returned by `tcgetwinsize()`. Requires at least one of the pairs `(termios.TIOCGWINSZ, termios.TIOCSWINSZ)`; `(termios.TIOCGSIZE, termios.TIOCSSIZE)` to be defined. Added in version 3.11.

See also

Module `tty`

Convenience functions for common terminal control operations.

Example

Here's a function that prompts for a password with echoing turned off. Note the technique using a separate `tcgetattr()` call and `atry...finally` statement to ensure that the old tty attributes are restored exactly no matter what happens:

```
def getpass(prompt="Password: "):
    import termios, sys
    fd = sys.stdin.fileno()
    old = termios.tcgetattr(fd)
    new = termios.tcgetattr(fd)
    new[3] = new[3] & ~termios.ECHO          # lflags
    try:
        termios.tcsetattr(fd, termios.TCSADRAIN, new)
        passwd = input(prompt)
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old)
    return passwd
```

tty— Terminal control functions

Source code: `Lib/tty.py`

The `tty` module defines functions for putting the `tty` into `cbreak` and `raw` modes.

Availability: Unix.

Because it requires the `termios` module, it will work only on Unix.

The `tty` module defines the following functions:

`tty.cfmakeraw(mode)`

Convert the `tty` attribute `listmode`, which is a list like the one returned by `termios.tcgetattr()`, to that of a `tty` in `raw` mode. Added in version 3.12.

`tty.cfmakecbreak(mode)`

Convert the `tty` attribute `listmode`, which is a list like the one returned by `termios.tcgetattr()`, to that of a `tty` in `cbreak` mode. This clears the `ECHO` and `CANON` local mode flags in `mode` as well as setting the minimum input to 1 byte with no delay. Added in version 3.12. Changed in version 3.12.2: The `ICRNL` flag is no longer cleared. This matches Linux and macOS `stty cbreak` behavior and what `setcbreak()` historically did.

`tty.setraw(fd, when=termios.TCSAFLUSH)`

Change the mode of the file descriptor `fd` to `raw`. If `when` is omitted, it defaults to `termios.TCSAFLUSH`, and is passed to `termios.tcsetattr()`. The return value of `termios.tcgetattr()` is saved before setting `fd` to `raw` mode; this value is returned. Changed in version 3.12: The return value is now the original `tty` attributes, instead of `None`.

`tty.setcbreak(fd, when=termios.TCSAFLUSH)`

Change the mode of file descriptor `fd` to `cbreak`. If `when` is omitted, it defaults to `termios.TCSAFLUSH`, and is passed to `termios.tcsetattr()`. The return value of `termios.tcgetattr()` is saved before setting `fd` to `cbreak` mode; this value is returned. This clears the `ECHO` and `CANON` local mode flags as well as setting the minimum input to 1 byte with no delay. Changed in version 3.12: The return value is now the original `tty` attributes, instead of `None`. Changed in version 3.12.2: The `ICRNL` flag is no longer cleared. This

See also

Module `termios`

Low-level terminal control interface.

pty— Pseudo-terminal utilities

Source code:Lib/pty.py

Theptymodule defines operations for handling the pseudo-terminal concept: starting another process and being able to write to and read from its controlling terminal programmatically.

Availability: Unix.

Pseudo-terminal handling is highly platform dependent. This code is mainly tested on Linux, FreeBSD, and macOS (it is supposed to work on other POSIX platforms but it's not been thoroughly tested).

Theptymodule defines the following functions:

pty.fork()

Fork. Connect the child's controlling terminal to a pseudo-terminal. Return value is(pid,fd). Note that the child getspid0, and thefdisinvalid. The parent's return value is thepidof the child, andfdis a file descriptor connected to the child's controlling terminal (and also to the child's standard input and output).WarningOn macOS the use of this function is unsafe when mixed with using higher-level system APIs, and that includes usingurllib.request.

pty.openpty()

Open a new pseudo-terminal pair, usingos.openpty()if possible, or emulation code for generic Unix systems. Return a pair of file descriptors(master,slave), for the master and the slave end, respectively.

pty.spawn(argv[,master_read[,stdin_read]])

Spawn a process, and connect its controlling terminal with the current process's standard io. This is often used to baffle programs which insist on reading from the controlling terminal. It is expected that the process spawned behind the pty will eventually terminate, and when it doesspawnwill return.A loop copies STDIN of the current process to the child and data received from the child to STDOUT of the current process. It is not signaled to the child if STDIN of the current process closes down

Example

The following program acts like the Unix commandscript(1), using a pseudo-terminal to record all input and output of a terminal session in a "typescript".

```
import argparse
import os
import pty
import sys
import time

parser = argparse.ArgumentParser()
parser.add_argument('-a', dest='append', action='store_true')
parser.add_argument('-p', dest='use_python', action='store_true')
parser.add_argument('filename', nargs='?', default='typescript')
options = parser.parse_args()

shell = sys.executable if options.use_python else os.environ.get('SHELL', 'sh')
filename = options.filename
mode = 'ab' if options.append else 'wb'

with open(filename, mode) as script:
    def read(fd):
        data = os.read(fd, 1024)
        script.write(data)
        return data
```

```
print('Script started, file is', filename)
script.write(('Script started on %s\n' % time.asctime()).encode())

pty.spawn(shell, read)

script.write(('Script done on %s\n' % time.asctime()).encode())
print('Script done, file is', filename)
```

fcntl— Thefcntl and ioctl system calls

This module performs file and I/O control on file descriptors. It is an interface to the `fcntl()` and `ioctl()` Unix routines. See the `fcntl(2)` and `ioctl(2)` Unix manual pages for full details.

Availability: Unix, not WASI.

All functions in this module take a file descriptor `fd` as their first argument. This can be an integer file descriptor, such as returned by `sys.stdin.fileno()`, or an `io.BaseObject`, such as `sys.stdin`, which provides a `fileno()` that returns a genuine file descriptor.

Changed in version 3.3: Operations in this module used to raise an `IOError` where they now raise an `OSError`.

Changed in version 3.8: The `fcntl` module now contains `F_ADD_SEALS`, `F_GET_SEALS`, and `F_SEAL_*` constants for sealing of `os.memfd_create()` file descriptors.

Changed in version 3.9: On macOS, the `fcntl` module exposes the `F_GETPATH` constant, which obtains the path of a file from a file descriptor. On Linux (≥ 3.15), the `fcntl` module exposes the `F_OFD_GETLK`, `F_OFD_SETLK` and `F_OFD_SETLKW` constants, which are used when working with open file description locks.

Changed in version 3.10: On Linux $\geq 2.6.11$, the `fcntl` module exposes the `F_GETPIPE_SZ` and `F_SETPIPE_SZ` constants, which allow to check and modify a pipe's size respectively.

Changed in version 3.11: On FreeBSD, the `fcntl` module exposes the `F_DUP2FD` and `F_DUP2FD_CLOEXEC` constants, which allow to duplicate a file descriptor, the latter setting `FD_CLOEXEC` flag in addition.

Changed in version 3.12: On Linux ≥ 4.5 , the `fcntl` module exposes the `FICLONE` and `FICLONERANGE` constants, which allow to share some data of one file with another file by reflinking on some filesystems (e.g., `btrfs`, `OCFS2`, and `XFS`). This behavior is commonly referred to as “copy-on-write”.

Changed in version 3.13: On Linux $\geq 2.6.32$, the `fcntl` module exposes the `F_GETOWN_EX`, `F_SETOWN_EX`, `F_OWNER_TID`, `F_OWNER_PID`, `F_OWNER_PGRP` constants, which allow to direct I/O availability signals to a specific thread, process, or process group. On Linux ≥ 4.13 , the `fcntl` module exposes the `F_GET_RW_HINT`, `F_SET_RW_HINT`, `F_GET_FILE_RW_HINT`, `F_SET_FILE_RW_HINT`, and `RWH_WRITE_LIFE_*` constants, which allow to inform the kernel about the relative expected lifetime of writes on a given inode or via a particular open file description. On Linux ≥ 5.1 and NetBSD, the `fcntl` module exposes the `F_SEAL_FUTURE_WRITE` constant for use with `F_ADD_SEALS` and `F_GET_SEALS` operations. On FreeBSD, the `fcntl` module exposes the `F_READAHEAD`, `F_ISUNIONSTACK`, and `F_KINFO` constants. On macOS and FreeBSD, the `fcntl` module exposes the `F_RDAHEAD` constant. On NetBSD and AIX, the `fcntl` module exposes the `F_CLOSEM` constant. On NetBSD, the `fcntl` module exposes the `F_MAXFD` constant. On macOS and NetBSD, the `fcntl` module exposes the `F_GETNOSIGPIPE` and `F_SETNOSIGPIPE` constant.

Changed in version 3.14: On Linux ≥ 6.1 , the `fcntl` module exposes the `F_DUPFD_QUERY` to query a file descriptor pointing to the same file.

The module defines the following functions:

fcntl.fcntl(*fd*, *cmd*, *arg*=0, *i*)

Perform the operation `cmd` on file descriptor `fd` (file objects providing a `fileno()` method are accepted as well). The values used for `cmd` are operating system dependent, and are available as constants in the `fcntl` module, using the same names as used in the relevant C header files. The argument `arg` can either be an integer value, a bytes-like object, or a string. The type and size of `arg` must match the type and size of the argument of the operation as specified in the relevant C documentation. When `arg` is an inte

fcntl.ioctl(fd,request,arg=0,mutate_flag=True,)

This function is identical to the `fcntl()` function, except that the argument handling is even more complicated. The `request` parameter is limited to values that can fit in 32-bits or 64-bits, depending on the platform. Additional constants of interest for use as the `request` argument can be found in the `termios` module, under the same names as used in the relevant C header files. The parameter `arg` can be an integer, a bytes-like object, or a string. The type and size of `arg` must match the type and size of the `arg`

fcntl.flock(fd,operation,)

Perform the lock operation `operation` on file descriptor `fd` (file objects providing a `fileno()` method are accepted as well). See the Unix manual `flock(2)` for details. (On some systems, this function is emulated using `fcntl()`.) If the `flock()` call fails, an `OSError` exception is raised. Raises an auditing event `fcntl.flock` with arguments `fd, operation`.

fcntl.lockf(fd,cmd,len=0,start=0,whence=0,)

fcntl.LOCK_UN

fcntl.LOCK_SH

fcntl.LOCK_EX

fcntl.LOCK_NB

This is essentially a wrapper around the `fcntl()` locking calls. `fd` is the file descriptor (file objects providing a `fileno()` method are accepted as well) of the file to lock or unlock, and `cmd` is one of the following values: `fcntl.LOCK_UN` Release an existing lock. `fcntl.LOCK_SH` Acquire a shared lock. `fcntl.LOCK_EX` Acquire an exclusive lock. `fcntl.LOCK_NB` Bitwise OR with any of the other three `LOCK_*` constants to make the request non-blocking. If `LOCK_NB` is used and the lock cannot be acquired, an `OSError` will be raised

Release an existing lock.

Acquire a shared lock.

Acquire an exclusive lock.

Bitwise OR with any of the other three `LOCK_*` constants to make the request non-blocking.

Examples (all on a SVR4 compliant system):

```
import struct, fcntl, os

f = open(...)
rv = fcntl.fcntl(f, fcntl.F_SETFL, os.O_NDELAY)

lockdata = struct.pack('hhllhh', fcntl.F_WRLCK, 0, 0, 0, 0, 0)
rv = fcntl.fcntl(f, fcntl.F_SETLKW, lockdata)
```

Note that in the first example the return value variable `rv` will hold an integer value; in the second example it will hold a bytes object. The structure lay-out for the `lockdata` variable is system dependent — therefore using the `flock()` call may be better.

See also

Module os

If the locking flags `O_SHLOCK` and `O_EXLOCK` are present in the `os` module (on BSD only), `os.open()` function provides an alternative to the `lockf()` and `flock()` functions.

resource— Resource usage information

This module provides basic mechanisms for measuring and controlling system resources utilized by a program.

Availability: Unix, not WASI.

Symbolic constants are used to specify particular system resources and to request usage information about either the current process or its children.

AnOSError is raised on syscall failure.

exception resource.error

A deprecated alias of `OSError`. Changed in version 3.3: Following PEP 3151, this class was made an alias of `OSError`.

Resource Limits

Resources usage can be limited using the `setrlimit()` function described below. Each resource is controlled by a pair of limits: a soft limit and a hard limit. The soft limit is the current limit, and may be lowered or raised by a process over time. The soft limit can never exceed the hard limit. The hard limit can be lowered to any value greater than the soft limit, but not raised. (Only processes with the effective UID of the super-user can raise a hard limit.)

The specific resources that can be limited are system dependent. They are described in the `getrlimit(2)` man page. The resources listed below are supported when the underlying operating system supports them; resources which cannot be checked or controlled by the operating system are not defined in this module for those platforms.

resource.RLIM_INFINITY

Constant used to represent the limit for an unlimited resource.

resource.getrlimit(resource)

Returns a tuple (soft, hard) with the current soft and hard limits of resource. Raises `ValueError` if an invalid resource is specified, or `error` if the underlying system call fails unexpectedly.

resource.setrlimit(resource, limits)

Sets new limits of consumption of resource. The `limits` argument must be a tuple (soft, hard) of two integers describing the new limits. A value of `RLIM_INFINITY` can be used to request a limit that is unlimited. Raises `ValueError` if an invalid resource is specified, if the new soft limit exceeds the hard limit, or if a process tries to raise its hard limit. Specifying a limit of `RLIM_INFINITY` when the hard or system limit for that resource is not unlimited will result in a `ValueError`. A process with the effective

resource.prlimit(pid, resource[, limits])

Combines `setrlimit()` and `getrlimit()` in one function and supports to get and set the resources limits of an arbitrary process. If `pid` is 0, then the call applies to the current process. `resource` and `limit` have the same meaning as in `setrlimit()`, except that `limits` is optional. When `limits` is not given the function returns the `resource` limit of the process `pid`. When `limits` is given the `resource` limit of the process is set and the former resource limit is returned. Raises `ProcessLookupError` when `pid` can't be found and `Permis`

These symbols define resources whose consumption can be controlled using the `setrlimit()` and `getrlimit()` functions described below. The values of these symbols are exactly the constants used by C programs.

The Unix man page `getrlimit(2)` lists the available resources. Note that not all systems use the same symbol or same value to denote the same resource. This module does not attempt to mask platform differences — symbols not defined for a platform will not be available from this module on that platform.

resource.RLIMIT_CORE

The maximum size (in bytes) of a core file that the current process can create. This may result in the creation of a partial core file if a larger core would be required to contain the entire process image.

resource.RLIMIT_CPU

The maximum amount of processor time (in seconds) that a process can use. If this limit is exceeded, `asigxcpusignal` is sent to the process. (See the `signal` module documentation for information about how to catch this signal and do something useful, e.g. flush open files to disk.)

resource.RLIMIT_FSIZE

The maximum size of a file which the process may create.

resource.RLIMIT_DATA

The maximum size (in bytes) of the process's heap.

resource.RLIMIT_STACK

The maximum size (in bytes) of the call stack for the current process. This only affects the stack of the main thread in a multi-threaded process.

resource.RLIMIT_RSS

The maximum resident set size that should be made available to the process.

resource.RLIMIT_NPROC

The maximum number of processes the current process may create.

resource.RLIMIT_NOFILE

The maximum number of open file descriptors for the current process.

resource.RLIMIT_OFILE

The BSD name for `RLIMIT_NOFILE`.

resource.RLIMIT_MEMLOCK

The maximum address space which may be locked in memory.

resource.RLIMIT_VMEM

The largest area of mapped memory which the process may occupy. Usually an alias of `RLIMIT_AS`. Availability: Solaris, FreeBSD, NetBSD.

resource.RLIMIT_AS

The maximum area (in bytes) of address space which may be taken by the process.

resource.RLIMIT_MSGQUEUE

The number of bytes that can be allocated for POSIX message queues. Availability: Linux $\geq$ 2.6.8. Added in version 3.4.

resource.RLIMIT_NICE

The ceiling for the process's nice level (calculated as `20 - rlim_cur`). Availability: Linux $\geq$ 2.6.12. Added in version 3.4.

resource.RLIMIT_RTPRIO

The ceiling of the real-time priority. Availability: Linux >= 2.6.12. Added in version 3.4.

resource.RLIMIT_RTIME

The time limit (in microseconds) on CPU time that a process can spend under real-time scheduling without making a blocking syscall. Availability: Linux >= 2.6.25. Added in version 3.4.

resource.RLIMIT_SIGPENDING

The number of signals which the process may queue. Availability: Linux >= 2.6.8. Added in version 3.4.

resource.RLIMIT_SBSIZE

The maximum size (in bytes) of socket buffer usage for this user. This limits the amount of network memory, and hence the amount of mbufs, that this user may hold at any time. Availability: FreeBSD, NetBSD. Added in version 3.4.

resource.RLIMIT_SWAP

The maximum size (in bytes) of the swap space that may be reserved or used by all of this user id's processes. This limit is enforced only if bit 1 of the vm.overcommit sysctl is set. Please see [tuning\(7\)](#) for a complete description of this sysctl. Availability: FreeBSD >= 8. Added in version 3.4.

resource.RLIMIT_NPTS

The maximum number of pseudo-terminals created by this user id. Availability: FreeBSD >= 8. Added in version 3.4.

resource.RLIMIT_KQUEUES

The maximum number of kqueues this user id is allowed to create. Availability: FreeBSD >= 11. Added in version 3.10.

Resource Usage

These functions are used to retrieve resource usage information:

resource.getrusage(who)

This function returns an object that describes the resources consumed by either the current process or its children, as specified by the `who` parameter. The `who` parameter should be specified using one of the `RUSAGE_*` constants described below. A simple example:

```
from resource import *
import time
# a non CPU-bound task
time.sleep(3)
print(getrusage(RUSAGE_SELF))
# a CPU-bound task
for i in range(10**8):
    _ = 1 + 1
print(getrusage(RUSAGE_SELF))
```

 The fields of the return value each describe how a particular system resource has

resource.getpagesize()

Returns the number of bytes in a system page. (This need not be the same as the hardware page size.)

The following `RUSAGE_*` symbols are passed to the `getrusage()` function to specify which processes information should be provided for.

resource.RUSAGE_SELF

Pass to `getrusage()` to request resources consumed by the calling process, which is the sum of resources used by all threads in the process.

resource.RUSAGE_CHILDREN

Pass `togetrusage()` to request resources consumed by child processes of the calling process which have been terminated and waited for.

`resource.RUSAGE_BOTH`

Pass `togetrusage()` to request resources consumed by both the current process and child processes. May not be available on all systems.

`resource.RUSAGE_THREAD`

Pass `togetrusage()` to request resources consumed by the current thread. May not be available on all systems. Added in version 3.2.

syslog— Unix syslog library routines

This module provides an interface to the Unix `syslog` library routines. Refer to the Unix manual pages for a detailed description of the `syslog` facility.

Availability: Unix, not WASI, not iOS.

This module wraps the `system` `syslog` family of routines. A pure Python library that can speak to a `syslog` server is available in the `logging.handlers` module as `ASysLogHandler`.

The module defines the following functions:

`syslog.syslog(message)`

`syslog.syslog(priority,message)`

Send the string `message` to the system logger. A trailing newline is added if necessary. Each message is tagged with a priority composed of a `facility` and a `level`. The optional `priority` argument, which defaults to `LOG_INFO`, determines the message priority. If the `facility` is not encoded in `priority` using logical-or (`LOG_INFO|LOG_USER`), the value given in the `openlog()` call is used. If `openlog()` has not been called prior to the call to `syslog()`, `openlog()` will be called with no arguments. Raises an `auditing` event

`syslog.openlog([ident[,logoption[,facility]]])`

Logging options of subsequent `syslog()` calls can be set by calling `openlog().syslog()` will call `openlog()` with no arguments if the log is not currently open. The optional `ident` keyword argument is a string which is prepended to every message, and defaults to `sys.argv[0]` with leading path components stripped. The optional `logoption` keyword argument (default is 0) is a bit field – see below for possible values to combine. The optional `facility` keyword argument (default is `LOG_USER`) sets the default facility for

`syslog.closelog()`

Reset the `syslog` module values and call the system library `closelog()`. This causes the module to behave as it does when initially imported. For example, `openlog()` will be called on the first `syslog()` call (if `openlog()` hasn't already been called), and `ident` and other `openlog()` parameters are reset to defaults. Raises an `auditing` event `syslog.closelog` with no arguments. Changed in version 3.12: This function is restricted in subinterpreters. (Only code that runs in multiple interpreters is affected and the restri

`syslog.setlogmask(maskpri)`

Set the priority mask to `maskpri` and return the previous mask value. Calls to `syslog()` with a priority level not set in `maskpri` are ignored. The default is to log all priorities. The function `LOG_MASK(pri)` calculates the mask for the individual priority `pri`. The function `LOG_UPTO(pri)` calculates the mask for all priorities up to and including `pri`. Raises an `auditing` event `syslog.setlogmask` with argument `maskpri`.

The module defines the following constants:

`syslog.LOG_EMERG`

`syslog.LOG_ALERT`

`syslog.LOG_CRIT`

`syslog.LOG_ERR`

`syslog.LOG_WARNING`

`syslog.LOG_NOTICE`

`syslog.LOG_INFO`

`syslog.LOG_DEBUG`

Priority levels (high to low).

syslog.LOG_AUTH

syslog.LOG_AUTHPRIV

syslog.LOG_CRON

syslog.LOG_DAEMON

syslog.LOG_FTP

syslog.LOG_INSTALL

syslog.LOG_KERN

syslog.LOG_LAUNCHD

syslog.LOG_LPR

syslog.LOG_MAIL

syslog.LOG_NETINFO

syslog.LOG_NEWS

syslog.LOG_RAS

syslog.LOG_REMOTEAUTH

syslog.LOG_SYSLOG

syslog.LOG_USER

syslog.LOG_UUCP

syslog.LOG_LOCAL0

syslog.LOG_LOCAL1

syslog.LOG_LOCAL2

syslog.LOG_LOCAL3

syslog.LOG_LOCAL4

syslog.LOG_LOCAL5

syslog.LOG_LOCAL6

syslog.LOG_LOCAL7

Facilities, depending on availability in<syslog.h>forLOG_AUTHPRIV,LOG_FTP,LOG_NETINFO,LOG_REMOTEAUTH,LOG_INSTALLandLOG_RAS.Changed in version 3.13:AddedLOG_FTP,LOG_NETINFO,LOG_REMOTEAUTH,LOG_INSTALL,LOG_RAS, andLOG_LAUNCHD.

syslog.LOG_PID

syslog.LOG_CONS

syslog.LOG_NDELAY

syslog.LOG_ODELAY

syslog.LOG_NOWAIT

syslog.LOG_PERROR

Log options, depending on availability in<syslog.h>forLOG_ODELAY,LOG_NOWAITandLOG_PERROR.

Examples

Simple example

A simple set of examples:

```
import syslog

syslog.syslog('Processing started')
if error:
    syslog.syslog(syslog.LOG_ERR, 'Processing started')
```

An example of setting some log options, these would include the process ID in logged messages, and write the messages to the destination facility used for mail logging:

```
syslog.openlog(logoption=syslog.LOG_PID, facility=syslog.LOG_MAIL)
syslog.syslog('E-mail processing initiated...')
```

Modules command-line interface (CLI)

The following modules have a command-line interface.

- ast
- asyncio
- base64
- calendar
- code
- compileall
- cProfile: see profile
- dis
- doctest
- encodings.rot_13
- ensurepip
- filecmp
- fileinput
- ftplib
- gzip
- http.server
- idlelib
- inspect
- json
- mimetypes
- pdb
- pickle
- pickletools
- platform
- poplib
- profile
- pstats
- py_compile
- pyclbr
- pydoc
- quopri
- random
- runpy

- `site`
- `sqlite3`
- `symtable`
- `sysconfig`
- `tabnanny`
- `tarfile`
- `this`
- `timeit`
- `tokenize`
- `trace`
- `turtledemo`
- `unittest`
- `uuid`
- `venv`
- `webbrowser`
- `zipapp`
- `zipfile`

See also the `Python` command-line interface.

Superseded Modules

The modules described in this chapter have been superseded by other modules for most use cases, and are retained primarily to preserve backwards compatibility.

Modules may appear in this chapter because they only cover a limited subset of a problem space, and a more generally applicable solution is available elsewhere in the standard library (for example, `getopt` covers the very specific task of “mimic the `Cgetopt()` API in Python”, rather than the broader command line option parsing and argument parsing capabilities offered by `optparse` and `argparse`).

Alternatively, modules may appear in this chapter because they are deprecated outright, and awaiting removal in a future release, or they are soft deprecated and their use is actively discouraged in new projects. With the removal of various obsolete modules through PEP 594, there are currently no modules in this latter category.

- `getopt`— C-style parser for command line options

getopt— C-style parser for command line options

Source code: `Lib/getopt.py`

■ **Nota:** Note This module is considered feature complete. A more declarative and extensible alternative to this API is provided in the `optparse` module. Further functional enhancements for command line parameter processing are provided either as third party modules on PyPI, or else as features in the `argparse` module.

This module helps scripts to parse the command line arguments in `sys.argv`. It supports the same conventions as the Unix `getopt()` function (including the special meanings of arguments of the form `'-'` and `'--'`). Long options similar to those supported by GNU software may be used as well via an optional third argument.

Users who are unfamiliar with the Unix `getopt()` function should consider using the `argparse` module instead. Users who are familiar with the Unix `getopt()` function, but would like to get equivalent behavior while writing less code and getting better help and error messages should consider using the `optparse` module. See [Choosing an argument parsing library](#) for additional details.

This module provides two functions and an exception:

getopt.getopt(args, shortopts, longopts=[])

Parses command line options and parameter list. `args` is the argument list to be parsed, without the leading reference to the running program. Typically, this means `sys.argv[1:]`. `shortopts` is the string of option letters that the script wants to recognize, with options that require an argument followed by a colon (':') and options that accept an optional argument followed by two colons (':: '); i.e., the same format that Unix `getopt()` uses. Note Unlike GNU `getopt()`, after a non-option argument, all further

getopt.gnu_getopt(args, shortopts, longopts=[])

This function works like `getopt()`, except that GNU style scanning mode is used by default. This means that option and non-option arguments may be intermixed. The `getopt()` function stops processing options as soon as a non-option argument is encountered. If the first character of the option string is '+', or if the environment variable `POSIXLY_CORRECT` is set, then option processing stops as soon as a non-option argument is encountered. If the first character of the option string is '-', non-option argumen

exception getopt.GetoptError

This is raised when an unrecognized option is found in the argument list or when an option requiring an argument is given none. The argument to the exception is a string indicating the cause of the error. For long options, an argument given to an option which does not require one will also cause this exception to be raised. The attributes `msg` and `opt` give the error message and related option; if there is no specific option to which the exception relates, `opt` is an empty string.

exception getopt.error

Alias for `GetoptError`; for backward compatibility.

An example using only Unix style options:

```
>>> import getopt
>>> args = '-a -b -cfoo -d bar a1 a2'.split()
>>> args
['-a', '-b', '-cfoo', '-d', 'bar', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'abc:d:')
>>> optlist
[('-a', ''), ('-b', ''), ('-c', 'foo'), ('-d', 'bar')]
>>> args
['a1', 'a2']
```

Using long option names is equally easy:

```
>>> s = '--condition=foo --testing --output-file abc.def -x a1 a2'
>>> args = s.split()
>>> args
['--condition=foo', '--testing', '--output-file', 'abc.def', '-x', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'x', [
...     'condition=', 'output-file=', 'testing'])
>>> optlist
[('--condition', 'foo'), ('--testing', ''), ('--output-file', 'abc.def'), ('-x', '')]
>>> args
['a1', 'a2']
```

Optional arguments should be specified explicitly:

```
>>> s = '-Con -C --color=off --color a1 a2'
>>> args = s.split()
>>> args
['-Con', '-C', '--color=off', '--color', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'C::', ['color=?'])
>>> optlist
[('-C', 'on'), ('-C', ''), ('--color', 'off'), ('--color', '')]
>>> args
['a1', 'a2']
```

The order of options and non-option arguments can be preserved:

```
>>> s = 'a1 -x a2 a3 a4 --long a5 a6'
>>> args = s.split()
>>> args
['a1', '-x', 'a2', 'a3', 'a4', '--long', 'a5', 'a6']
>>> optlist, args = getopt.gnu_getopt(args, '-x:', ['long='])
>>> optlist
[(None, ['a1']), ('-x', 'a2'), (None, ['a3', 'a4']), ('--long', 'a5')]
>>> args
['a6']
```

In a script, typical usage is something like this:

```
import getopt, sys

def main():
    try:
        opts, args = getopt.getopt(sys.argv[1:], "ho:v", ["help", "output="])
    except getopt.GetoptError as err:
        # print help information and exit:
        print(err) # will print something like "option -a not recognized"
        usage()
        sys.exit(2)
    output = None
    verbose = False
    for o, a in opts:
        if o == "-v":
            verbose = True
        elif o in ("-h", "--help"):
            usage()
            sys.exit()
        elif o in ("-o", "--output"):
            output = a
        else:
            assert False, "unhandled option"
    process(args, output=output, verbose=verbose)

if __name__ == "__main__":
    main()
```

Note that an equivalent command line interface could be produced with less code and more informative help and error messages by using the `optparse` module:

```
import optparse

if __name__ == '__main__':
    parser = optparse.OptionParser()
    parser.add_option('-o', '--output')
    parser.add_option('-v', dest='verbose', action='store_true')
    opts, args = parser.parse_args()
    process(args, output=opts.output, verbose=opts.verbose)
```

A roughly equivalent command line interface for this case can also be produced by using the `argparse` module:

```
import argparse

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-o', '--output')
    parser.add_argument('-v', dest='verbose', action='store_true')
    parser.add_argument('rest', nargs='*')
    args = parser.parse_args()
    process(args.rest, output=args.output, verbose=args.verbose)
```

See [Choosing an argument parsing library](#) for details on how the `argparse` version of this code differs in behaviour from the `optparse` (and `getopt`) version.

See also

Module `optparse`

Module `argparse`

Declarative command line option parsing.

More opinionated command line option and argument parsing library.

Removed Modules

The modules described in this chapter have been removed from the Python standard library. They are documented here to help people find replacements.

- `aifc`— Read and write AIFF and AIFC files
- `asynchat`— Asynchronous socket command/response handler
- `asyncore`— Asynchronous socket handler
- `audioop`— Manipulate raw audio data
- `cgi`— Common Gateway Interface support
- `cgitb`— Traceback manager for CGI scripts
- `chunk`— Read IFF chunked data
- `crypt`— Function to check Unix passwords
- `distutils`— Building and installing Python modules
- `imghdr`— Determine the type of an image
- `imp`— Access the import internals
- `mailcap`— Mailcap file handling
- `msilib`— Read and write Microsoft Installer files
- `nis`— Interface to Sun's NIS (Yellow Pages)
- `nntplib`— NNTP protocol client
- `ossaudiodev`— Access to OSS-compatible audio devices
- `pipes`— Interface to shell pipelines
- `smtpd`— SMTP Server
- `sndhdr`— Determine type of sound file
- `spwd`— The shadow password database
- `sunau`— Read and write Sun AU files
- `telnetlib`— Telnet client
- `uu`— Encode and decode uuencode files
- `xdrlib`— Encode and decode XDR data

Security Considerations

The following modules have specific security considerations:

- `base64`: `base64` security considerations in RFC 4648
- `hashlib`: all constructors take a “`usedforsecurity`” keyword-only argument disabling known insecure and blocked algorithms
- `http.server`: is not suitable for production use, only implementing basic security checks. See these security considerations.
- `logging`: Logging configuration uses `eval()`
- `multiprocessing`: `Connection.recv()` uses `pickle`
- `pickle`: Restricting globals in `pickle`
- `random`: shouldn't be used for security purposes, use `secrets` instead
- `shelve`: `shelve` is based on `pickle` and thus unsuitable for dealing with untrusted sources
- `ssl`: SSL/TLS security considerations
- `subprocess`: Subprocess security considerations
- `tempfile`: `mktemp` is deprecated due to vulnerability to race conditions
- `xml`: XML security
- `zipfile`: maliciously prepared `.zip` files can cause disk volume exhaustion

The `-I` command line option can be used to run Python in isolated mode. When it cannot be used, the `-P` option or the `PYTHONSAFEPATH` environment variable can be used to not prepend a potentially unsafe path to `sys.path` such as the current directory, the script's directory or an empty string.